



LAUREA MAGISTRALE
UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di **Informatica, Sistemistica e Comunicazione**

Laurea magistrale in **informatica**

Sviluppo di un sistema RAG aziendale e valutazione di LLM locali

Francesco Romeo
Matricola: 885880

Relatrice: **Dott.ssa Elisabetta Fersini**

Co-relatore: **Dott. Marco Loregian**

Anno Accademico **2025/2026**

Sommario

Questa tesi presenta la progettazione, l'implementazione e la valutazione di un sistema *Retrieval-Augmented Generation* integrato in i3Guild, la piattaforma usata da Intré S.r.l. per gestire le Gilde aziendali. Il problema affrontato riguarda la consultazione di un corpus interno eterogeneo, composto da descrizioni, obiettivi, risultati, partecipanti e metadati, che contiene conoscenza utile ma non sempre facilmente recuperabile tramite strumenti tradizionali.

Il lavoro non introduce un nuovo modello linguistico, ma studia come integrare componenti esistenti in un prototipo enterprise on-premise, coerente con vincoli di riservatezza, risorse hardware locali e manutenibilità. L'architettura proposta separa la sorgente dati, la pipeline di ingestione, l'indice vettoriale, il microservizio RAG e l'integrazione applicativa. Il sistema usa PostgreSQL come sorgente autoritativa, una pipeline dedicata per normalizzare i contenuti e produrre embedding, Qdrant per il retrieval denso, sparso e ibrido, Haystack per l'orchestrazione, FastAPI per esporre il servizio RAG, Ollama per l'inferenza locale e Next.js per l'integrazione con l'applicazione esistente.

La tesi descrive inoltre le scelte adottate per rendere il retrieval più aderente al dominio: query rewriting, filtri strutturati, deduplicazione, soglie di similarità, selezione controllata dei documenti e generazione in streaming. Un workflow agentic per la proposta di nuove Gilde è stato esplorato e testato, ma non adottato come nucleo della soluzione finale, poiché ha mostrato un costo computazionale e operativo superiore rispetto a un RAG più deterministico.

La seconda parte del lavoro valuta la sostenibilità dell'inferenza locale su Apple Silicon. L'esperimento confronta modelli open-weight tra 3B e 14B parametri in diverse varianti di quantizzazione MLX, misurando dimensione su disco, memoria di picco, tempo di caricamento, time-to-first-token, throughput di decodifica e qualità su subset ridotti MMLU e GSM8K. I risultati indicano che, sui modelli piccoli, la quantizzazione Q4 offre il compromesso più interessante tra memoria, velocità e qualità osservata; sui modelli medi e grandi, invece, la quantizzazione è soprattutto una condizione di fattibilità entro il vincolo dei 16 GB di memoria unificata.

Nel complesso, il lavoro mostra che un sistema RAG locale per un contesto aziendale è praticabile, ma richiede controllo simultaneo di corpus, indice, retrieval, prompt, modello generativo e risorse hardware. Il contributo principale della tesi è quindi un prototipo integrato e valutato in condizioni realistiche, insieme a un'analisi dei compromessi necessari per usare LLM locali in un flusso applicativo aziendale.

Indice

Sommario	i
Introduzione	v
1 Fondamenti Tecnologici del Progetto RAG Aziendale	1
1.1 Architetture Transformer	1
1.1.1 Dalle reti neurali ricorrenti ai Transformer	1
1.1.2 Il meccanismo di Self-Attention	2
1.1.3 Architettura encoder-decoder e varianti decoder-only	2
1.1.4 Limiti in contesto zero-shot e motivazione del RAG	2
1.2 LLM: panorama e analisi comparativa	3
1.2.1 Modelli proprietari	3
1.2.2 Motivazioni per l'adozione di modelli locali	4
1.2.3 Modelli open-weight per uso on-premise	4
1.2.4 Inferenza locale nel prototipo aziendale	5
1.2.5 Ollama: server di inferenza locale	5
1.3 Retrieval-Augmented Generation (RAG)	6
1.3.1 Motivazione e definizione	6
1.3.2 Architettura del sistema RAG	6
1.3.3 Modelli di embedding	8
1.3.4 Database vettoriali	8
1.3.5 Agentic RAG	8
1.3.6 Tecniche di adattamento al dominio	9
1.4 Framework di orchestrazione: Haystack	10
1.5 Sintesi e posizionamento del lavoro	10
2 Analisi dei Requisiti e Progettazione del Sistema	11
2.1 Il contesto aziendale: Intré S.r.l.	11
2.1.1 Il modello delle Gilde	12
2.2 Caso d'uso: Proposta di nuove Gilde Intré	12
2.2.1 Motivazione del caso d'uso	13
2.2.2 Obiettivi del caso d'uso	13
2.2.3 Descrizione dei dati e analisi del corpus	13
2.2.4 Struttura informativa delle Gilde e implicazioni per il retrieval	14
2.3 Requisiti di sistema	15

2.3.1	Vincoli architetturali	16
2.4	Scelte tecnologiche: analisi comparativa	16
2.4.1	Framework di orchestrazione RAG	17
2.4.2	Database vettoriale	18
2.4.3	Modello di embedding	19
2.4.4	Server di inferenza LLM e selezione del modello di chat	20
2.5	Proposta architetturale	20
2.5.1	Componenti principali	21
2.5.2	Progettazione della collection Qdrant	22
2.5.3	Pipeline di ingestione ETL	23
2.5.4	Pipeline di retrieval e logica di ranking	23
2.5.5	Endpoint API di riferimento	24
2.6	Integrazione con i3Guild	25
2.6.1	Servizi Docker aggiuntivi	25
2.6.2	Variabili d'ambiente	25
2.6.3	Integrazione lato Next.js	25
2.6.4	Sincronizzazione dei dati	26
2.7	Sintesi del capitolo	26
3	Implementazione del Sistema RAG	27
3.1	Struttura dei repository e responsabilità	27
3.2	Pipeline di ingestione	28
3.2.1	Configurazione e controlli preliminari	28
3.2.2	Estrazione dei dati da PostgreSQL	29
3.2.3	Normalizzazione e costruzione dei documenti	29
3.2.4	Embedding e scrittura in Qdrant	30
3.3	Microservizio RAG	30
3.3.1	Avvio, middleware e gestione errori	31
3.3.2	Endpoint di retrieval	31
3.3.3	Pipeline Haystack di query	31
3.4	Logica di retrieval	31
3.4.1	Riscrittura della query e filtri	32
3.4.2	Cache e deduplicazione	32
3.4.3	Selezione dinamica dei documenti	32
3.5	Generazione con Ollama	32
3.5.1	Risoluzione del modello	32
3.5.2	Costruzione del prompt aumentato	33
3.5.3	Streaming NDJSON	33
3.6	Workflow agentico	33
3.7	Integrazione lato Next.js	34
3.7.1	Client di retrieval	34
3.7.2	Client chat e generazione controllata	34
3.8	Generazione della bozza di Gilda	34
3.9	Aspetti operativi	35
3.9.1	Timeout e dipendenze lente	35
3.9.2	System prompt e modello derivato	35

3.9.3	Tracciabilità	36
3.10	Sintesi del capitolo	36
4	Disegno sperimentale per l’inferenza locale quantizzata	37
4.1	Motivazione e perimetro	37
4.2	Riferimenti scientifici e tecnici	38
4.3	Hardware e runtime	39
4.3.1	Apple Silicon come system-on-chip	39
4.3.2	Memoria unificata	39
4.3.3	GPU, Metal e runtime MLX	40
4.3.4	Implicazioni per l’inferenza LLM locale	40
4.4	Matrice dei modelli	41
4.5	Artefatti e politica di quantizzazione	41
4.6	Benchmark prestazionale	42
4.7	Struttura dei run e aggregazione	42
4.8	Benchmark di qualità	43
4.9	Riproducibilità	44
4.10	Gestione degli artefatti e dei fallimenti	44
4.11	Ipotesi sperimentali	44
4.12	Minacce alla validità	45
4.13	Sintesi del capitolo	45
5	Valutazione sperimentale e analisi dei risultati	46
5.1	Dataset dei risultati	46
5.2	Risultati sui modelli piccoli	47
5.3	Risultati sui modelli medi e grandi	48
5.4	Effetto della lunghezza del prompt	49
5.5	Stabilità delle misure	50
5.6	Qualità su MMLU ridotto	51
5.6.1	Analisi dei casi a punteggio nullo su MMLU	52
5.7	Qualità su GSM8K ridotto	52
5.7.1	Analisi degli errori GSM8K	53
5.8	Qualità per GiB e trade-off pratico	53
5.9	Efficienza prestazionale per GiB	53
5.10	Criteri di lettura delle metriche	54
5.11	Analisi degli errori di output e del parsing	56
5.12	Discussione complessiva	56
5.13	Profili operativi consigliati	57
5.14	Implicazioni per un sistema RAG locale	58
5.15	Limiti dell’analisi	58
	Conclusioni e Sviluppi Futuri	60
A	Annotazioni e Convenzioni	63
A.1	Repository e componenti	63
A.2	Formato dei documenti indicizzati	63
A.3	Eventi NDJSON	63

B Codice Utilizzato	65
B.1 Listati richiamati nel Capitolo 2	65
B.2 Listati richiamati nel Capitolo 3: pipeline di ingestione	66
B.3 Listati richiamati nel Capitolo 3: retrieval e generazione	68
B.4 Listati richiamati nel Capitolo 3: frontend e generazione bozza	69
Bibliografia	71
Ringraziamenti	75

Introduzione

I Large Language Model hanno reso più accessibile l'interazione con grandi quantità di testo: posso sintetizzare documenti, formulare risposte in linguaggio naturale e assistere attività di scrittura o analisi. In ambito aziendale, però, queste capacità non bastano. Una risposta utile deve riferirsi a dati interni aggiornati, deve essere verificabile e deve rispettare vincoli di riservatezza, controllo dell'infrastruttura e manutenibilità. Un modello pre-addestrato, usato da solo, non conosce la documentazione privata di un'organizzazione e non possiede un meccanismo nativo per controllarla. Il problema emerge soprattutto quando il sistema deve rispondere su contenuti specifici dell'azienda, non su conoscenza generale.

Il paradigma *Retrieval-Augmented Generation* (RAG) affronta questo problema combinando due componenti: un modulo di recupero, che seleziona documenti pertinenti da una base di conoscenza esterna, e un modello generativo, che produce la risposta a partire dalla domanda e dal contesto recuperato [23]. In questo modo la conoscenza usata dal sistema non risiede soltanto nei parametri del modello, ma anche in un indice aggiornabile e ispezionabile. Per scenari enterprise, tale impostazione è interessante perché consente di integrare fonti interne, mantenere traccia dei documenti usati e separare il problema della generazione dal problema della gestione della conoscenza.

Questa tesi nasce all'interno di Intré S.r.l., una software factory italiana che adotta pratiche strutturate di apprendimento organizzativo. Tra queste, le Gilde rappresentano cicli periodici di apprendimento di gruppo: le persone propongono temi, formano gruppi di lavoro, producono risultati e condividono l'esperienza con il resto dell'azienda. Nel tempo, la piattaforma interna i3Guild ha raccolto proposte, descrizioni, partecipanti, risultati e materiali collegati. Tale corpus costituisce una memoria organizzativa di valore, ma non sempre facile da interrogare: le informazioni sono distribuite tra database, metadati e contenuti testuali, e il recupero manuale richiede conoscenza pregressa del dominio.

L'obiettivo principale della tesi è progettare, implementare e valutare un sistema RAG integrato in i3Guild, capace di rendere interrogabile il corpus delle Gilde tramite linguaggio naturale. Il sistema deve rispettare tre vincoli di fondo. Il primo è la riservatezza: i dati aziendali devono restare entro un perimetro controllato, in coerenza con le esigenze di protezione del dato e con il quadro normativo europeo [14]. Il secondo è la sostenibilità infrastrutturale: il prototipo deve poter funzionare con risorse realistiche, senza assumere la disponibilità di GPU server dedicate. Il terzo è la manutenibilità: la soluzione deve integrarsi con l'applicazione esistente e conservarne la separazione tra sorgente dati, indicizzazione, retrieval e generazione.

Il contributo della tesi non consiste nell'addestramento di un nuovo modello linguistico. Il lavoro si concentra invece sull'integrazione di componenti esistenti in un prototipo enterprise on-premise: PostgreSQL come sorgente autoritativa, una pipeline di ingestione per normalizzare i dati e produrre embedding, Qdrant come database vettoriale, Haystack come framework di orchestrazione,

FastAPI come microservizio RAG, Ollama come server di inferenza locale e Next.js come livello applicativo di integrazione. La parte implementativa include inoltre query rewriting, filtri strutturati, deduplicazione, selezione dinamica dei documenti, streaming della risposta e generazione controllata di bozze di nuove Gilde.

Durante lo sviluppo e' stato esplorato anche un workflow agentico per guidare l'utente nella proposta di nuove Gilde. Tale workflow e' stato implementato e testato, ma non adottato come nucleo della soluzione finale. La ragione e' pratica: a parita' di hardware locale, l'approccio agentico introduce piu' passaggi, maggiore costo computazionale e maggiore complessita' operativa. Nel perimetro del prototipo, un RAG piu' classico e deterministico e' risultato piu' stabile e piu' semplice da mantenere. La componente agentica rimane quindi documentata come sperimentazione e possibile sviluppo futuro.

La seconda parte del lavoro affronta un problema emerso dal prototipo: la sostenibilita' dell'inferenza locale. Un sistema RAG aziendale puo' essere correttamente progettato, ma la sua utilita' pratica dipende anche dal modello generativo usato nella fase finale. Il modello deve essere caricabile, deve rientrare nella memoria disponibile, deve rispondere con una latenza accettabile e deve mantenere un livello qualitativo compatibile con il caso d'uso. Per questo la tesi include un capitolo sperimentale dedicato all'inferenza locale quantizzata su Apple Silicon, usando MLX e modelli open-weight tra 3B e 14B parametri.

L'esperimento misura il compromesso introdotto dalla quantizzazione MLX su un MacBook Pro con chip M1 Pro e 16 GB di memoria unificata. Le varianti FP16, Q8, Q6 e Q4 vengono confrontate rispetto a dimensione su disco, memoria di picco, tempo di caricamento, time-to-first-token, throughput di decodifica e qualita' su subset ridotti MMLU e GSM8K. I risultati mostrano che, per i modelli piccoli, Q4 riduce la dimensione di circa il 72%, riduce il picco di memoria di circa 67-70% e aumenta il throughput di decodifica di circa 2.6-2.8 volte rispetto a FP16, senza mostrare un degrado macroscopico nei controlli automatici ridotti. Per i modelli medi e grandi, la quantizzazione e' soprattutto una leva di fattibilita': rende eseguibili configurazioni che sarebbero meno realistiche nel vincolo dei 16 GB, ma richiede una valutazione attenta di latenza, formato dell'output e uso concorrente con gli altri servizi del sistema.

La tesi e' organizzata come segue. Il Capitolo 1 introduce i fondamenti tecnologici: Transformer, LLM, inferenza locale, RAG, embedding, database vettoriali, Agentic RAG e Haystack. Il Capitolo 2 analizza il contesto aziendale di Intré, il modello delle Gilde, il caso d'uso, i requisiti e le scelte architetturali. Il Capitolo 3 descrive l'implementazione del sistema: pipeline di ingestione, microservizio RAG, retrieval, generazione locale, workflow agentico sperimentale e integrazione Next.js. Il Capitolo 4 definisce il protocollo sperimentale per l'inferenza locale quantizzata su Apple Silicon. Il Capitolo 5 presenta e discute i risultati, collegandoli alle implicazioni operative per un sistema RAG locale.

Capitolo 1

Fondamenti Tecnologici del Progetto RAG Aziendale

Il sistema sviluppato durante il progetto aziendale nasce da un'esigenza applicativa precisa: rendere interrogabile la conoscenza prodotta in i3Guild senza trasferire dati interni verso servizi esterni. Prima di descrivere il caso d'uso, i requisiti e l'implementazione, è quindi necessario introdurre i concetti tecnici minimi su cui si fonda la soluzione: modelli linguistici generativi, inferenza locale, *Retrieval-Augmented Generation* (RAG), embedding, database vettoriali e orchestrazione della pipeline.

Il capitolo non intende esaurire lo stato dell'arte sull'inferenza locale o sulla quantizzazione, temi che saranno ripresi in modo specifico nei Capitoli 4 e 5. In questa sede vengono introdotti solo gli elementi necessari a leggere il prototipo realizzato in azienda e a comprendere le decisioni progettuali discusse nei capitoli successivi.

1.1 Architetture Transformer

1.1.1 Dalle reti neurali ricorrenti ai Transformer

Il modello Transformer, introdotto da Vaswani et al. nel 2017 [33], costituisce il punto di svolta da cui derivano gli attuali modelli linguistici. Prima della sua introduzione, gran parte dei sistemi di trattamento del linguaggio naturale (*Natural Language Processing*, NLP) si basava su reti neurali ricorrenti (*Recurrent Neural Networks*, RNN) e, in particolare, su architetture LSTM (*Long Short-Term Memory*) [16]. Tali modelli elaboravano la sequenza un elemento alla volta, aggiornando progressivamente uno stato nascosto. Questa impostazione era coerente con la natura ordinata del testo, ma introduceva due limiti rilevanti: la scarsa parallelizzabilità dell'addestramento e la difficoltà nel preservare dipendenze informative tra elementi molto distanti nella sequenza (*vanishing gradient*).

Il Transformer affronta tali limiti sostituendo la ricorrenza con un meccanismo di *self-attention* globale, che mette ogni token in relazione diretta con tutti gli altri token della sequenza, indipendentemente dalla loro distanza posizionale. Da questa impostazione derivano sia la maggiore efficienza in addestramento sia la capacità dei modelli successivi di costruire rappresentazioni contestuali più ricche.

1.1.2 Il meccanismo di Self-Attention

Il nucleo computazionale del Transformer è il meccanismo di *attention* scalato. Data una sequenza di token rappresentati come vettori di dimensione d_{model} , si calcolano tre proiezioni lineari per ogni elemento: la query $\mathbf{Q}$, la chiave $\mathbf{K}$ e il valore $\mathbf{V}$. Il punteggio di attenzione [33] è definito come:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (1.1)$$

dove d_k è la dimensione dei vettori di chiave; il fattore $1/\sqrt{d_k}$ previene la saturazione della funzione softmax in presenza di prodotti scalari molto grandi.

Il medesimo calcolo viene poi replicato su più *teste* di attenzione (*attention heads*). Ciascuna testa apprende una proiezione diversa dello spazio di input e può quindi specializzarsi su relazioni differenti, ad esempio sintattiche, semantiche o posizionali [33]. La concatenazione degli output di h teste definisce il meccanismo di *Multi-Head Attention*:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (1.2)$$

con $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

La posizione dei token nella sequenza non è catturata dall'attention; viene iniettata tramite *positional encoding*, ossia vettori sommati agli embedding di input che forniscono informazioni sull'ordine relativo degli elementi.

1.1.3 Architettura encoder-decoder e varianti decoder-only

L'architettura originale del Transformer è di tipo encoder-decoder. L'encoder trasforma la sequenza di input in una rappresentazione contestuale densa; il decoder utilizza tale rappresentazione, tramite *cross-attention*, per produrre la sequenza di output. Questa struttura è particolarmente adatta a compiti *sequence-to-sequence*, come la traduzione automatica [33].

I modelli linguistici generativi contemporanei adottano prevalentemente un'architettura *decoder-only*, nella quale l'attenzione è applicata in modo *causale* (o *masked*): ogni token dipende esclusivamente dai precedenti. Questo vincolo è necessario per l'addestramento autoregressivo, nel quale il modello impara a predire il token successivo dato il contesto.

1.1.4 Limiti in contesto zero-shot e motivazione del RAG

La capacità dei Transformer di modellare il linguaggio non elimina tuttavia un problema centrale per l'uso aziendale: un LLM pre-addestrato non conosce, per definizione, i dati interni prodotti dopo l'addestramento e non dispone di un meccanismo nativo per verificarli. In un dominio come quello considerato in questa tesi, dove le risposte devono riferirsi a documenti aziendali specifici, questa caratteristica diventa un limite operativo. La Tabella 1.1 riassume le criticità principali e le relative strategie di mitigazione.

Da questa analisi deriva la scelta di privilegiare il paradigma RAG, discusso nella Sezione 1.3. Rispetto al fine-tuning, RAG permette di aggiornare la conoscenza disponibile intervenendo sul corpus documentale e non sui pesi del modello; per questo risulta più adatto a basi di conoscenza aziendali soggette a modifiche frequenti.

Tabella 1.1: Limiti strutturali degli LLM in contesti aziendali e relative strategie di mitigazione.

Limitazione	Descrizione	Strategia di mitigazione
Conoscenza statica	I parametri codificano la conoscenza fino al <i>knowledge cutoff</i> ; nessun accesso a dati interni o aggiornati dopo l'addestramento.	RAG: recupero dinamico da knowledge base esterna, aggiornabile senza riaddestrare il modello.
Allucinazioni [21]	Il modello può generare testo plausibile ma fattualmente errato su informazioni assenti dalla distribuzione di addestramento.	RAG con <i>grounding</i> : la risposta è ancorata a documenti recuperati e verificabili.
Privacy dei dati	Il fine-tuning su modelli cloud può esporre dati riservati a infrastrutture di terze parti, con rischi di conformità normativa.	Esecuzione on-premise con modelli open-weight: nessuna trasmissione di dati verso servizi esterni.

1.2 LLM: panorama e analisi comparativa

Una volta chiariti i limiti dei modelli generativi, occorre distinguere tra le famiglie di LLM disponibili. A partire dal 2022 l'ecosistema si è diviso lungo una linea rilevante per questa tesi: da un lato i modelli proprietari, accessibili prevalentemente tramite API cloud; dall'altro i modelli open-weight, i cui pesi possono essere scaricati ed eseguiti su infrastruttura locale, nel rispetto delle relative licenze. La Tabella 1.2 offre una panoramica dei modelli considerati come riferimento tecnologico.

Tabella 1.2: Panorama comparativo dei principali LLM rilevanti per il progetto (cutoff: 1 maggio 2026). [†] Esecuzione on-premise senza dipendenze cloud; [‡] pesi scaricabili pubblicamente; MoE = *Mixture of Experts*.

Modello	Sviluppatore	Licenza	Accesso	Architettura / Param. attivi	Ctx	Multi-modale
GPT-5.x	OpenAI	Proprietaria	API cloud	Dense, n.d.	400k–1M	✓
Claude 4.x	Anthropic	Proprietaria	API cloud	Dense, n.d.	200k	✓
Gemini 3.x	Google DM	Proprietaria	API cloud	Dense, n.d.	fino a 1M	✓
Gemma 3 4B	Google	Gemma terms [‡]	Self-hosted [†]	Dense, 4B	128k	✓
Qwen3 4B/8B	Alibaba Cloud	Apache 2.0 [‡]	Self-hosted [†]	Dense, 4–8B	128k	–
Llama 3.2 3B	Meta AI	Open-weight [‡]	Self-hosted [†]	Dense, 3B	128k	–
Mistral 7B v0.3	Mistral AI	Apache 2.0 [‡]	Self-hosted [†]	Dense, 7B	32k	–
Phi-3.5 mini	Microsoft	MIT [‡]	Self-hosted [†]	Dense, 3.8B	128k	–

1.2.1 Modelli proprietari

I modelli proprietari offrono prestazioni elevate sui benchmark generali, ma l'adozione in contesti con dati riservati richiede una valutazione separata del perimetro contrattuale e tecnico. L'uso tramite API pubbliche non è equivalente all'uso tramite servizi enterprise: Azure OpenAI Service,

Amazon Bedrock per modelli Anthropic e piattaforme cloud analoghe possono prevedere accordi di trattamento dati, isolamento logico, regioni controllate e opzioni di mancata conservazione dei prompt [61, 41]. Queste garanzie riducono il rischio rispetto a un accesso consumer, ma non eliminano la dipendenza da infrastrutture esterne. Nel progetto i3Guild, tale dipendenza resta incompatibile con il vincolo architetturale scelto: mantenere dati, retrieval e generazione entro un ambiente controllato.

GPT-5.x e OpenAI o-series. La famiglia GPT-5.x [61] rappresenta un riferimento recente per pianificazione complessa, sviluppo software e integrazione multimodale. La serie "o" rimane orientata a compiti che richiedono ragionamento deliberato, con miglioramenti osservati in ambiti scientifici e logico-matematici [60]. L'erogazione avviene tramite API cloud proprietarie o servizi enterprise collegati.

Claude 4.x (Anthropic). I modelli Claude adottano approcci come la *Constitutional AI* [3], un protocollo di addestramento guidato da principi per migliorare l'allineamento. Anche questi modelli sono erogati in modalità cloud; Anthropic documenta l'uso dei propri modelli anche tramite Amazon Bedrock e altre piattaforme enterprise, con opzioni di routing e gestione dati adatte a scenari regolati [41].

Gemini (Google DeepMind). La famiglia Gemini [53] è caratterizzata da multimodalità nativa e da finestre di contesto molto estese. Anche in questo caso l'accesso è fornito tramite servizi cloud.

1.2.2 Motivazioni per l'adozione di modelli locali

Nel caso in esame, le prestazioni assolute non sono l'unico criterio di scelta. Il sistema descritto in questa tesi deve poter operare su dati aziendali interni e deve quindi privilegiare controllo, tracciabilità e assenza di dipendenze da servizi esterni. Per questo l'architettura utilizza modelli eseguiti interamente in locale (*self-hosted*). La scelta si fonda su tre motivazioni, sintetizzate nella Tabella 1.3.

Per queste ragioni, i modelli proprietari sono mantenuti come riferimento comparativo, mentre l'analisi progettuale si concentra sulle soluzioni eseguibili localmente. Tale impostazione rende coerente la scelta del modello con i vincoli di privacy, costo e governo dell'infrastruttura discussi nei capitoli successivi.

1.2.3 Modelli open-weight per uso on-premise

Gemma 3 (Google). Gemma 3 [54] è una famiglia open-weight pensata anche per l'esecuzione locale. La variante da 4B parametri è particolarmente rilevante per questa tesi: mantiene un footprint compatibile con hardware aziendale non dedicato e coincide con il modello adottato nel prototipo i3Guild tramite Ollama.

Qwen3 (Alibaba Cloud). La generazione Qwen3 [70] copre taglie diverse, dai modelli compatti fino a varianti più grandi e MoE. Le taglie 4B e 8B sono interessanti per l'esecuzione locale perché offrono un compromesso tra capacità generativa, finestra di contesto e requisiti di memoria. Nei test sperimentali dei Capitoli 4 e 5, Qwen3 è incluso proprio per valutare questo equilibrio.

Tabella 1.3: Motivazioni per l'adozione di modelli LLM on-premise.

Motivazione	Descrizione	Riferimento normativo / letteratura
Sovranità del dato e conformità	L'esecuzione locale evita la trasmissione di dati verso infrastrutture di terze parti, anche in presenza di garanzie contrattuali (ZDR, DPA).	GDPR [13]; rischi del cloud nella gestione di dati sensibili.
Costi operativi nel lungo periodo	Il pricing a consumo dei provider cloud può risultare più oneroso rispetto all'ammortamento dell'hardware locale in scenari ad alto volume.	[7]; rischio di <i>vendor lock-in</i> .
Controllo e personalizzazione	I modelli open-weight (LLaMA, Mistral, Qwen) consentono fine-tuning supervisionato e allineamento su dati propri, senza le restrizioni delle policy dei sistemi proprietari.	[32, 22, 27]

Llama 3.2, Mistral 7B e Phi-3.5 mini. Accanto a Gemma e Qwen, il panorama locale comprende modelli compatti come Llama 3.2 3B, Mistral 7B Instruct v0.3 e Phi-3.5 mini. Queste famiglie non sono equivalenti per licenza, lingua, template di chat e comportamento di output, ma appartengono alla fascia più realistica per una macchina aziendale senza GPU server dedicata. Per questo compaiono nella matrice sperimentale successiva: non come classifica definitiva, ma come campione rappresentativo di modelli eseguibili entro un budget locale.

1.2.4 Inferenza locale nel prototipo aziendale

L'esecuzione locale di un LLM introduce un vincolo ulteriore rispetto all'uso di API cloud: il modello deve convivere con database, servizi applicativi, pipeline RAG e runtime di inferenza nello stesso ambiente controllato. Nel progetto i3Guild questo requisito deriva dalla necessità di non esporre dati aziendali a servizi esterni. La conseguenza pratica è che la scelta del modello non può basarsi solo sulla qualità linguistica, ma deve considerare memoria disponibile, latenza di caricamento, tempo al primo token e throughput.

La quantizzazione viene quindi introdotta qui come motivazione tecnica generale: ridurre la precisione dei pesi può diminuire l'occupazione di memoria e rendere eseguibili modelli altrimenti troppo costosi. Tuttavia, l'effetto reale dipende dal runtime, dal formato del modello e dal costo di dequantizzazione. Per questo l'analisi sistematica della quantizzazione viene separata dal progetto aziendale e ripresa nei capitoli finali, dedicati all'inferenza locale su Apple Silicon.

1.2.5 Ollama: server di inferenza locale

Alla luce dei vincoli descritti, il progetto adotta un'infrastruttura di inferenza locale gestita tramite **Ollama** [59], un server open source che espone modelli locali in formato GGUF tramite un'API REST compatibile con la specifica *OpenAI Chat Completions*. Ollama semplifica il ciclo di vita dei modelli locali: l'installazione avviene tramite un comando semplice (`ollama pull <modello>`), la

gestione del contesto e del formato di prompt è automatizzata, e il server può essere distribuito come container Docker. Nel contesto di questo progetto, Ollama serve il modello di chat; la generazione degli embedding è invece affidata a una pipeline separata basata su FastEmbed e Haystack, come descritto nella Sezione 1.3.3.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Motivazione e definizione

Il paradigma *Retrieval-Augmented Generation* (RAG), formalizzato da Lewis et al. [24], risponde al limite degli LLM come sistemi a conoscenza chiusa (*closed-book*). In un modello generativo tradizionale, infatti, la conoscenza disponibile è codificata nei parametri e non può essere aggiornata senza un nuovo ciclo di addestramento o fine-tuning.

In un sistema RAG, la generazione viene preceduta da una fase di *retrieval* dinamico. Prima di invocare l'LLM, il sistema interroga una base di conoscenza esterna e recupera i documenti più pertinenti rispetto alla query. Tali documenti vengono poi inseriti nel prompt (*prompt augmentation*), così che la risposta sia fondata non solo sulla conoscenza parametrica del modello, ma anche su fonti aggiornate e verificabili [30]. Questa separazione tra modello e conoscenza esterna è il motivo per cui RAG si adatta bene a domini aziendali in evoluzione.

1.3.2 Architettura del sistema RAG

Dal punto di vista architetturale, un sistema RAG separa il momento in cui la conoscenza viene preparata dal momento in cui viene interrogata. Ne derivano due pipeline distinte, messe a confronto nella Tabella 1.4 e illustrate nella Figura 1.1: la pipeline di **ingestione**, eseguita offline, e la pipeline di **inferenza**, attivata in tempo reale a ogni richiesta utente.

Tabella 1.4: Le due pipeline del sistema RAG a confronto.

Fase	Ingestion pipeline (offline)	Query pipeline (real-time)
1	Pre-processing e chunking dei documenti sorgente	Embedding della query utente nello stesso spazio vettoriale dei documenti
2	Generazione di embedding densi e sparsi tramite modelli locali	Ricerca ibrida nel database vettoriale (<i>top-k</i>) con filtri su metadati
3	Indicizzazione in Qdrant con payload di metadati strutturati	Augmented generation: contesto recuperato + query → LLM
<i>Trigger</i>	Esecuzione manuale o schedulata (es. giornaliera)	Ad ogni richiesta utente
<i>Latenza</i>	Minuti / ore (elaborazione batch)	Millisecondi (inferenza real-time)

Pipeline di ingestione

La pipeline di ingestione trasforma i documenti grezzi in una rappresentazione adatta alla ricerca semantica. Il risultato non è più soltanto un archivio di testi, ma un indice interrogabile per

similarità. Il documento viene anzitutto normalizzato e, quando necessario, suddiviso in unità testuali di dimensione controllata. La granularità del chunking incide direttamente sulla qualità del retrieval: chunk troppo piccoli perdono contesto, mentre chunk troppo grandi possono diluire il segnale semantico o superare la finestra del modello di embedding [10].

Ogni chunk viene poi trasformato in due rappresentazioni complementari. La prima è un vettore denso prodotto da un modello di embedding $f: \mathcal{T} \rightarrow \mathbb{R}^d$, in modo che documenti semanticamente simili producano vettori vicini secondo la similarità coseno:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} \quad (1.3)$$

La rappresentazione sparsa descrive invece il testo attraverso pesi associati a termini o feature lessicali. Questa componente conserva segnali che l'embedding denso può attenuare, come nomi propri, tecnologie, sigle o termini specifici del dominio.

Le rappresentazioni dense e sparse vengono infine persistite in **Qdrant** [68], insieme ai metadati strutturati necessari al filtraggio. Per la componente densa Qdrant usa l'algoritmo HNSW (*Hierarchical Navigable Small World*) [26] per la ricerca approssimata (*Approximate Nearest Neighbor*, ANN), mentre la componente sparsa abilita il matching lessicale pesato. La scelta di Qdrant rispetto ad alternative quali pgvector, Chroma e Milvus è motivata nel Capitolo 2.

Pipeline di inferenza

La pipeline di inferenza utilizza l'indice costruito offline per rispondere a una query utente. A differenza dell'ingestione, questa fase è vincolata dalla latenza percepita e deve bilanciare qualità del recupero, numero di documenti restituiti e costo della generazione. La query viene trasformata nelle stesse due rappresentazioni usate in indicizzazione: un embedding denso per la similarità semantica e una rappresentazione sparsa per il matching lessicale. Su queste rappresentazioni si esegue una ricerca ibrida in Qdrant, combinando il punteggio denso e quello sparso; quando la query contiene vincoli strutturati, il retrieval può essere limitato tramite *metadata filtering*, ad esempio ai documenti appartenenti a una specifica epoca temporale. Il contesto recuperato viene quindi concatenato alla query nel prompt inviato all'LLM, che genera la risposta usando sia la propria conoscenza parametrica sia le informazioni esterne fornite dal sistema.

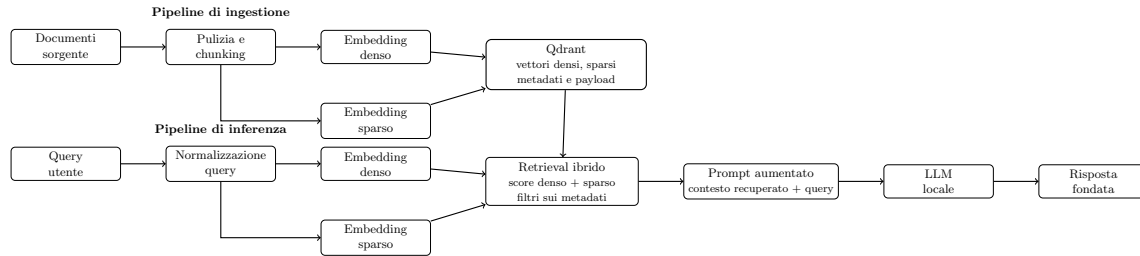


Figura 1.1: Schema della pipeline RAG ibrida: la fase di ingestione costruisce rappresentazioni dense, sparse e metadati; la fase di inferenza combina retrieval semantico, matching lessicale e filtri strutturati prima della generazione aumentata [24].

1.3.3 Modelli di embedding

La qualità del retrieval dipende in larga misura dal modello di embedding: se la rappresentazione vettoriale non preserva correttamente il significato dei documenti, anche il modello generativo riceverà un contesto poco pertinente. Il vincolo di esecuzione on-premise restringe quindi la scelta a modelli distribuibili localmente. Nel sistema progettato, FastEmbed viene usato tramite Haystack per produrre due segnali complementari: embedding densi, adatti alla similarità semantica, ed embedding sparsi, utili per il matching di termini specifici. L'analisi completa della configurazione adottata è presentata nel Capitolo 2.

1.3.4 Database vettoriali

Un database vettoriale è un sistema di storage specializzato per vettori densi ad alta dimensionalità, progettato per rispondere a query di tipo *k-nearest neighbor* (kNN). A differenza dei database relazionali, dove le query sono esatte, la ricerca vettoriale è per natura approssimata: si cercano i vettori più vicini accettando un trade-off tra richiamo (*recall*) e velocità (*throughput*).

L'algoritmo HNSW costruisce un grafo gerarchico multi-livello nello spazio vettoriale [26]. Ogni punto è collegato a un numero limitato di vicini; i livelli superiori contengono meno nodi e consentono salti lunghi, mentre i livelli inferiori raffinano la ricerca nella regione più promettente. La query parte da un nodo di ingresso, si muove verso vicini più simili e scende progressivamente di livello fino a individuare un insieme di candidati. Il parametro M controlla il numero massimo di collegamenti per nodo, mentre ef regola ampiezza e accuratezza della ricerca: valori più alti aumentano il richiamo, ma richiedono più memoria e più confronti.

Rispetto alla scansione lineare, che confronta la query con tutti i vettori e ha costo $O(n)$, HNSW riduce drasticamente il numero di confronti necessari. La complessità effettiva dipende dalla struttura del grafo e dai parametri scelti, ma nella pratica rende possibile l'Approximate Nearest Neighbor su collezioni molto più ampie di quelle gestibili con ricerca esatta. Qdrant usa HNSW per la componente vettoriale densa e lo combina con indici di payload per applicare filtri strutturati prima o durante il retrieval [67].

La ricerca sparsa segue una logica diversa. Un vettore sparso contiene solo gli indici dei termini o delle feature presenti e i relativi pesi; per questo può essere interrogato con strutture simili a un indice invertito, dove il sistema recupera rapidamente i documenti che condividono feature non nulle con la query. Nel progetto, questa componente non sostituisce HNSW: affianca la ricerca densa e permette di valorizzare nomi propri, sigle e tecnologie che potrebbero essere poco evidenti nella sola similarità semantica. La ricerca ibrida combina quindi due segnali, uno semantico e uno lessicale, tramite meccanismi di fusione o tramite il retriever ibrido esposto da Qdrant/Haystack [66, 49].

1.3.5 Agentic RAG

Il RAG lineare descritto finora segue un flusso deterministico: recupero dei documenti, costruzione del prompt e generazione della risposta. Nei casi in cui la query richiede più passaggi logici, confronto tra fonti o riformulazioni successive, tale schema può risultare insufficiente. L'*Agentic RAG* estende questo approccio utilizzando l'LLM come unità di ragionamento capace di invocare strumenti, pianificare ricerche successive e valutare la pertinenza dei documenti recuperati. Architetture come ReAct [38] formalizzano questo ciclo combinando ragionamento verbale e azioni.

In contesti RAG avanzati, tecniche come Self-RAG [2] e Corrective RAG (CRAG) [37] introducono meccanismi di controllo sulla qualità del retrieval e della risposta generata. Il sistema può così

rilevare lacune informative, richiedere nuovi documenti o modificare il prompt prima di produrre la risposta finale. Le rassegne più recenti evidenziano che questa evoluzione è particolarmente utile quando le query richiedono sintesi da fonti eterogenee o passaggi intermedi di ragionamento [15].

Un'altra direzione di sviluppo è rappresentata da **GraphRAG**, formalizzato da Microsoft Research nel 2024 [12]. A differenza del RAG vettoriale classico, che è efficace nel recupero di passaggi localmente simili alla query ma meno adatto a interrogazioni globali sull'intero dataset, GraphRAG estrae entità e relazioni dai testi sorgente e costruisce un grafo di conoscenza gerarchico. In fase di retrieval, il sistema può quindi aggregare riassunti semantici di intere *community* di concetti, ottenendo un livello di astrazione più adeguato all'analisi di corpora documentali estesi.

1.3.6 Tecniche di adattamento al dominio

Il collegamento tra LLM e conoscenza aziendale può essere realizzato in modi diversi. RAG mantiene separati i pesi del modello e il corpus documentale; il fine-tuning, invece, modifica il modello affinché si adatti a un dominio o a uno stile di risposta specifico. Questa sezione introduce le principali tecniche di adattamento a parametri ridotti, utili per inquadrare una scelta progettuale del lavoro: mantenere il corpus aziendale esterno ai pesi del modello e aggiornabile tramite pipeline di ingestione.

Fine-tuning con adattatori: LoRA

Il fine-tuning completo di un LLM richiede risorse computazionali elevate. Hu et al. [17] hanno proposto *Low-Rank Adaptation* (LoRA), che vincola gli aggiornamenti dei pesi a matrici di rango basso. Dato un layer lineare con matrice $\mathbf{W}_0 \in \mathbb{R}^{m \times n}$, LoRA introduce:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA} \quad (1.4)$$

con $\mathbf{A} \in \mathbb{R}^{r \times n}$, $\mathbf{B} \in \mathbb{R}^{m \times r}$ e $r \ll \min(m, n)$. I pesi originali $\mathbf{W}_0$ rimangono congelati; solo $\mathbf{A}$ e $\mathbf{B}$ vengono ottimizzati. Con $r = 8$ e $m = n = 4096$, il numero di parametri addestrabili si riduce di circa due ordini di grandezza rispetto al fine-tuning completo.

QLoRA: fine-tuning a 4-bit

Quantized LoRA (QLoRA) [11] combina la quantizzazione a 4-bit del modello base con adattatori LoRA a precisione piena. I pesi vengono quantizzati in formato NF4 (*NormalFloat4*), ottimizzato per le distribuzioni tipiche dei pesi neurali; durante il forward pass i pesi vengono de-quantizzati dinamicamente in BF16, mentre gli adattatori operano in BF16.

La Tabella 1.5 riassume i principali trade-off tra le due varianti.

Tabella 1.5: Confronto tra LoRA e QLoRA per un modello base da 7B parametri.

Caratteristica	LoRA	QLoRA
Precisione pesi base	FP16/BF16	NF4 (4-bit)
Precisione adattatori	FP16/BF16	BF16
Memoria GPU (modello 7B)	~14 GB	~4 GB
Overhead de-quantizzazione	assente	presente
Degradazione qualitativa stimata	trascurabile	<1% su benchmark principali

DoRA: Weight-Decomposed Low-Rank Adaptation

Nel 2024 Liu et al. hanno introdotto **DoRA** (*Weight-Decomposed Low-Rank Adaptation*) [25]. Il metodo parte dall'osservazione che LoRA approssima bene l'aggiornamento direzionale dei pesi, ma cattura con minore efficacia le variazioni di magnitudine osservate nel fine-tuning completo. DoRA separa quindi il tensore dei pesi in una componente di magnitudine e una componente direzionale: la prima viene aggiornata direttamente, mentre la seconda sfrutta l'approssimazione a rango basso di LoRA. Il divario prestazionale rispetto al fine-tuning completo si riduce così senza rinunciare al vantaggio principale degli adattatori, cioè il numero contenuto di parametri addestrabili.

Prompt engineering e inference-time intervention

In alternativa al fine-tuning parametrico, è possibile adattare il comportamento di un LLM intervenendo sulla formulazione dell'input senza modificare i pesi. Tecniche come *few-shot prompting* [8] e *chain-of-thought prompting* [36] guidano il modello verso risposte più strutturate, arricchendo il prompt con esempi o istruzioni di ragionamento. Nel sistema descritto in questa tesi, il prompt engineering è impiegato nella definizione del *system prompt* e nell'iniezione strutturata del contesto RAG. Le strategie adottate sono descritte nel Capitolo 3, insieme agli endpoint che costruiscono il prompt aumentato.

1.4 Framework di orchestrazione: Haystack

La costruzione di un sistema RAG non consiste soltanto nella scelta del modello generativo. È necessario coordinare modelli di embedding, database vettoriali, LLM e fasi di pre-processing attraverso interfacce coerenti e componibili. **Haystack** [50], sviluppato da deepset, è un framework Python open source progettato per organizzare questi componenti in pipeline esplicite.

Nella versione 2.x adottata in questo progetto, ogni pipeline è un grafo diretto aciclico (DAG) di componenti con interfacce tipizzate, collegabili in modo dichiarativo. Sono disponibili integrazioni native con Qdrant (`QdrantDocumentStore`), FastEmbed (`FastembedDocumentEmbedder`, `FastembedTextEmbedder` e componenti sparse corrispondenti), Ollama (`OllamaGenerator`) e le principali operazioni di pre-processing (`DocumentCleaner`, `DocumentSplitter`). Il dettaglio implementativo della pipeline di ingestione e di retrieval è descritto nel Capitolo 3.

1.5 Sintesi e posizionamento del lavoro

Il capitolo ha definito il quadro tecnico necessario per comprendere il progetto aziendale descritto nel seguito della tesi. Le architetture Transformer sono il substrato comune della generazione e degli embedding; i loro limiti nei contesti a conoscenza chiusa, in particolare conoscenza statica, allucinazioni e assenza di accesso nativo ai dati interni, spiegano l'adozione del paradigma RAG. La scelta di operare con modelli e infrastrutture on-premise, come Ollama, Qdrant e Haystack, risponde invece ai vincoli di riservatezza e manutenibilità del caso i3Guild. Su queste basi, il capitolo successivo traduce i fondamenti teorici in requisiti, scelte tecnologiche e architettura del sistema.

Capitolo 2

Analisi dei Requisiti e Progettazione del Sistema

Il capitolo traduce il quadro teorico introdotto in precedenza in un problema progettuale concreto. L'obiettivo non è costruire un sistema RAG generico, ma integrare un assistente nel contesto operativo di Intré, rispettando i vincoli organizzativi, infrastrutturali e di riservatezza già presenti. Per questo la trattazione parte dal modello aziendale delle Gilde, identifica il caso d'uso e il corpus informativo disponibile, formalizza i requisiti e motiva infine le scelte tecnologiche che portano alla proposta architetturale.

2.1 Il contesto aziendale: Intré S.r.l.

Intré S.r.l.¹ è una software factory italiana con sede a Monza. L'azienda sviluppa prodotti digitali per settori eterogenei, tra cui energia, assicurazioni, automotive ed entertainment, e organizza il lavoro in team Scrum cross-funzionali. La dimensione aziendale, pari a circa settanta persone, consente di mantenere processi interni formalizzati senza rinunciare a una circolazione diretta della conoscenza tra i team [18].

Il payoff *Learn / Code / Deploy Value* esplicita l'ordine con cui Intré interpreta il proprio modo di produrre valore: prima l'apprendimento, poi la realizzazione del software, infine il rilascio di valore al cliente. Questa impostazione è coerente con il concetto di *Learning Organization* proposto da Peter Senge in *The Fifth Discipline* [29]. Nel modello di Senge l'apprendimento organizzativo non coincide con la sola formazione individuale, ma richiede padronanza personale, modelli mentali esplicitati, visione condivisa, apprendimento di gruppo e pensiero sistemico. La quinta disciplina, cioè il pensiero sistemico, collega le altre quattro e permette di leggere l'organizzazione come un insieme di relazioni, feedback e dipendenze, non come una somma di attività isolate.

Nel caso di Intré questa prospettiva non rimane un principio astratto. Il modello aziendale prevede un insieme di pratiche dedicate all'apprendimento continuo: Gilde, Camp, Community, Skill Matrix, certificazioni, Academy, Community of Practice per speaker, obiettivi di apprendimento, formazione in aula e corsi di lingua [18]. Tali pratiche rendono l'apprendimento una responsabilità distribuita, sostenuta da tempo, budget e momenti di restituzione pubblica. La certificazione

¹<https://www.intre.it>

ISO 27001 e le competenze maturate in ambiti come cloud-native, cybersecurity e intelligenza artificiale rendono inoltre particolarmente rilevante il tema della gestione sicura della conoscenza interna.

2.1.1 Il modello delle Gilde

Le **Gilde** (*gilde*) sono uno degli strumenti principali con cui Intré rende operativa la propria idea di apprendimento organizzativo. Sono cicli rapidi di apprendimento di gruppo, della durata di quattro mesi, costruiti a partire da temi proposti direttamente dalle persone dell'azienda [18, 19]. Una Gilda nasce quando una proposta raccoglie un numero minimo di partecipanti; il gruppo lavora in autogestione e dedica al tema scelto una quota stabile del proprio tempo lavorativo. L'esito non è un esame o una valutazione formale, ma un risultato di apprendimento da presentare al Camp successivo. Il modello combina quindi autonomia nella scelta dei temi, durata definita, lavoro di gruppo e produzione di un risultato condivisibile. Le sue caratteristiche principali sono:

- **Formazione spontanea:** ogni persona può proporre o aderire a una Gilda su qualsiasi argomento, tecnico o non tecnico (es. linguaggi di programmazione, ceramica, game development, intelligenza artificiale).
- **Ciclo temporale (Epoch):** le Gilde si ricreano ogni quattro mesi; ogni ciclo costituisce un'“epoca” distinta, con nuovi temi e nuovi gruppi. Questa scansione periodica evita che l'apprendimento rimanga legato a iniziative isolate e lo inserisce in una routine aziendale riconoscibile.
- **Output condivisibile:** al termine del periodo la Gilda produce un risultato tangibile (prototipo, report, repository, presentazione) che viene archiviato e condiviso. La restituzione pubblica durante il Camp trasforma l'esperienza del gruppo in conoscenza disponibile anche per il resto dell'organizzazione.
- **Ampiezza tematica:** dall'archivio pubblico del sito aziendale emergono Gilde su temi come *Angular*, *F#*, *IA agentiva*, *game development* con Phaser, ceramica, selezione fotografica con AI, e molti altri [19].

Il ciclo di vita delle Gilde è gestito tramite **i3Guild**, una piattaforma web interna sviluppata con Next.js, PostgreSQL e Keycloak. Questa applicazione conserva nel tempo proposte, descrizioni, partecipanti e risultati delle Gilde. Di conseguenza, i3Guild non è soltanto uno strumento amministrativo: è una memoria organizzativa parziale, nella quale si depositano interessi, competenze, esperimenti e risultati prodotti nel tempo. Proprio per questo rappresenta sia il contesto applicativo di integrazione sia la principale sorgente dati del sistema RAG progettato in questa tesi.

2.2 Caso d'uso: Proposta di nuove Gilde Intré

Il caso d'uso individuato riguarda il supporto alla proposta di nuove Gilde. Quando una persona vuole avviare una nuova iniziativa, può essere utile conoscere esperienze precedenti, temi già affrontati, risultati ottenuti e partecipanti coinvolti. Oggi queste informazioni esistono, ma sono distribuite tra database e contenuti editoriali. Il sistema proposto mira a renderle consultabili tramite query in linguaggio naturale, generando risposte contestualizzate e ancorate a fonti interne. A tale

scopo viene adottato il paradigma *Retrieval-Augmented Generation* (RAG), introdotto da Lewis et al. [23] per estendere la memoria non parametrica dei Large Language Model (LLM) tramite recupero dinamico di documenti da un corpus esterno.

2.2.1 Motivazione del caso d'uso

Il corpus delle Gilde archiviate in i3Guild, insieme ai relativi blog post, costituisce una base di conoscenza aziendale di valore: raccoglie descrizioni di progetti, obiettivi, rischi, risultati raggiunti e annotazioni dei partecipanti, accumulate in oltre trenta epoch a partire dal 2016 [19]. Tuttavia questa conoscenza è frammentata. Una parte risiede in un database relazionale, ottimo come sorgente applicativa ma poco accessibile tramite domande in linguaggio naturale; un'altra parte è distribuita in post e contenuti non sempre facili da reperire. Un assistente RAG consente di trasformare tale patrimonio in uno strumento operativo: riduce il tempo di ricerca, facilita il riuso delle esperienze pregresse e rende più trasferibile la conoscenza tra i team.

2.2.2 Obiettivi del caso d'uso

Gli obiettivi del caso d'uso derivano direttamente dalla natura del corpus e dal contesto aziendale in cui il sistema deve essere inserito. Il sistema deve fornire risposte affidabili e tracciabili sui contenuti presenti nei repository aziendali, riducendo il rischio di allucinazioni tipico della generazione puramente parametrica [23], e deve rendere più rapido il recupero di informazioni utili per attività operative e decisionali interne a Intré. Questi obiettivi devono essere raggiunti senza spostare i dati fuori dall'infrastruttura aziendale, in coerenza con i requisiti di sovranità del dato e con il Regolamento Generale sulla Protezione dei Dati (GDPR) [14]. L'integrazione, inoltre, deve inserirsi nel flusso i3Guild esistente senza alterare il processo operativo già consolidato.

2.2.3 Descrizione dei dati e analisi del corpus

Per progettare correttamente la pipeline di ingestione è necessario chiarire quali informazioni debbano alimentare l'indice. I dati di ingresso al sistema RAG comprendono tre categorie:

- **Documenti delle Gilde:** specifiche di progetto, obiettivi, rischi, risultati attesi e raggiunti, annotati con metadati strutturati (epoche, partecipanti, modalità di partecipazione).
- **Blog post:** post pubblicati dai partecipanti alle gilde che ne sintetizzano i risultati e le esperienze maturate.
- **Documenti non strutturati:** file PDF, HTML e note operative con metadati associati.

Dall'analisi del corpus i3Guild, estratto il 15/02/2026, emergono le statistiche riportate nelle Tabelle 2.1 e 2.2. Questi valori non hanno solo funzione descrittiva: determinano la granularità dell'indicizzazione, la strategia di chunking e la scelta del modello di embedding.

La somma media dei campi testuali rilevanti della Gilda, inclusi sommario, obiettivi, rischi, outcome e risultati raggiunti, ammonta a circa 1.900 caratteri, corrispondenti a circa 400 token. Anche nel caso peggiore la somma non supera i 7.000–8.000 token, rimanendo compatibile con la finestra di contesto del modello di embedding selezionato (Sezione 2.4.3). Da ciò deriva una conseguenza progettuale importante: nella maggior parte dei casi una Gilda può essere trattata come documento consolidato, senza frammentare aggressivamente i campi testuali e senza perdere il contesto semantico che lega obiettivi, rischi e risultati.

Tabella 2.1: Statistiche delle entità principali del corpus i3Guild (estrazione 15/02/2026).

Entità	Conteggio	Note
Gilda	270	Gestibile per re-indicizzazioni complete
Epoch	30	≈ 9 gilde per epoca in media
Relazioni Utente-Gilda	1.166	≈ 4,3 partecipanti medi per gilda
Utenti esterni	3	Trascurabile ai fini del RAG

Tabella 2.2: Statistiche dei campi testuali delle entità Gilda (valori in caratteri).

Campo	Media	Min	Max	Vuoti	Note
summary	610	0	18.224	5%	Campo più ricco; outlier rilevante
goals	258	0	2.698	35%	Spesso breve o assente
outcome	133	0	2.145	36%	Spesso breve o assente
risks	106	0	1.245	38%	Spesso breve o assente
achievedResultsDescription	782	0	6.421	74%	Lungo quando presente
journal	≈1	1	1	99%	Praticamente inutilizzato; escluso

2.2.4 Struttura informativa delle Gilde e implicazioni per il retrieval

L'analisi dello schema dati i3Guild conferma che la Gilda è l'unità informativa più adatta all'indicizzazione. La tabella **Guild** raccoglie campi descrittivi, campi progettuali e metadati organizzativi: nome, sommario, obiettivi, opportunità, risultati attesi, rischi, risorse, budget, modalità di partecipazione, riferimento all'epoch e descrizione dei risultati raggiunti. Le tabelle collegate hanno un ruolo diverso. **Epoch** fornisce il contesto temporale; **GuildUserRelation** collega partecipanti, Gilde ed epoch; **GuildAchievedResult** contiene allegati binari, dei quali in questa fase può essere indicizzato solo il nome del file, salvo introdurre una pipeline dedicata di estrazione testuale.

Questa distinzione separa contenuto da metadato. I campi testuali della Gilda alimentano embedding e ricerca semantica; epoch, partecipanti, modalità di partecipazione e identificativi restano nel payload strutturato e servono per filtrare o contestualizzare la risposta. La separazione evita due problemi: inserire nel testo indicizzato informazioni poco semantiche, come identificativi o date, e perdere però la possibilità di rispondere a domande operative che richiedono vincoli strutturati. Una domanda come “quali Gilde sull'intelligenza artificiale sono state proposte in una certa epoch?” richiede entrambe le componenti: similarità semantica sul tema e filtro temporale sull'epoch.

Gli esempi presenti nella documentazione interna del progetto chiariscono il tipo di interrogazioni attese. Una query esplorativa come “vorrei fondare una gilda sulla musica” deve essere ridotta ai termini informativi principali e recuperare Gilde tematicamente vicine, ad esempio *The Rolling Updates* o *Visual note for piano*, quando presenti nel corpus indicizzato. Domande come “quali Gilde si occupano di AI?” privilegiano invece il recupero semantico su **summary**, **goals** e **achievedResultsDescription**; domande come “chi ha partecipato alla Gilda X?” o “quali erano i rischi della Gilda Y?” richiedono il raccordo tra retrieval, metadati e campi specifici. Il corpus, quindi, non è una semplice collezione di testi: è un insieme di documenti con struttura interna, relazioni e vincoli temporali.

Da questa lettura deriva la scelta di costruire documenti consolidati con etichette di campo

Tabella 2.3: Ruolo dei principali dati i3Guild nella pipeline RAG.

Elemento	Informazione disponibile	Uso nella pipeline
Guild	Nome, sommario, obiettivi, opportunità, outcome, rischi, risorse, budget, risultati raggiunti	Documento principale da incorporare e recuperare semanticamente.
Epoch	Inizio e fine del ciclo di Gilde	Filtro temporale e contesto della risposta.
GuildUserRelation	Partecipante, Gilda, epoch, data di adesione	Payload per domande su partecipazione e composizione dei gruppi.
GuildAchievedResult	File allegati e nome del file	Fonte secondaria; il contenuto binario richiede estrazione separata.
GuildDraft	Proposte non ancora consolidate	Esclusa dall'indice principale se non validata come conoscenza storica.

esplicite, ad esempio [SOMMARIO], [OBIETTIVI], [RISCHI] e [RISULTATI RAGGIUNTI]. Le etichette mantengono riconoscibile la provenienza dell'informazione anche dopo la concatenazione dei campi e aiutano il generatore a distinguere tra intenzioni, rischi e risultati effettivamente raggiunti. La strategia è coerente con le dimensioni osservate: poiché la Gilda media rimane breve, il documento consolidato conserva più contesto rispetto a una segmentazione per singolo campo, senza introdurre un carico eccessivo per il modello di embedding.

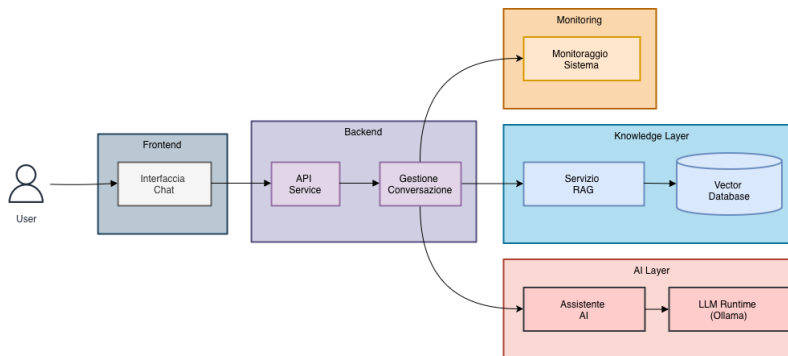


Figura 2.1: Diagramma del caso d'uso "Intré": attori, interazioni principali e confini del sistema.

2.3 Requisiti di sistema

Il caso d'uso appena descritto permette di passare dalla motivazione generale ai requisiti del sistema. Essi sono stati ricavati dall'analisi del corpus, dai vincoli infrastrutturali di Intré e dalle policy aziendali in materia di protezione dei dati. La Tabella 2.4 sintetizza i requisiti ad alto livello, suddivisi per categoria.

Tabella 2.4: Riepilogo requisiti ad alto livello.

Categoria	Requisiti principali
Funzionali	Ricerca semantica full-text su documenti interni; generazione di risposte contestualizzate con riferimenti alle fonti; capacità di filtraggio per metadati (epoca, modalità di partecipazione, proprietario della gilda).
Privacy e conformità	Esecuzione on-premise; nessun invio di dati a servizi cloud esterni; logging e auditing degli accessi; protezione dei dati in conformità al GDPR [14]. I rischi relativi alla privacy nei sistemi RAG, tra cui la possibile estrazione dei dati di retrieval, sono discussi in letteratura [39]; la scelta on-premise costituisce la principale misura mitigativa adottata.
Performance	Latenza compatibile con uso interattivo per query semplici, distinguendo tra modello già caricato e cold start; throughput adeguato a carichi concorrenti moderati; inferenza CPU-only come scenario primario.
Costi e sostenibilità	Minimizzazione dei costi operativi tramite uso efficiente delle risorse (quantizzazione dei modelli, inferenza CPU); adozione di soluzioni open-source senza costi per licenze.
Manutenibilità	Componenti modulari con interfacce standardizzate; automazione della pipeline di ingestione; documentazione e test automatizzati.

2.3.1 Vincoli architetturali

I requisiti precedenti si traducono in vincoli architetturali non negoziabili. L'esecuzione on-premise è il vincolo più rilevante: i dati delle Gilde, incluse le informazioni sui partecipanti e sui progetti, non possono transitare su infrastrutture cloud di terze parti. Il deployment locale diventa quindi la condizione tecnica per rispettare i principi di integrità e riservatezza previsti dal GDPR [14]. L'integrazione deve inoltre rimanere compatibile con lo stack i3Guild esistente, basato su Docker Compose, PostgreSQL/Prisma, Next.js e Keycloak per l'autenticazione.

Un vincolo meno visibile, ma altrettanto determinante, riguarda le risorse hardware. L'ambiente Docker dispone di 8 GB di RAM complessivi, condivisi tra database, servizi applicativi, pipeline di embedding quando attiva e modello di generazione del testo. Questa disponibilità limita la scelta del modello LLM, discussa nella Sezione 2.4.4, e impone di evitare componenti con requisiti di memoria elevati. Per la stessa ragione, generazione ed embedding devono restare locali senza fare affidamento su API esterne: Ollama gestisce il modello generativo, mentre FastEmbed/Haystack produce gli embedding senza dipendere da provider cloud.

2.4 Scelte tecnologiche: analisi comparativa

Le scelte tecnologiche sono state effettuate a partire dai vincoli appena definiti. Per ciascun componente principale dello stack sono state valutate alternative compatibili con l'esecuzione on-premise, privilegiando soluzioni semplici da integrare nello stack i3Guild, osservabili e sostenibili rispetto alle risorse disponibili.

2.4.1 Framework di orchestrazione RAG

Il framework di orchestrazione deve collegare ingestione, embedding, retrieval e generazione senza introdurre eccessiva complessità operativa. Sono stati considerati tre framework open-source: **LangChain**, **LlamaIndex** e **Haystack 2.x**.

LangChain offre l'ecosistema più ampio di integrazioni e una forte vocazione verso workflow agentici e *multi-step reasoning*. Nel 2024 l'introduzione di **LangGraph** [56] ha ampliato questa impostazione, permettendo di modellare applicazioni multi-agente come grafi di stato. Tale flessibilità è utile in scenari complessi, ma nel progetto in esame introduce un costo non trascurabile in termini di curva di apprendimento, stabilità delle API e manutenzione, anche alla luce dei *breaking changes* frequenti riportati in letteratura [63].

LlamaIndex è orientato alla gestione dei dati e al retrieval avanzato. Offre numerosi data connector e strutture di indicizzazione specializzate, risultando particolarmente adatto a pipeline di question-answering documentale [46]. Rispetto al caso considerato, tuttavia, l'obiettivo principale non è solo indicizzare sorgenti eterogenee, ma integrare una pipeline osservabile e facilmente testabile nello stack applicativo esistente.

Haystack 2.x (deepset) adotta un modello a componenti tipizzati con input/output espliciti, organizzati in pipeline dichiarative serializzabili in YAML. Studi comparativi evidenziano che Haystack offre il minore overhead di orchestrazione (≈ 5.9 ms) e il minore consumo di token rispetto a LangChain (≈ 10 ms) su benchmark standardizzati [40]. La sua architettura pipeline-first favorisce la tracciabilità, i test e la sostituzione dei componenti in produzione [62].

Tabella 2.5: Confronto sintetico dei framework RAG candidati.

Criterio		LangChain	LlamaIndex	Haystack 2.x
Filosofia		Agente/imperativa	Data-first	Pipeline dichiarativa
Overhead	orche-	≈ 10 ms	≈ 6 ms	≈ 5.9 ms
Integrazione		Nativa	Nativa	Nativa
Qdrant				
Integrazione	Olla-	Sì	Sì	Sì
ma				
Testabilità pipeline		Media	Media	Alta
Serializzazione		No	Parziale	Sì
YAML				
Adatto per produ-		Sì (con overhead)	Sì	Sì (preferito)
zione RAG				

Alla luce di questo confronto, Haystack 2.x risulta la soluzione più coerente con il progetto. L'architettura a componenti tipizzati, la documentazione rigorosa, le integrazioni native con Qdrant (**QdrantDocumentStore**), FastEmbed per embedding densi e sparsi e Ollama per la generazione permettono di costruire una pipeline osservabile senza introdurre un framework agentico più complesso del necessario. Il minore overhead di orchestrazione e la possibilità di serializzare le pipeline

rafforzano questa scelta in un sistema RAG orientato alla produzione e vincolato all'esecuzione on-premise [63].

2.4.2 Database vettoriale

La ricerca di similarità approssimata (*Approximate Nearest Neighbor*, ANN) su vettori ad alta dimensionalità è il componente infrastrutturale che abilita il retrieval. La scelta del database vettoriale incide su latenza, filtraggio per metadati, semplicità di deployment e capacità di manutenzione dell'indice. Sono stati quindi valutati quattro database vettoriali self-hostable.

Tabella 2.6: Analisi comparativa dei database vettoriali candidati.

Database	Impl.	Indice	Filtering	Dashboard	Deployment
Qdrant	Rust	HNSW	Pre-filtering payload	Web UI integrata	1 container
pgvector	C	IVFFlat / HNSW	SQL WHERE	No (via PgAdmin)	Estensione PG
Milvus	Go/C++	HNSW, IVF	Attribute-based	Attu (separato)	Multi-container
Chroma	Python	HNSW (hnswlib)	Metadata base	No	1 container

Il confronto non si limita alle prestazioni pure: nel contesto i3Guild hanno peso anche la semplicità operativa e la possibilità di applicare filtri sui metadati delle Gilde. I principali criteri di esclusione e selezione sono i seguenti:

- **pgvector** è stato considerato in quanto l'infrastruttura i3Guild utilizza già PostgreSQL. Tuttavia, la letteratura segnala che le prestazioni di pgvector degradano significativamente oltre le centinaia di migliaia di vettori, con ritardi fino a 15 volte superiori rispetto a Qdrant in termini di throughput su dataset grandi [55]. Benchmark più recenti mostrano un divario più contenuto (Qdrant: ≈ 41 QPS a 99% recall su 50M vettori; pgvectorscale: ≈ 471 QPS), ma evidenziano che Qdrant ottiene latenze tail significativamente migliori (39% migliore a p95, 48% migliore a p99) [72]. Assenza di dashboard integrata e di payload filtering flessibile completano le ragioni per la non-selezione.
- **Milvus** è stato escluso per l'elevata complessità di deploy: richiede componenti aggiuntivi (etcd, MinIO), aumentando il carico operativo incompatibile con i vincoli del progetto.
- **Chroma** è implementato in Python e soffre di instabilità della persistenza dati nelle versioni precedenti; è più adatto a prototipi che a sistemi di produzione.
- **Qdrant** è scritto in Rust e offre buone prestazioni con latenze stabili nei benchmark disponibili. La funzionalità di pre-filtering, che applica i filtri sui payload *prima* della ricerca vettoriale, è particolarmente rilevante per query contestuali come “gilda dell'epoca X”: il filtering riduce lo spazio di ricerca prima del calcolo ANN, mantenendo il top- K accurato. Offre Web UI integrata su porta 6333, client TypeScript ufficiale compatibile con Next.js, e deploy in singolo container Docker [69].

Per le ragioni discusse, e in particolare per il bilanciamento tra prestazioni, semplicità di deployment e supporto al filtering sui metadati, Qdrant è stato selezionato come database vettoriale del progetto. La scelta è coerente con il vincolo aziendale di self-hosting e con la necessità di mantenere il sistema manutenibile anche in un contesto prototipale.

2.4.3 Modello di embedding

Il modello di embedding converte i documenti testuali in rappresentazioni numeriche interrogabili da Qdrant. Nel progetto questa funzione è separata dall’inferenza generativa: Ollama serve il modello di chat, mentre FastEmbed, tramite Haystack, produce le rappresentazioni usate per l’indicizzazione. La pipeline genera sia embedding densi sia embedding sparsi, così il retrieval può combinare similarità semantica e corrispondenza lessicale su nomi propri, tecnologie e termini ricorrenti nelle Gilde [47, 48].

Tabella 2.7: Configurazione embedding del sistema RAG.

Ruolo	Componente Haystack	Default	Funzione
Embedding denso	Document/text embedder denso	sentence-transformers/ paraphrase-multilingual- mpnet-base-v2	Similarità semantica tra query e documenti.
Embedding sparso	Document/text embedder sparso	prithivida/ Splade_PP_en_v1	Matching lessicale pesato, utile per termini specifici.
Store ibrido	QdrantDocumentStore con use_sparse_embeddings= true	i3guild_knowledge	Persistenza con vettori densi e rappresentazioni sparse.

La separazione tra indicizzazione e generazione riduce l’accoppiamento operativo: Ollama rimane dedicato alla risposta LLM, mentre il job batch produce gli embedding senza dipendere dal server di chat. La componente sparsa rende inoltre il retrieval meno dipendente dalla sola similarità densa, aspetto rilevante in un corpus dove nomi propri, tecnologie e termini ricorrenti hanno spesso valore informativo letterale. Ingestione e query seguono quindi lo stesso schema operativo: in scrittura vengono generati embedding densi e sparsi dei documenti; in lettura la query viene trasformata nelle due rappresentazioni corrispondenti e inviata al retriever ibrido Qdrant [49]. Il valore `EMBEDDING_DIMENSIONS=768` rimane il riferimento per la componente densa.

La scelta della ricerca ibrida non deriva da una valutazione quantitativa condotta sul corpus i3Guild, perché durante il progetto non è stato costruito un dataset annotato con query e documenti rilevanti. È però coerente con evidenze esterne recenti: nei task RAG di TREC 2025 un sistema sparse+dense con reranking ottiene i migliori risultati riportati dal team NITATREC (*MAP* 0.1037, *nDCG@30* 0.527 e *Recall@100* 0.158) [31]; in un benchmark finanziario su documenti misti testotabella, una pipeline a due stadi con retrieval ibrido e reranking raggiunge *Recall@5* 0.816 e *MRR@3* 0.605, superando i metodi single-stage [1]. Altri studi mostrano però che il vantaggio non è universale: nel dominio biomedicale una baseline densa resta molto competitiva e le differenze tra strategie dipendono dal corpus e dalle metriche considerate [4]. Per i3Guild questi risultati vanno quindi

usati come supporto alla scelta progettuale, non come prova sperimentale locale: la componente sparsa è motivata soprattutto dalla presenza di nomi di Gilde, ruoli, modalità di partecipazione e termini tecnici che hanno valore informativo letterale.

2.4.4 Server di inferenza LLM e selezione del modello di chat

Il componente di generazione del testo è gestito da **Ollama**, server di inferenza locale che supporta modelli in formato GGUF e consente l'uso di varianti quantizzate a 4 e 8 bit. Nel prototipo aziendale la scelta del modello di chat è stata condizionata da un vincolo operativo preciso: l'intero stack Docker dispone di risorse limitate e deve eseguire, nello stesso ambiente, PostgreSQL, Qdrant, il servizio Next.js, la pipeline RAG e Ollama. Per questo la selezione non mira a identificare il modello migliore in senso assoluto, ma un modello sufficientemente stabile per validare l'integrazione end-to-end.

Il modello adottato nel prototipo è **gemma3:4b**, distribuito in Ollama come **gemma3:latest**. La scelta deriva da prove preliminari interne su modelli disponibili nel registry Ollama, valutati rispetto a latenza percepita, consumo di memoria e qualità minima delle risposte in presenza di contesto RAG. Il modello resta caricabile nello stesso ambiente dei servizi PostgreSQL, Qdrant, Next.js e FastAPI, ma offre un margine qualitativo maggiore rispetto a modelli da circa 1B parametri. Le risposte non sono usate come prova generale della qualità del modello; sono sufficienti per validare il flusso applicativo quando il contesto recuperato da Qdrant è pertinente.

Le prove locali hanno confrontato dodici modelli Ollama su 14 query, divise tra prompt generali e prompt RAG. La valutazione manuale ha assegnato tre punteggi per accuratezza o fedeltà, pertinenza e coerenza. I risultati non costituiscono un benchmark scientifico, perché il numero di query è ridotto e il carico della macchina non è stato isolato; sono però utili come criterio progettuale. In media, **gemma3:1b** ha mostrato la latenza totale più bassa (≈ 6.7 s) e un buon throughput (≈ 28 parole/s), ma una coerenza più debole nelle risposte RAG. **gemma3:4b** ha richiesto più tempo (≈ 17.3 s in media), mantenendo però il punteggio medio più alto tra i modelli di dimensione compatibile con il prototipo. Modelli ragionativi come **phi4-mini-reasoning** hanno prodotto latenze troppo elevate per l'uso interattivo, mentre alcune alternative molto compatte hanno mostrato output meno controllabili. La scelta di **gemma3:4b** privilegia quindi un compromesso pratico: qualità sufficiente, integrazione immediata in Ollama e risorse ancora compatibili con l'ambiente condiviso.

Questa valutazione resta volutamente interna al progetto aziendale: serve a rendere eseguibile il prototipo i3Guild, non a produrre conclusioni generali sull'inferenza locale. La generalizzazione del problema, cioè la misura sistematica di memoria, cold start, *time-to-first-token*, throughput e impatto della quantizzazione su Apple Silicon, viene affrontata nei Capitoli 4 e 5.

2.5 Proposta architetturale

Le scelte tecnologiche descritte convergono in una soluzione modulare e containerizzata. L'architettura separa responsabilità diverse – ingestione e normalizzazione dei dati, generazione degli embedding, persistenza vettoriale, retrieval e inferenza LLM – in modo da mantenere il sistema osservabile, sostituibile e coerente con lo stack i3Guild. La Figura 2.2 riporta lo schema di alto livello dell'architettura.

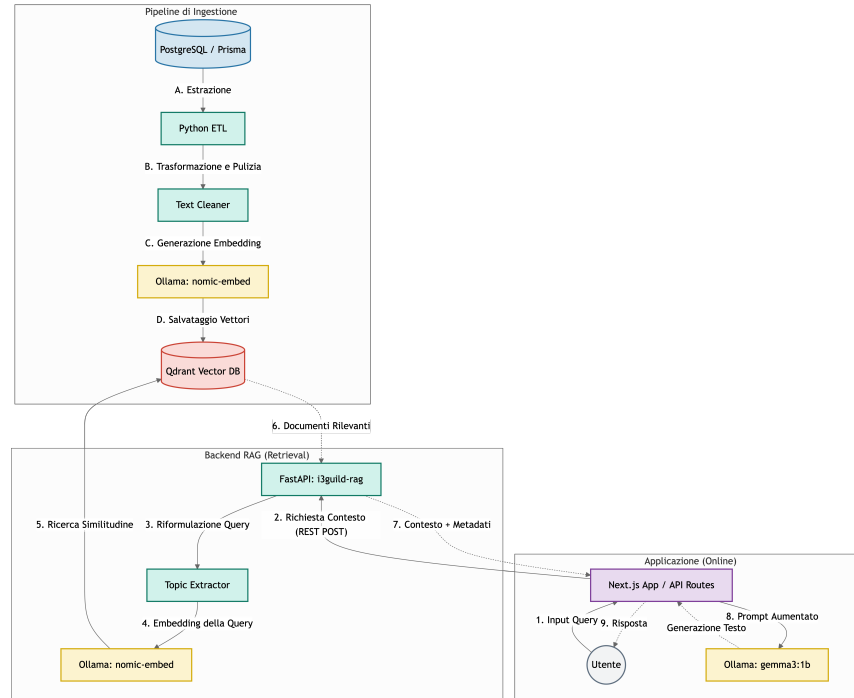


Figura 2.2: Schema architetturale del sistema RAG on-premise per i3Guild. Il flusso di ingestione (ETL) popola Qdrant a partire da PostgreSQL; il flusso di retrieval serve le query di Next.js tramite FastAPI.

2.5.1 Componenti principali

I componenti non sono separati soltanto per ragioni di deployment: ciascuno risponde a un requisito emerso nelle sezioni precedenti. PostgreSQL rimane la sorgente autoritativa dei dati; Qdrant rende interrogabile il corpus per similarità; Ollama consente di mantenere la generazione in locale; FastAPI espone verso Next.js un confine applicativo chiaro, senza incorporare la logica RAG direttamente nel frontend.

Ingestione e pre-processing Pipeline batch Python che esegue le fasi ETL descritte nella Sezione 2.5.3. Acquisisce i documenti da PostgreSQL, normalizza il testo, esegue il chunking, genera embedding densi e sparsi con FastEmbed/Haystack e indicizza su Qdrant.

Strategia di chunking Sulla base dell'analisi dei dati (Sezione 2.2.3), viene adottato l'approccio a *documento consolidato* (1 Gilda ↔ 1 documento vettoriale): tutti i campi testuali rilevanti sono concatenati in un unico documento con separatori strutturati. Per gli outlier che superano il limite configurato viene applicato chunking con overlap, mantenendo nel chunk l'intestazione della Gilda. Questo approccio preserva il contesto semantico nella maggior parte dei casi e mantiene gestibili upsert e cancellazioni su Qdrant.

Database vettoriale (Qdrant) Collection principale `i3guild_knowledge` configurata per retrieval ibrido: vettore denso a 768 dimensioni, rappresentazione sparsa e payload JSON con metadati indicizzati per il filtering (Sezione 2.5.2).

Orchestrazione RAG (Haystack 2.x) Haystack è usato sia nel job di indicizzazione sia nel servizio di query. La pipeline batch collega gli embedder densi e sparsi con il writer; la pipeline di query usa i componenti corrispondenti e il retriever ibrido Qdrant.

Inference LLM (Ollama) Server di inferenza locale; modello di chat adottato: `gemma3:4b`, selezionato sulla base di prove preliminari coerenti con i vincoli del prototipo (Sezione 2.4.4).

Servizio API (FastAPI) Microservizio Python che espone endpoint di retrieval e generazione, tra cui `POST /rag/retrieve` (porta 8000, mappata a 8002 su host). Restituisce documenti con score di similarità e campi diagnostici.

Osservabilità e sicurezza Logging centralizzato, metriche di latenza e utilizzo risorse; autenticazione tramite Keycloak (già presente in i3Guild); cifratura a riposo per i dataset sensibili.

2.5.2 Progettazione della collection Qdrant

La collection `i3guild_knowledge` è il punto in cui la conoscenza estratta da i3Guild viene trasformata in indice interrogabile. La configurazione, riportata nella Tabella 2.8, supporta retrieval ibrido: similarità vettoriale densa e componente sparsa nello stesso store.

Tabella 2.8: Parametri di configurazione della collection Qdrant.

Parametro	Valore	Motivazione
<code>embedding_dim</code>	768	Dimensione della componente densa FastEmbed
<code>vectors.distance</code>	Cosine	Standard per similarità semantica testuale
<code>use_sparse_embeddings</code>	true	Abilita la componente sparsa per retrieval ibrido
<code>recreate_index</code>	true (job batch)	Ricrea la collection ibrida mantenendo lo stesso nome
<code>hnsw_config.m</code>	16 (default)	Compromesso velocità/precisione
<code>hnsw_config.ef_construct</code>	100 (default)	Qualità dell'indice in fase di build

Ogni punto nella collection porta un payload JSON con i campi elencati nella Tabella 2.9. I campi marcati come indicizzati abilitano il pre-filtering, caratteristica distintiva di Qdrant che applica i filtri sul payload *prima* della ricerca vettoriale, garantendo che il calcolo del top-*K* sia effettuato solo sul sottoinsieme filtrato.

Gli ID dei documenti sono deterministici nel formato `guild-{guild_id}-chunk-{index}`. La reindicizzazione della stessa Gilda produce quindi gli stessi identificativi e consente a Qdrant di sovrascrivere i punti esistenti quando la policy è `DuplicatePolicy.OVERWRITE`.

Tabella 2.9: Struttura del payload Qdrant per la collection `i3guild_knowledge`.

Campo	Tipo	Descrizione	Ind.
<code>guild_id</code>	integer	ID della gilda sorgente	Sì
<code>guild_name</code>	keyword	Nome della gilda	Sì
<code>epoch_id</code>	integer	ID dell'epoca di appartenenza	Sì
<code>epoch_start</code>	datetime	Data inizio epoca	Sì
<code>epoch_end</code>	datetime	Data fine epoca (nullable)	Sì
<code>participation_mode</code>	keyword	FullRemote / Hybrid / OnSite	Sì
<code>is_individual</code>	bool	Gilda individuale	Sì
<code>owner_id</code>	keyword	ID del proprietario	Sì
<code>participants</code>	keyword[]	Nomi dei partecipanti	Sì
<code>participant_count</code>	integer	Numero di partecipanti	No
<code>source_text</code>	text	Testo originale del chunk	No
<code>last_synced</code>	datetime	Timestamp ultima sincronizzazione	No

2.5.3 Pipeline di ingestione ETL

La pipeline ETL ha il compito di mantenere allineata la rappresentazione vettoriale con i dati applicativi. Il job batch esegue le seguenti fasi:

1. **Extract:** connessione a PostgreSQL e query SQL con JOIN su `Epoch`, `GuildUserRelation` ed `ExternalUser` per estrarre tutti i campi semantici delle Gilde insieme ai nomi dei partecipanti.
2. **Transform:** normalizzazione HTML tramite `BeautifulSoup`, composizione di documenti Haystack con sezioni marcate e chunking con overlap quando il testo supera il limite configurato.
3. **Embed:** generazione di embedding densi tramite `FastembedDocumentEmbedder` e di embedding sparsi tramite `FastembedSparseDocumentEmbedder`.
4. **Load:** scrittura su Qdrant tramite `DocumentWriter` con policy configurabile, di default `DuplicatePolicy.OVERWRITE`.

La struttura della pipeline Haystack è riportata in Appendice B, Listato B.1. Nel capitolo principale è sufficiente evidenziare la sequenza logica Extract–Transform–Embed–Load, perché il dettaglio del codice non modifica la scelta progettuale.

Il container della pipeline mantiene comportamento batch: esegue ingestione, upsert su Qdrant e termina con codice 0 in caso di successo o 1 in caso di errore. Questa separazione evita che indicizzazione e query interattive competano nello stesso processo.

2.5.4 Pipeline di retrieval e logica di ranking

La pipeline di retrieval usa l'indice ibrido Qdrant per recuperare i documenti più pertinenti alla query e restituisce al livello applicativo un contesto già filtrato. La query viene trasformata sia in embedding denso sia in embedding sparso; le due rappresentazioni alimentano `QdrantHybridRetriever`. Il sistema non adotta un valore `top.k` fisso: una soglia rigida rischierebbe infatti di includere documenti marginali nelle query semplici o di tagliare informazioni utili nelle query più ampie. Per questo viene applicato un approccio adattivo in tre stadi:

1. **Soglia di similarità** (default: cosine score ≥ 0.5): esclude documenti semanticamente non pertinenti.
2. **Rilevamento del calo relativo**: se la caduta percentuale tra due score consecutivi supera una soglia configurabile (default: 0.08), la lista viene troncata al confine naturale della rilevanza.
3. **Cap di sicurezza** (default: ≤ 5 documenti): limite massimo per controllare la dimensione del contesto passato al modello generativo.

Prima del taglio finale viene applicata una deduplicazione post-retrieval: il servizio rimuove documenti ripetuti per ID, fingerprint testuale e similarità fuzzy. Una cache in-process con TTL breve riduce inoltre richieste ripetute con stessi parametri, senza cambiare il modello dati persistente.

Query rewriting e Topic-First Search

Nel contesto i3Guild, molte query contengono parole ricorrenti come “gilda”, “fondare” o “consigli”. Questi termini descrivono l’interazione con il sistema, ma non il tema informativo cercato; se mantenuti nella query, possono dominare lo spazio degli embedding e produrre risultati generici. Il *query rewriting*, discusso anche in framework come RQ-RAG [9], viene quindi utilizzato per isolare il topic rilevante prima del retrieval. L’approccio implementato tramite la funzione `extract_topic_query()` opera in tre passaggi:

1. Normalizzazione e rimozione della punteggiatura.
2. Filtraggio di un insieme di circa 80 *domain stopwords* (DOMAIN_STOPWORDS): termini specifici del dominio i3Guild e stopwords italiane generiche.
3. Estrazione del topic residuo; se diverso dalla query originale, la ricerca viene eseguita sul topic pulito (*topic-first strategy*).

La Tabella 2.10 illustra l’effetto del rewriting su query campione.

Tabella 2.10: Effetto del query rewriting su esempi di query utente.

Query originale	Topic estratto	Risultato atteso
“vorrei fondare una gilda sulla musica, cosa mi consigli?”	musica	Gilda a tema musicale
“intelligenza artificiale”	intelligenza artificiale	(nessun rewriting)
“chi ha partecipato al progetto di design?”	design	Gilda con partecipanti nel settore design

2.5.5 Endpoint API di riferimento

Il risultato della pipeline viene esposto tramite un endpoint FastAPI, che costituisce il contratto tra il servizio RAG e l’applicazione Next.js. L’endpoint principale è il seguente:

POST /rag/retrieve

Un esempio di request body e la struttura della response sono riportati in Appendice B, Listati B.2 e B.3.

I campi `total_candidates`, `threshold_used`, `cutoff_reason` e `topic_query` (presente solo quando il rewriting è attivo) sono utili per il debug, per il controllo applicativo e per eventuali verifiche successive della qualità del retrieval.

2.6 Integrazione con i3Guild

L'integrazione del sistema RAG nell'infrastruttura i3Guild esistente è pensata per ridurre l'impatto sul sistema già operativo. PostgreSQL, Next.js, Keycloak e Kong mantengono il proprio ruolo; il sistema RAG si aggiunge come insieme di servizi containerizzati. Il retrieval semantico viene così introdotto senza modificare il modello dati principale né il flusso di autenticazione.

2.6.1 Servizi Docker aggiuntivi

La Tabella 2.11 riassume i servizi coinvolti dall'integrazione RAG. Il dettaglio delle porte host e delle immagini Docker è lasciato alla configurazione operativa, perché non incide sulla scelta architetturale.

Tabella 2.11: Servizi Docker introdotti dall'integrazione RAG in i3Guild.

Servizio	Responsabilità	Note progettuali
Qdrant	Database vettoriale ibrido	Conserva vettori densi, rappresentazioni sparse e payload filtrabili.
Ollama Servizio RAG	Inferenza LLM locale API FastAPI e pipeline Haystack	Serve il modello di chat <code>gemma3:4b</code> . Espone retrieval, chat e generazione controllata verso Next.js.
Pipeline embedding	Job batch di indicizzazione	Legge PostgreSQL, produce embedding densi e sparsi e aggiorna Qdrant.
PostgreSQL	Sorgente autoritativa	Rimane il database applicativo principale, senza duplicare la logica di dominio.

2.6.2 Variabili d'ambiente

2.6.3 Integrazione lato Next.js

Lato applicazione Next.js, l'integrazione è concentrata in un client TypeScript dedicato. Il client espone funzioni per retrieval, risposta sincrona e streaming, inoltra al microservizio il modello LLM configurato e supporta opzioni applicative come `use_rag`, `json_mode`, timeout client e parametri di generazione. In caso di errore del retrieval, la ricerca può degradare restituendo un insieme vuoto; nei flussi di generazione strutturata, invece, l'errore viene propagato al chiamante per attivare fallback applicativi espliciti.

Tabella 2.12: Configurazione esterna della componente RAG.

Gruppo di variabili	Ruolo
Connessione Qdrant	Endpoint del database vettoriale e nome della collection principale.
Modello LLM	Endpoint Ollama e modello di chat configurabile, con gemma3:4b come scelta adottata nel prototipo.
Embedding	Modelli FastEmbed denso e sparso, dimensione del vettore denso e directory di cache locale.
Retrieval	Parametri di soglia, numero massimo di documenti, cache in-process e opzioni di query rewriting.
Integrazione applicativa	Endpoint interno del servizio RAG usato da Next.js e timeout delle chiamate di generazione.

2.6.4 Sincronizzazione dei dati

PostgreSQL rimane la sorgente autoritativa; Qdrant è una vista derivata e deve essere mantenuto sincronizzato con i dati applicativi. Sono state considerate tre strategie, in ordine crescente di complessità: re-indicizzazione completa, sincronizzazione incrementale e aggiornamento event-driven. Per il prototipo la strategia raccomandata è il **full re-index periodico**: il job di embedding ricrea la collection di default e riscrive i documenti con policy di overwrite. La scelta è semplice da implementare, riduce il rischio di inconsistenze tra indici densi e sparsi e rimane sostenibile perché l'embedding è eseguito localmente, senza costi a consumo per token. L'evoluzione verso un *sync incrementale* basato sul campo `@updatedAt` di Prisma, o verso un meccanismo *event-driven* attivato dalle operazioni CRUD, è lasciata come sviluppo futuro.

2.7 Sintesi del capitolo

Il capitolo ha trasformato il caso d'uso i3Guild in una proposta architetturale coerente con i vincoli aziendali. Il corpus delle Gilde è abbastanza contenuto da rendere praticabile una reindicizzazione completa, ma abbastanza ricco da richiedere una rappresentazione semantica e metadati filtrabili. Da qui deriva la scelta di Qdrant come store ibrido, FastEmbed/Haystack per embedding densi e sparsi, Ollama per l'inferenza locale e FastAPI come confine applicativo tra retrieval e frontend.

Le decisioni principali restano guidate da un criterio pratico: mantenere i dati nel perimetro aziendale e usare componenti sostituibili, senza introdurre complessità non necessaria. Il modello di chat **gemma3:4b**, la soglia di similarità pari a 0.5 e il limite di 5 documenti recuperati riflettono questo equilibrio tra qualità del contesto, capacità del modello e costo operativo del prompt. Il capitolo successivo descrive come queste scelte siano state tradotte in componenti eseguibili.

Capitolo 3

Implementazione del Sistema RAG

Il capitolo descrive l'implementazione del sistema RAG integrato in i3Guild. L'obiettivo non è ripetere le scelte architetturali già motivate nel Capitolo 2, ma mostrare come tali scelte siano state tradotte in componenti eseguibili: una pipeline di ingestione per popolare Qdrant, un microservizio FastAPI per il retrieval e la generazione, un'integrazione applicativa lato Next.js e una generazione controllata delle bozze di nuove Gilde. Durante lo sviluppo è stato anche implementato ed esplorato un workflow agentico; viene documentato come soluzione sperimentale, ma non come scelta finale, perché a parità di risorse hardware introduceva un costo computazionale più elevato e un vantaggio limitato rispetto a un flusso RAG più classico.

La realizzazione è distribuita su tre repository applicativi, ciascuno associato a una responsabilità precisa. Il repository principale i3Guild contiene l'applicazione web Next.js e il database PostgreSQL [64]; la pipeline di embedding gestisce l'estrazione dei dati, la normalizzazione testuale, la generazione di embedding densi e sparsi con FastEmbed/Haystack e l'upsert su Qdrant; il microservizio RAG espone infine le API Python per retrieval e chat. La separazione riduce l'accoppiamento tra applicazione e componente RAG: i3Guild rimane la sorgente autoritativa dei dati, Qdrant conserva la knowledge base ibrida e Ollama viene usato solo per la generazione LLM [68, 59].

3.1 Struttura dei repository e responsabilità

La suddivisione dei moduli segue il flusso dei dati. PostgreSQL contiene le Gilde, le epoche e le relazioni con gli utenti; la pipeline di embedding legge queste informazioni e produce documenti Haystack [50]; Qdrant memorizza vettori densi, rappresentazioni sparse e metadati; il microservizio RAG interroga l'indice ibrido e, quando richiesto, invoca Ollama per generare la risposta. L'applicazione Next.js consuma il servizio tramite un client HTTP dedicato [73].

L'applicazione web contiene il client TypeScript che dialoga con il microservizio RAG e gestisce lo streaming verso l'interfaccia. La pipeline di embedding svolge il lavoro batch: estrae i dati da PostgreSQL, normalizza i testi, produce embedding densi e sparsi e aggiorna Qdrant. Il microservizio Python concentra invece retrieval, ranking, chiamate a Ollama e formato di risposta. La parte agentica è rimasta isolata dal flusso principale: è stata utile per valutare una generazione più guidata della proposta di Gilda, ma non è stata adottata come architettura finale.

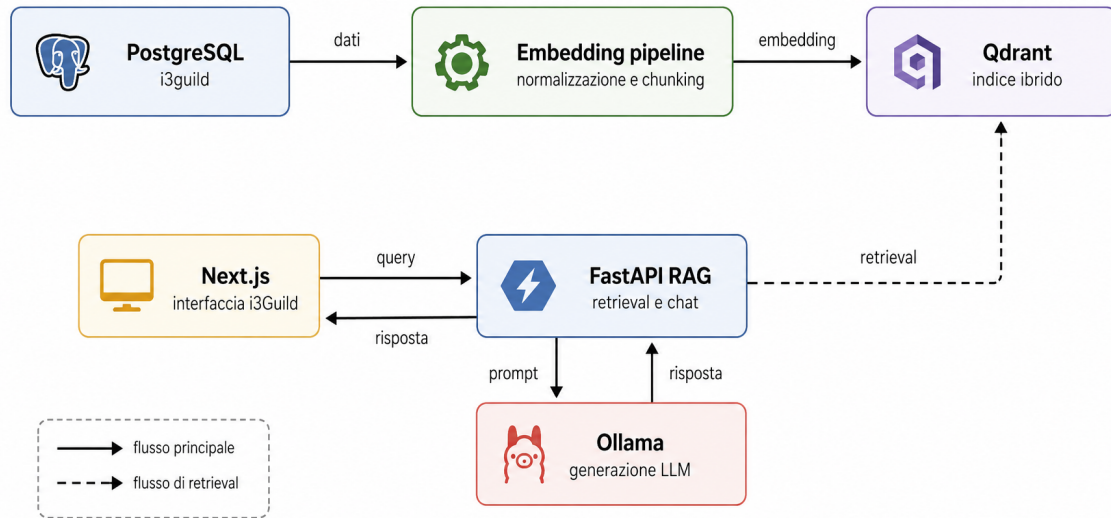


Figura 3.1: Flusso implementativo end-to-end tra sorgente dati, indicizzazione ibrida, retrieval e generazione.

Mantenere l'ingestione fuori dal servizio di query evita che operazioni batch e richieste interattive competano nello stesso processo. La pipeline di embedding può essere eseguita manualmente, schedata o avviata in un container dedicato; il servizio RAG rimane invece ottimizzato per rispondere a richieste HTTP a bassa latenza. La containerizzazione completa questa separazione, isolando dipendenze e servizi di supporto [51].

3.2 Pipeline di ingestione

La pipeline di ingestione viene avviata come job batch: verifica le dipendenze, legge le sorgenti dati, produce documenti Haystack, genera embedding densi e sparsi tramite FastEmbed e scrive i risultati in Qdrant [50, 68]. In caso di errore in una dipendenza essenziale, ad esempio PostgreSQL o Qdrant non raggiungibile, il processo termina con codice diverso da zero. Questo comportamento è utile in ambiente Docker o CI, perché rende immediatamente visibile il fallimento dell'indicizzazione [51].

3.2.1 Configurazione e controlli preliminari

La configurazione avviene tramite variabili d'ambiente, così da mantenere lo stesso container riutilizzabile in ambienti diversi. I parametri principali definiscono la connessione a PostgreSQL, l'endpoint e la collection Qdrant, i modelli FastEmbed e la dimensione del vettore denso. A questi si aggiungono opzioni operative per ricreare l'indice, controllare la dimensione dei batch e stabilire quando applicare il chunking con overlap. La generazione degli embedding usa i modelli riportati nella Tabella 2.7 e rimane quindi separata da Ollama.

Tabella 3.1: Parametri principali della pipeline di ingestione.

Variabile	Ambito	Uso nell'implementazione
DB_URL	PostgreSQL	Connessione alla sorgente autoritativa delle Gilde.
QDRANT_URL	Qdrant	Endpoint del vector database usato in scrittura.
QDRANT_COLLECTION_NAME	Qdrant	Nome della collection da creare o aggiornare.
FASTEMBED_DENSE_MODEL	FastEmbed	Modello usato per gli embedding densi.
FASTEMBED_SPARSE_MODEL	FastEmbed	Modello usato per gli embedding sparsi.
FASTEMBED_CACHE_DIR	FastEmbed	Directory locale per la cache dei modelli.
EMBEDDING_DIMENSIONS	Indice vettoriale	Dimensione dei vettori densi salvati in Qdrant.
RECREATE_INDEX	Operatività	Ricrea la collection; il default operativo del job batch è <code>true</code> .
INGEST_PIPELINE_BATCH_SIZE	Operatività	Numero di documenti elaborati per run Haystack.
INGEST_MAX_DOCUMENT_CHARS	Chunking	Limite oltre il quale un documento viene suddiviso.

Prima dell'ingestione vengono eseguiti controlli sulle configurazioni essenziali: connessione a PostgreSQL, configurazione FastEmbed, dimensione del vettore denso e raggiungibilità della collection Qdrant tramite il document store Haystack. La collection viene creata con distanza coseno, dimensione pari a `EMBEDDING_DIMENSIONS` e `use_sparse_embeddings=true`. Il valore usato per la componente densa è 768.

3.2.2 Estrazione dei dati da PostgreSQL

La fase di estrazione recupera le informazioni semantiche delle Gilde tramite una query SQL con join su `Epoch`, `GuildUserRelation` ed `ExternalUser`. Il risultato include i campi testuali principali della Gilda, i riferimenti all'epoca, la modalità di partecipazione, l'indicazione di Gilda individuale e la lista dei partecipanti. Il listato completo dei campi estratti è riportato in Appendice B, Listato B.4.

La query reale include anche una sottoquery per aggregare i partecipanti, combinando utenti interni ed esterni. Questo passaggio è importante perché i nomi dei partecipanti sono utili sia come metadati filtrabili sia come contesto testuale per le domande dell'utente.

3.2.3 Normalizzazione e costruzione dei documenti

Ogni riga estratta viene trasformata in uno o più documenti Haystack. Il processo di costruzione compone un testo articolato in sezioni riconoscibili, dedicate a sommario, obiettivi, rischi, risultati raggiunti e partecipanti. I campi HTML vengono normalizzati con BeautifulSoup [71]: il codice rimuove i tag, conserva il contenuto testuale e compatta gli spazi. In questa fase non interessa ottenere una resa editoriale del documento, ma una rappresentazione pulita e stabile, adatta al

retrieval. Uno schema semplificato della costruzione del documento è riportato in Appendice B, Listato B.5.

La strategia descritta nel Capitolo 2 prevedeva un documento consolidato per Gilda. L'implementazione mantiene questa impostazione come caso normale, ma introduce una soglia di sicurezza: `INGEST_MAX_DOCUMENT_CHARS`. Se il testo supera il limite, il documento viene diviso in chunk con sovrapposizione controllata. Gli outlier non impediscono così l'ingestione e i chunk successivi mantengono il riferimento alla Gilda tramite l'intestazione. La granularità dei chunk condiziona qualità del retrieval e quantità di contesto disponibile per il modello generativo [10].

Gli identificativi dei documenti sono deterministici: `guild-{guild_id}-chunk-{index}`. La scelta rende idempotente la reindicizzazione: rieseguire la pipeline sulla stessa Gilda produce gli stessi ID e permette a Qdrant di sovrascrivere i punti esistenti invece di duplicarli. I metadati associati includono `guild_id`, `guild_name`, `epoch_id`, `is_individual`, `participation_mode`, `participants`, `chunk_index` e `chunk_count`.

3.2.4 Embedding e scrittura in Qdrant

La fase finale costruisce una pipeline Haystack composta da un embedder denso, un embedder sparso e un writer. I due embedder arricchiscono gli stessi documenti con rappresentazioni complementari: la prima orientata alla similarità semantica, la seconda al matching lessicale. Il writer persiste poi entrambi i segnali nel document store Qdrant. La configurazione essenziale della pipeline è riportata in Appendice B, Listato B.6.

La pipeline viene eseguita in batch applicativi, con una dimensione configurabile tramite variabile d'ambiente. L'elaborazione a batch riduce il picco di memoria dello sparse embedder e mantiene il comportamento atteso del container: il job parte, scrive su Qdrant e termina. La normalizzazione dei campi avviene prima della costruzione dei documenti, rendendo esplicito quali testi entrano nella knowledge base.

3.3 Microservizio RAG

Il microservizio RAG è implementato con FastAPI ed espone endpoint per retrieval, chat sincrona e chat in streaming. La configurazione è centralizzata nel modulo delle impostazioni e usa `pydantic-settings`; i valori di default permettono lo sviluppo locale, mentre gli ambienti Docker possono sovrascriverli tramite variabili d'ambiente [52, 65].

Tabella 3.2: Endpoint principali esposti dal microservizio RAG.

Endpoint	Formato	Responsabilità
POST <code>/rag/retrieve</code>	JSON	Recupera documenti semanticamente pertinenti e restituisce metadati di diagnostica sul taglio dei risultati.
POST <code>/chat/reply</code>	JSON	Esegue retrieval, costruisce il prompt aumentato e restituisce una risposta completa.
POST <code>/chat/stream</code>	NDJSON	Espone la chat incrementale, separando eventi RAG, token, warning ed errori.

3.3.1 Avvio, middleware e gestione errori

L'applicazione FastAPI registra CORS, rate limiting e handler globale delle eccezioni. Il rate limiter usa `slowapi` con limite predefinito di 100 richieste al minuto per IP sugli endpoint principali. La gestione degli errori distingue i fallimenti dovuti a dipendenze non raggiungibili, come Qdrant o Ollama, dagli errori applicativi generici: nel primo caso il servizio restituisce 503, nel secondo 500.

Per ridurre il rischio operativo in ambienti con risorse limitate, il servizio controlla la memoria disponibile prima delle chiamate a modelli locali. La funzione `get_model_memory_requirement()` stima il requisito RAM in base al nome del modello; `get_ram_warning()` produce un warning se la memoria libera è inferiore alla soglia. Il warning non blocca la richiesta, ma viene propagato nella risposta quando disponibile.

3.3.2 Endpoint di retrieval

L'endpoint `POST /rag/retrieve` riceve una query, i parametri di retrieval e gli eventuali filtri applicativi. Il modello Pydantic `RetrieveRequest` definisce valori di default conservativi: `top_k=20`, `score_threshold=0.5`, `max_documents=5` e `gap_threshold=0.08`. La risposta include i documenti selezionati e una serie di informazioni diagnostiche: numero di candidati iniziali, soglia usata, motivo del taglio, eventuale query riscritta e filtri estratti automaticamente. Lo schema della richiesta è riportato in Appendice B, Listato B.7.

L'endpoint è volutamente sottile: valida l'input, esegue il controllo RAM e delega la logica al livello applicativo del retrieval. Questa separazione rende più semplice testare il retrieval senza dover passare da FastAPI.

3.3.3 Pipeline Haystack di query

La pipeline di query collega il document store Qdrant agli embedder FastEmbed usati per la query. La collection usa embedding densi di dimensione 768 con similarità coseno e abilita anche la componente sparsa. La query utente viene quindi trasformata in due rappresentazioni, una densa e una sparsa, che alimentano il retriever ibrido Qdrant [50, 68]. Il listato della pipeline è riportato in Appendice B, Listato B.8.

La pipeline è inizializzata in modo lazy e protetta da un lock. Alla prima richiesta viene creato l'oggetto Haystack e vengono predisposti gli indici sui payload Qdrant per i campi usati più spesso nei filtri: `epoch_id`, `is_individual`, `participation_mode`, `participants` e `guild_name`. Le chiamate successive riusano la stessa pipeline in memoria. La ricerca vettoriale su Qdrant usa indici approssimati per nearest neighbor; HNSW è il riferimento adottato per questa classe di problemi [26].

3.4 Logica di retrieval

La logica di retrieval contiene la parte più specifica del sistema. Il retrieval non è una semplice chiamata a Qdrant: prima della ricerca la query viene normalizzata e, quando possibile, riscritta; dopo la ricerca i risultati vengono deduplicati e filtrati con una logica dinamica. Questa struttura segue il principio RAG di separare conoscenza parametrica e recupero esplicito di documenti esterni [24].

3.4.1 Riscrittura della query e filtri

Il rewriting rimuove parole frequenti nel dominio i3Guild, come "gilda", "fondare" o "consigli", quando non contribuiscono al tema ricercato. Se dalla frase "vorrei fondare una gilda sulla musica" viene estratto il topic "musica", la ricerca viene eseguita su tale topic. La risposta conserva la query riscritta nel campo `topic_query`, così il frontend può mostrarla o usarla per diagnostica.

In parallelo, un estrattore di filtri cerca vincoli espressi in linguaggio naturale: epoca, Gilde individuali o di gruppo, modalità di partecipazione e partecipanti. Se l'utente passa anche filtri espliciti, il servizio li combina con quelli estratti tramite un operatore `AND`. Questo permette di usare la ricerca vettoriale senza perdere vincoli strutturati presenti nei metadati.

3.4.2 Cache e deduplicazione

Per evitare interrogazioni ripetute identiche, il servizio mantiene una cache in memoria con TTL configurabile. La chiave include query effettiva, parametri di retrieval e filtri. Si tratta di una cache locale al processo, sufficiente per lo scenario prototipale; in un deployment multi replica andrebbe sostituita con una cache condivisa o disabilitata per evitare differenze tra istanze.

Dopo il recupero da Qdrant, il servizio rimuove duplicati su tre livelli. Prima controlla l'ID del documento, poi usa una fingerprint testuale normalizzata e infine confronta i testi tramite similarità fuzzy. La soglia fuzzy è configurabile con `rag_dedup_fuzzy_threshold`. Questa parte è stata aggiunta perché il corpus può contenere documenti equivalenti provenienti da sorgenti diverse o chunk molto simili generati dalla sovrapposizione.

3.4.3 Selezione dinamica dei documenti

La selezione finale ordina i documenti per score, applica la soglia minima e poi taglia la lista quando il calo relativo tra due score consecutivi supera `gap_threshold`. Se il numero di risultati resta troppo alto, viene applicato `max_documents`. La risposta indica il motivo del taglio: `threshold`, `relative_gap`, `max_cap` o `exhausted`. Uno schema della logica di taglio è riportato in Appendice B, Listato B.9.

L'approccio evita di fissare a priori un unico valore di `top_k`. Per query molto specifiche il sistema restituisce pochi documenti, mentre per query più ampie può mantenere più contesto entro il limite configurato. Il risultato è più stabile per l'utente e riduce la probabilità di passare al modello generativo documenti debolmente pertinenti. La riduzione del contesto non pertinente contribuisce inoltre a mitigare risposte non fondate sulle fonti recuperate [30].

3.5 Generazione con Ollama

Il servizio di generazione gestisce le chiamate al modello di chat. Espone sia risposte complete sia risposte in streaming, usando in entrambi i casi l'endpoint `/api/chat` di Ollama [59].

3.5.1 Risoluzione del modello

Il modello configurato in `OLLAMA_LLM_MODEL` viene confrontato con la lista dei modelli installati ottenuta da `/api/tags`. Il codice accetta sia corrispondenze esatte sia varianti con tag, ad esempio

`:latest`. Se la configurazione è ambigua o il modello non è installato, il servizio restituisce un errore esplicito. Questa verifica evita fallimenti meno leggibili durante la generazione.

La lista dei modelli disponibili viene mantenuta in cache per pochi secondi, così il servizio non interroga Ollama a ogni richiesta. La cache è locale al processo e ha solo funzione di riduzione dell'overhead.

3.5.2 Costruzione del prompt aumentato

Prima di inviare la richiesta a Ollama, il servizio costruisce la lista dei messaggi. Il system prompt, se presente, viene aggiunto per primo. La cronologia conversazionale viene limitata agli ultimi `max_history_messages`, in modo da controllare la dimensione del contesto. Se il retrieval ha restituito documenti, il testo viene aggiunto al messaggio utente dentro una sezione esplicita: `--- CONTESTO RAG ---`. L'aumento del prompt con documenti recuperati è il meccanismo con cui RAG rende disponibili al modello informazioni esterne senza modificare i pesi [24]. Il formato del messaggio aumentato è riportato in Appendice B, Listato B.10.

Ogni documento viene troncato a `rag_max_chars_per_doc` caratteri. Il troncamento è intenzionale: una singola Gilda molto lunga non deve consumare tutta la finestra di contesto né impedire al modello di vedere altre fonti.

3.5.3 Streaming NDJSON

Per la chat interattiva il servizio espone `POST /chat/stream`. Prima invia un evento `rag` con i metadati del retrieval, poi inoltra gli eventi generati da Ollama. La risposta HTTP usa il media type `application/x-ndjson`. NDJSON significa *Newline Delimited JSON*: la risposta non è un singolo array JSON, ma una sequenza di oggetti JSON separati da newline. Ogni riga è quindi completa e parsabile in modo indipendente. Questa proprietà è utile nello streaming, perché il frontend può processare subito un evento `rag`, un token, un warning o un errore senza attendere la chiusura dell'intera risposta. Il formato è anche più robusto di un JSON parziale: se una generazione viene interrotta, gli eventi già ricevuti restano validi. Un esempio di sequenza NDJSON è riportato in Appendice B, Listato B.11.

3.6 Workflow agentic

Durante il progetto è stata esplorata anche una modalità agentic per guidare l'utente nella proposta di nuove Gilde. Questa linea non costituisce però il flusso finale del prototipo: il servizio RAG rimane centrato su retrieval ibrido, chat e generazione controllata, mentre la compilazione della bozza viene gestita lato applicazione attraverso prompt strutturati e validazione del risultato.

La scelta è coerente con i vincoli osservati durante lo sviluppo. Un agente che mantiene stato, pianifica passaggi e invoca più volte il modello introduce maggiore latenza e maggiore consumo di memoria, soprattutto con LLM locali di piccole dimensioni. Il flusso finale privilegia quindi un approccio più deterministico: il retrieval recupera contesto quando serve; la generazione strutturata viene attivata solo per produrre una bozza; la validazione resta nel codice applicativo. La componente agentic rimane una possibile direzione futura, da rivalutare con modelli locali più capaci o con hardware più ampio.

3.7 Integrazione lato Next.js

L'applicazione `i3Guild` accede al microservizio tramite un client TypeScript dedicato. Il client centralizza l'URL del servizio, legge `I3GUILD_RAG_URL` o `RAG_API_URL`, serializza le richieste e normalizza gli errori. Le chiamate HTTP dirette al servizio RAG rimangono così concentrate in un unico punto, invece di essere replicate in più componenti React o route API [73].

3.7.1 Client di retrieval

Il client di retrieval invoca `/rag/retrieve` e restituisce sia documenti sia metadati. In caso di errore non solleva un'eccezione: restituisce una lista vuota e un `cutoff_reason` pari a `error`. Per la sola ricerca semantica questo comportamento è corretto: il chatbot non deve bloccarsi se il servizio Python è temporaneamente non raggiungibile. La chiamata TypeScript essenziale è riportata in Appendice B, Listato B.14.

Una seconda interfaccia, più semplice, restituisce solo i documenti. Viene mantenuta per i punti dell'applicazione che non hanno bisogno della diagnostica.

3.7.2 Client chat e generazione controllata

Per la generazione sono disponibili `generateRAGChatReply()`, `generateRAGChatStream()` e le funzioni di retrieval. La prima usa l'endpoint non streaming `/chat/reply`; la seconda restituisce direttamente la `Response`, lasciando al chiamante la lettura incrementale dello stream NDJSON. Il client può inoltrare il modello LLM configurato, attivare o disattivare il retrieval con `use_rag`, richiedere JSON mode e passare opzioni di generazione.

Questa interfaccia consente di riusare lo stesso microservizio in due modi distinti: chat RAG, con retrieval attivo, e generazione strutturata, in cui il prompt contiene già tutto il contesto necessario e il retrieval viene saltato. La distinzione riduce accoppiamento tra caso d'uso conversazionale e generazione di bozze.

3.8 Generazione della bozza di Gilda

Accanto alla chat RAG, `i3Guild` include un flusso per generare una bozza di Gilda a partire dalla conversazione. La funzionalità è documentata come *Guild Draft Generation* e si attiva dal pulsante "Proponi bozza" nella chat. Il flusso non crea direttamente una nuova Gilda e non invia l'utente alla pagina di creazione: produce invece un oggetto strutturato che rappresenta un draft, modificabile e verificabile prima di qualunque salvataggio applicativo.

Il formato atteso è `GuildDraft`. Include nome, sommario, obiettivi, risultati attesi, rischi, opportunità, budget, risorse, journal, modalità di partecipazione, numero minimo e massimo di partecipanti, indicazione di Gilda individuale e avatar. La route Next.js `/api/guild/draft` recupera la conversazione, costruisce un prompt istruttivo, chiama il microservizio RAG con `use_rag=false` e `json_mode=true`, quindi prova a estrarre un JSON compatibile con lo schema. La validazione è quindi parte del contratto applicativo: una risposta testuale plausibile, ma non riconducibile allo schema atteso, non viene trattata come bozza valida.

La gestione degli errori è intenzionalmente esplicita. Se la conversazione non è sufficiente, se il modello restituisce JSON non valido o se mancano campi obbligatori, il sistema informa l'utente del problema e lo invita a riprovare, eventualmente aggiungendo dettagli più chiari nella conversazione.

Questo comportamento rende misurabile il tasso di fallimento della generazione: in una valutazione applicativa, tale tasso può essere espresso come rapporto tra bozze non valide e numero totale di richieste di generazione.

La route restituisce una risposta NDJSON in streaming. Durante l'elaborazione invia eventi di stato periodici e conclude con un evento **draft** oppure con un evento **error**. Gli eventi intermedi mantengono viva la connessione anche quando l'operazione LLM è lenta, evitando che un gateway con timeout aggressivo interrompa la richiesta prima che il frontend riceva un esito interpretabile.

3.9 Aspetti operativi

L'implementazione include alcune scelte operative che non cambiano la logica RAG ma ne migliorano l'affidabilità durante l'uso locale.

3.9.1 Timeout e dipendenze lente

Ollama può impiegare diversi secondi, o più di un minuto su CPU, per caricare un modello non ancora residente in memoria. Il problema si presenta soprattutto alla prima richiesta dopo l'avvio del container: il modello è installato, ma il processo deve ancora caricarlo e inizializzare l'inferenza. Per questo il servizio RAG usa `OLLAMA_REQUEST_TIMEOUT_SECONDS` per le chiamate sincrone e consente un override per richiesta tramite `timeout_seconds`. Nei flussi lunghi, un valore minore o uguale a zero disabilita il timeout HTTP verso Ollama.

La stessa attenzione serve lato Next.js. La generazione della bozza usa una risposta NDJSON streaming con heartbeat periodici, così la route rimane viva mentre il modello produce il JSON. Il frontend distingue eventi di stato, bozza valida ed errori, invece di trattare ogni ritardo come fallimento indistinto.

3.9.2 System prompt e modello derivato

Una seconda ottimizzazione operativa riguarda la gestione del system prompt. Nella configurazione standard il prompt di sistema viene inviato a ogni richiesta insieme alla conversazione e al contesto recuperato. Per ridurre la duplicazione del payload, il progetto considera anche la creazione di un modello Ollama derivato tramite l'endpoint `/api/create`, incorporando il system prompt nella definizione del modello. Il nome del modello derivato include un hash del prompt, così una modifica del prompt produce una nuova configurazione senza sovrascrivere quella precedente.

Questa soluzione migliora soprattutto la stabilità configurativa e riduce la quantità di testo serializzata nelle richieste. Non va però interpretata come un riuso effettivo della KV Cache del modello: il system prompt incorporato nella definizione del modello rende più pulita la configurazione, ma non elimina di per sé il costo computazionale del prompt durante l'inferenza.

La soluzione ha anche limiti operativi. Se il system prompt cambia spesso, i modelli derivati possono accumularsi nello storage di Ollama e richiedere una policy di pulizia. In un deployment multi replica, più processi potrebbero tentare di creare lo stesso modello derivato nello stesso momento; un lock locale è sufficiente nel prototipo, ma non basta come garanzia distribuita. Un miglioramento futuro consiste nel versionare esplicitamente i prompt, salvare metadati di creazione e usare una procedura di cleanup controllata. Un secondo miglioramento riguarda la misurazione: per le risposte non streaming Ollama espone metriche finali come token di prompt e durate, mentre nello streaming sarebbe utile emettere un evento conclusivo con le stesse statistiche. In questo modo

diventerebbe possibile confrontare payload, latenza e costo del prompt senza introdurre strumenti esterni.

3.9.3 Tracciabilità

Il servizio espone dati diagnostici in più punti: `cutoff_reason`, `topic_query`, `extracted_filters`, warning RAM, numero di candidati, documenti restituiti e log di timeout verso Ollama. Oltre al debug, questi campi permettono di ricostruire il comportamento del sistema durante la valutazione applicativa: quale query è stata effettivamente cercata, quanti documenti sono stati recuperati, perché alcuni risultati sono stati scartati e se il collo di bottiglia si trova nel retrieval o nella generazione.

3.10 Sintesi del capitolo

Il sistema implementato combina componenti relativamente semplici, ma separati con attenzione. La pipeline di embedding prepara l'indice ibrido a partire da PostgreSQL; il microservizio RAG espone retrieval e generazione; Qdrant gestisce vettori densi, rappresentazioni sparse e metadati; Ollama mantiene l'inferenza generativa in locale; Next.js integra chat, generazione della bozza e fallback applicativi. Il contributo implementativo risiede soprattutto nella normalizzazione del corpus i3Guild, nel retrieval ibrido con query rewriting, cache e deduplicazione, e nella separazione tra chat RAG e generazione strutturata senza retrieval. Il workflow agentico è stato trattato come una linea sperimentale e non come soluzione finale, perché a parità di hardware introduceva maggiore complessità e maggiore costo computazionale. Nel complesso, queste scelte rendono il prototipo coerente con i vincoli di privacy e di risorse discussi nei capitoli precedenti, lasciando aperti margini di evoluzione su sincronizzazione incrementale, agenti più flessibili e valutazione sistematica della qualità delle risposte.

L'esperienza aziendale porta anche a un problema più generale, che supera il solo caso i3Guild: quando un sistema RAG deve essere eseguito localmente, la qualità applicativa dipende dalla sostenibilità dell'inferenza. Non basta che il modello sia disponibile in locale; occorre capire se entra nella memoria disponibile, quanto costa caricarlo, quanto tempo impiega a produrre il primo token e quale throughput mantiene con prompt di lunghezza diversa. Da questa esigenza nasce la seconda parte della tesi. I Capitoli 4 e 5 isolano questo tema dal caso aziendale e lo studiano in modo sistematico su Apple Silicon, usando MLX, più modelli e più livelli di quantizzazione.

Capitolo 4

Disegno sperimentale per l'inferenza locale quantizzata

Questo capitolo definisce il protocollo usato per valutare l'inferenza locale di modelli linguistici open-weight su Apple Silicon. Dopo la progettazione e l'implementazione del sistema RAG aziendale, l'analisi si concentra su un vincolo piu' specifico: capire quali modelli generativi possano essere eseguiti su un MacBook Pro con chip M1 Pro e 16 GB di memoria unificata, e quanto la quantizzazione MLX contribuisca a renderli utilizzabili.

La scelta di separare l'esperimento dal prototipo RAG e' intenzionale. Un sistema RAG enterprise usa il modello generativo come ultimo stadio di una pipeline piu' ampia, ma le sue prestazioni dipendono prima di tutto dalla fattibilita' dell'inferenza locale: il modello deve essere convertibile, caricabile, stabile in generazione e compatibile con la memoria disponibile. Il presente capitolo misura quindi il comportamento del modello isolato. I risultati non sostituiscono una valutazione applicativa end-to-end del RAG, ma forniscono la base tecnica per capire quali famiglie e quali livelli di quantizzazione siano realistici nel perimetro hardware scelto.

La domanda sperimentale puo' essere formulata cosi': quali compromessi introduce la quantizzazione MLX su dimensione su disco, uso di memoria, latenza di caricamento, time-to-first-token, throughput di decodifica e qualita' su piccoli benchmark, quando modelli tra 3B e 14B parametri vengono eseguiti localmente su Apple Silicon consumer? Per rispondere serve un disegno controllato. Gli artefatti quantizzati devono derivare dagli stessi snapshot Hugging Face, il runtime deve rimanere costante, le impostazioni di generazione devono essere deterministiche e i fallimenti devono restare nell'analisi.

4.1 Motivazione e perimetro

L'inferenza locale e' rilevante per scenari aziendali in cui i dati non possono essere inviati liberamente a servizi cloud oppure in cui si vuole mantenere un prototipo riproducibile senza dipendenze da API esterne. Questa motivazione, tuttavia, non implica automaticamente che un modello locale sia utilizzabile. Su hardware consumer il vincolo piu' stretto e' la memoria: oltre ai pesi del modello, devono coesistere sistema operativo, runtime Python, librerie MLX, tokenizer, KV cache, prompt, buffer intermedi e altri processi applicativi.

Il lavoro si concentra sull’esecuzione MLX locale. RAG, GGUF, llama.cpp e workflow Ollama non rientrano nel perimetro operativo dell’esperimento principale. Sono componenti rilevanti per il contesto applicativo e per la letteratura sull’inferenza locale, ma aggiungerebbero altre variabili: formato dei pesi, kernel, template di chat, politiche di memoria e server runtime. Per questo il confronto rimane interno a MLX, con il livello di quantizzazione come variabile controllata.

Il contributo sperimentale non propone un nuovo metodo di quantizzazione. Costruisce invece una catena riproducibile di conversione, quantizzazione e benchmark locale su Apple Silicon, e misura come cambiano le metriche di sistema passando da FP16 a Q8, Q6 e Q4. Questa impostazione è coerente con gli studi recenti su Apple Silicon, che mostrano come dimensione del modello, bandwidth di memoria, costo di dequantizzazione e backend Metal interagiscano in modo non banale [5, 6].

4.2 Riferimenti scientifici e tecnici

La letteratura usata per progettare l’esperimento serve a due scopi. Da un lato indica quali metriche osservare e quali ipotesi evitare; dall’altro colloca Apple Silicon rispetto ad altre piattaforme di inferenza locale. Il confronto con questi lavori è metodologico: aiuta a scegliere misure come memoria, TTFT, throughput e stabilità del runtime, ma non viene usato per confrontare direttamente valori numerici ottenuti su hardware e modelli diversi.

La quantizzazione è solo una delle tecniche disponibili per comprimere modelli linguistici. Le survey recenti distinguono quantizzazione, pruning, distillazione e architetture compatte come famiglie diverse di intervento [35]. Questa tesi isola la quantizzazione post-training dei pesi per una ragione pratica: si applica ad artefatti già addestrati, non richiede fine-tuning, si integra nel workflow `mlx-lm` e agisce sul vincolo dominante del caso di studio, cioè la memoria disponibile sul dispositivo locale.

Il riferimento più diretto è *Benchmarking and Characterization of Large Language Model Inference on Apple Silicon*, che misura memoria, prestazioni, costo ed energia per LLM eseguiti su hardware Apple e NVIDIA [5]. Per questa tesi conta soprattutto un dato: la memoria unificata di Apple Silicon può rendere eseguibili modelli che su GPU consumer sarebbero limitati dalla VRAM, ma non elimina i colli di bottiglia di bandwidth e dequantizzazione. Il preprint *Profiling Large Language Model Inference on Apple Silicon: A Quantization Perspective* approfondisce lo stesso aspetto e mostra che ridurre i bit dei pesi non garantisce una latenza minore [6].

Il report *Production-Grade Local LLM Inference on Apple Silicon* confronta MLX, MLC-LLM, Ollama, llama.cpp e PyTorch MPS, e indica MLX come runtime competitivo su hardware Apple recente [28]. Questo risultato sostiene la scelta MLX-first, ma non dimostra una superiorità generale: il setup della tesi usa un M1 Pro con 16 GB, non una macchina Apple di fascia alta. Open-TQ-Metal affronta invece il problema della KV cache e dell’attenzione in dominio compresso per contesti molto lunghi [34]. Il tema rimane fuori perimetro, ma chiarisce che la sola quantizzazione dei pesi non esaurisce il problema dell’inferenza locale. Infine, *Silicon Showdown* evidenzia il compromesso tra throughput grezzo delle GPU discrete e capacità di memoria/efficienza energetica di Apple Silicon [20].

Nel complesso, questi riferimenti portano a tre scelte di disegno. Le metriche prestazionali vengono separate, perché memoria, latenza iniziale e throughput di decodifica non descrivono lo stesso fenomeno. La quantizzazione viene valutata per variante, senza assumere che meno bit significhino sempre più velocità. Il confronto resta infine interno a MLX, così le differenze osservate dipendono il più possibile da modello e precisione dei pesi, non dal cambio di runtime.

4.3 Hardware e runtime

L'ambiente di misura e' un Apple MacBook Pro con chip M1 Pro, 16 GB di memoria unificata, macOS e backend Metal. Non e' una macchina di fascia massima, ma e' un target realistico per valutare prototipi locali senza workstation dedicate. Il vincolo dei 16 GB fa parte del disegno sperimentale: un modello che funziona solo su configurazioni con molta piu' memoria non risponde alla domanda di ricerca della tesi.

4.3.1 Apple Silicon come system-on-chip

Apple Silicon indica la famiglia di processori progettati da Apple per i propri dispositivi Mac e mobili. A differenza di una configurazione desktop tradizionale, in cui CPU, GPU discreta, memoria video e altri controller possono essere componenti separati, Apple Silicon adotta un'impostazione da *system-on-chip*. Nel caso di M1 Pro, sullo stesso chip o nello stesso package convivono CPU, GPU, Neural Engine, controller di memoria, motori multimediali, Secure Enclave e controller di I/O [42]. Questa integrazione riduce la distanza fisica e logica tra sottosistemi che, in altre architetture, comunicano attraverso bus e copie esplicite di memoria.

Per questa tesi non conta la presenza di acceleratori in senso generico, ma il modo in cui CPU e GPU condividono il carico dell'inferenza. Un LLM locale legge ripetutamente i pesi, esegue operazioni matriciali, gestisce il tokenizer, alloca la KV cache, esegue il prefill del prompt e poi procede con la decodifica autoregressiva. Alcune parti restano sul runtime Python e sulla CPU, altre vengono delegate a kernel Metal sulla GPU. L'architettura integrata rende piu' diretto questo passaggio rispetto a un sistema con memoria CPU e VRAM separate, ma non elimina limiti di bandwidth, capacita' e concorrenza con altri processi.

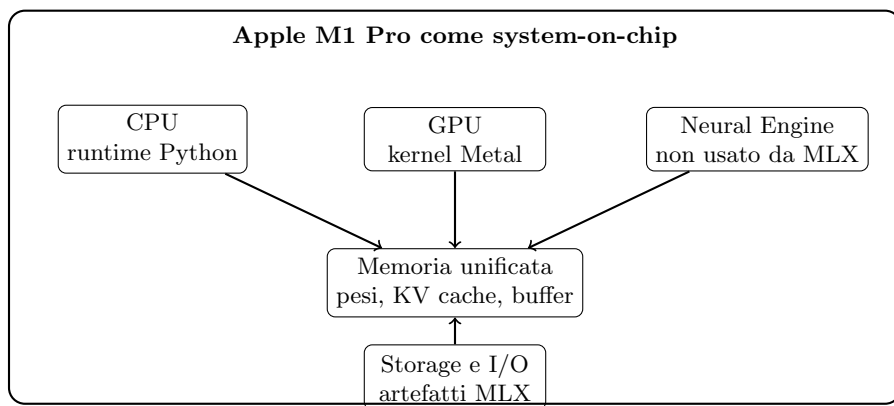


Figura 4.1: Schema semplificato dei sottosistemi rilevanti per l'inferenza LLM locale su Apple Silicon.

4.3.2 Memoria unificata

La caratteristica architetturale piu' rilevante per l'esperimento e' la memoria unificata. Nei Mac con Apple Silicon, CPU e GPU condividono lo stesso pool di memoria fisica; la documentazione

Metal descrive questo modello come un caso in cui la GPU condivide la memoria con la CPU [45]. Il MacBook Pro usato nei test dispone di 16 GB di memoria unificata, mentre M1 Pro supporta una bandwidth di memoria fino a 200 GB/s [43].

Questa impostazione ha due conseguenze per i modelli locali. Da un lato, non esiste una VRAM fissa separata dalla RAM di sistema: un modello che supererebbe la memoria video di una GPU consumer può diventare eseguibile su una macchina Apple se rientra nel budget complessivo della memoria unificata. Dall'altro, quei 16 GB non sono dedicati al modello. Devono ospitare sistema operativo, applicazioni aperte, processo Python, runtime MLX, tokenizer, pesi, cache e buffer temporanei. Per questo il capitolo misura sia la dimensione su disco degli artefatti sia il picco di memoria durante la generazione.

4.3.3 GPU, Metal e runtime MLX

La GPU Apple è esposta agli sviluppatori attraverso Metal, l'API grafica e compute di basso livello. MLX si colloca sopra questo strato: fornisce un framework di array ottimizzato per Apple Silicon e per la memoria unificata [44]. Il pacchetto `mlx-lm` usa MLX per caricare modelli linguistici, applicare template di chat, generare testo e produrre artefatti quantizzati [58]. La scelta di MLX riduce quindi il numero di adattamenti intermedi tra modello, runtime e hardware.

La presenza di Metal non rende automaticamente più veloce ogni variante quantizzata. Durante l'inferenza il runtime legge pesi compressi, applica scale o gruppi di quantizzazione e usa kernel adattati alla precisione scelta. Se la riduzione del traffico di memoria supera il costo di dequantizzazione, il throughput migliora; altrimenti la latenza può restare simile o peggiorare. Il protocollo misura quindi FP16, Q8, Q6 e Q4 separatamente, senza assumere una relazione monotona tra numero di bit e prestazioni.

4.3.4 Implicazioni per l'inferenza LLM locale

Nel contesto dell'inferenza LLM, Apple Silicon va quindi interpretato come una piattaforma vincolata dalla memoria e dipendente dal carico. La memoria unificata aiuta a caricare modelli più grandi rispetto a una GPU con poca VRAM, ma non elimina il costo della decodifica autoregressiva. Ogni token generato richiede un nuovo passaggio del modello, accessi ripetuti ai pesi e aggiornamento della KV cache. La quantizzazione riduce il footprint dei pesi e può ridurre il traffico di memoria, ma non comprime automaticamente tutti gli altri costi.

Per un sistema RAG questo aspetto pesa sul comportamento percepito. Il prompt finale non contiene solo la domanda dell'utente, ma anche istruzioni, frammenti recuperati e metadati. Su Apple Silicon, aumentare la lunghezza del prompt tende a pesare soprattutto sul prefill e quindi sul time-to-first-token; il throughput di decodifica misura invece la fase successiva, quando la risposta è già in streaming. La valutazione sperimentale considera quindi memoria, TTFT, decode token/s e qualità dell'output senza ridurre il giudizio a una sola metrica.

Alla luce di queste caratteristiche, l'esperimento usa MLX tramite il pacchetto `mlx-lm`. La valutazione riguarda un runtime Apple-native, non un backend generico adattato successivamente a Metal.

I runtime alternativi non sono ignorati; restano fuori dal confronto principale. MLC-LLM è interessante per compilazione e deployment ottimizzato [57]; llama.cpp e Ollama sono maturi e diffusi per esecuzione locale; PyTorch MPS è utile in ambito di sviluppo. Tuttavia, includerli nella

matrice avrebbe reso difficile attribuire le differenze osservate alla sola quantizzazione. Per questo il confronto e’ interno a MLX.

4.4 Matrice dei modelli

La matrice sperimentale include otto modelli open-weight, divisi in tre classi. La classe *small* permette il confronto completo tra FP16 e varianti quantizzate. La classe *medium* serve a osservare la soglia in cui la quantizzazione diventa necessaria per restare entro i limiti di memoria. La classe *large* verifica se modelli piu’ grandi possano almeno essere eseguiti in varianti compresse.

Tabella 4.1: Matrice dei modelli valutati.

Classe	Modello	Parametri	Ruolo sperimentale
Small	Gemma 3 4B Instruct	4.0B	Confronto completo FP16/Q8/Q6/Q4.
Small	Qwen3 4B Instruct	4.0B	Confronto completo FP16/Q8/Q6/Q4.
Small	Llama 3.2 3B Instruct	3.0B	Confronto completo FP16/Q8/Q6/Q4.
Small	Phi-3.5 mini Instruct	3.8B	Confronto completo FP16/Q8/Q6/Q4.
Medium	Qwen3 8B	8.2B	Fattibilita’ quantizzata; FP16 come controllo di fattibilita’.
Medium	Mistral 7B Instruct v0.3	7.3B	Fattibilita’ quantizzata; FP16 come controllo di fattibilita’.
Large	Gemma 3 12B Instruct	12.0B	Valutazione del limite superiore su 16 GB.
Large	Qwen3 14B	14.8B	Valutazione del limite superiore su 16 GB.

La selezione non mira a decretare il miglior modello in assoluto. Serve a osservare famiglie, dimensioni e soglie di fattibilita’ diverse. I modelli piccoli sono il gruppo controllato; medi e grandi mostrano cosa accade quando il vincolo di memoria diventa dominante.

4.5 Artefatti e politica di quantizzazione

Tutti gli artefatti quantizzati usati nell’esperimento sono generati localmente. La catena rimane sempre la stessa: download dello snapshot originale da Hugging Face, conversione locale in modello MLX FP16, quantizzazione locale con `mlx_lm.convert`, benchmark sugli artefatti prodotti. Per l’esperimento principale non vengono usati modelli pre-quantizzati di terze parti. In questo modo si evitano variabili non controllate, come versione del runtime, parametri di conversione, template del tokenizer, revisione del modello e differenze nella procedura di quantizzazione.

Le varianti sperimentali sono FP16, Q8, Q6 e Q4. Q8 rappresenta una compressione leggera, Q6 una compressione intermedia e Q4 una compressione aggressiva. Il gruppo small esegue il confronto completo contro FP16. Per medium e large, FP16 e’ trattato come controllo di fattibilita’: eventuali

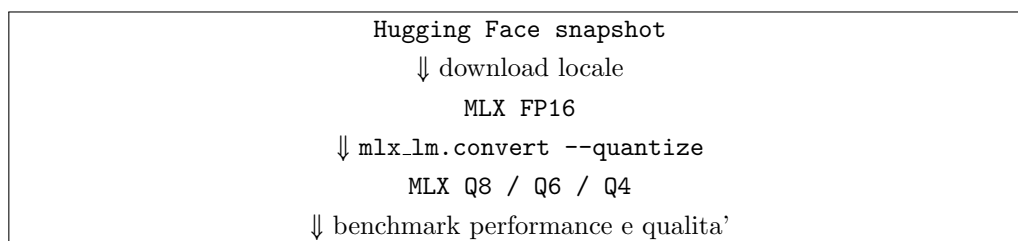


Figura 4.2: Catena riproducibile di produzione degli artefatti MLX.

fallimenti o omissioni per politica sperimentale vengono registrati e non interpretati come dati mancanti casuali.

Tabella 4.2: Varianti MLX e significato sperimentale.

Variante	Bit	Ruolo	Interpretazione
FP16	16	Baseline	Riferimento per dimensione, memoria, latenza e qualita' nei modelli piccoli.
Q8	8	Compressione leggera	Riduzione del footprint con rischio qualitativo atteso basso.
Q6	6	Compressione intermedia	Possibile equilibrio tra memoria, velocita' e qualita'.
Q4	4	Compressione aggressiva	Massima riduzione del footprint; rischio maggiore di degrado o instabilita'.

4.6 Benchmark prestazionale

Il benchmark prestazionale misura due fasi. La prima e' il *cold load*, cioe' il caricamento del modello e l'inizializzazione del runtime. La seconda e' la *warm generation*, cioe' la generazione dopo warm-up, quando il modello e' gia' caricato. Separare queste fasi e' necessario per un sistema interattivo: un modello puo' caricarsi lentamente ma generare in modo stabile, o viceversa.

I prompt di performance sono in inglese e divisi in classi *short*, *medium* e *long*. La lingua inglese e' mantenuta per coerenza con la pratica comune nei benchmark MMLU/GSM8K e per non introdurre la lingua come variabile sperimentale. La generazione usa impostazioni deterministiche: contesto 4096 token, massimo 256 token generati, temperatura 0, `top_p` pari a 1, `top_k` pari a 0 e seed fisso quando supportato. Ogni misura usa due generazioni di warm-up e tre run temporizzati.

4.7 Struttura dei run e aggregazione

Ogni combinazione modello-variante produce una riga di caricamento a freddo e, quando la policy prevede la generazione, piu' righe di generazione calda. Per ogni classe di prompt vengono eseguite

Tabella 4.3: Metriche raccolte nel benchmark prestazionale.

Metrica	Significato
Dimensione modello	Spazio su disco dell'artefatto MLX, espresso in GiB.
Compression ratio	Rapporto tra dimensione FP16 dello stesso modello e variante misurata.
Load time	Tempo di caricamento a freddo del modello.
TTFT	Time-to-first-token osservato durante generazione calda.
Decode token/s	Throughput della fase di decodifica, escluso il costo iniziale.
End-to-end token/s	Throughput complessivo della risposta, includendo overhead e TTFT.
Peak memory	Picco di memoria MLX/Metal rilevato quando disponibile.
Process RSS	Approssimazione della memoria residente del processo.
Status / error	Stato del run; fallimenti, OOM e instabilita' sono risultati validi.

due generazioni di warm-up e tre generazioni temporizzate. Le generazioni di warm-up non entrano nelle tabelle finali: servono a ridurre effetti transitori dovuti a inizializzazione del runtime, compilazione interna dei kernel e allocazione dei buffer. I tre run temporizzati vengono invece conservati nei file grezzi e usati per calcolare medie e variabilita'.

L'aggregazione del Capitolo 5 segue una regola conservativa: per ogni chiave logica composta da modello, variante, benchmark, fase, classe di prompt e indice del run viene mantenuto l'ultimo record disponibile in ordine temporale. Un nuovo run puo' quindi correggere o aggiornare una misura precedente senza cancellare la storia sperimentale. La scelta evita anche un problema comune nei benchmark locali: mescolare misure vecchie, ottenute con artefatti o script intermedi, con misure finali prodotte dalla versione consolidata della pipeline.

Le misure non vengono interpretate come deterministiche in senso stretto. Anche con temperatura nulla e seed fissato, il tempo di esecuzione puo' variare per pressione di memoria, stato termico, processi concorrenti e scheduling del sistema operativo. Per questo la valutazione separa grandezze strutturali quasi statiche, come dimensione su disco e bit di quantizzazione, da grandezze osservate, come TTFT, throughput e memoria di picco. Nel Capitolo 5 la stabilita' viene discussa usando il coefficiente di variazione dei run di decodifica.

Un'ulteriore scelta riguarda la lunghezza dei prompt. Il file di benchmark assegna a ogni prompt una classe nominale, ma il numero effettivo di token dipende dal tokenizer del modello e dal template di chat applicato. Le classi *short*, *medium* e *long* non vanno quindi lette come soglie assolute, ma come carichi relativi crescenti. Questa distinzione e' rilevante per l'uso RAG: il contesto recuperato puo' aumentare il prefill anche quando il numero di token generati resta fisso.

4.8 Benchmark di qualita'

La qualita' viene valutata come controllo secondario, non come benchmark esaustivo delle capacita' dei modelli. Vengono usati due subset ridotti: 16 domande stile MMLU e 16 problemi aritmetici stile GSM8K. La dimensione ridotta rende la valutazione eseguibile su M1 Pro con 16 GB, ma richiede cautela: una sola risposta cambia il punteggio di 6,25 punti percentuali. I risultati servono

quindi a individuare segnali macroscopici di degrado o comportamenti anomali, non a produrre classifiche generali.

Lo scoring e' automatico. Per MMLU viene estratta la risposta a scelta multipla prevista; per GSM8K viene estratto il numero finale. Ogni variante viene valutata sugli stessi esempi, con le stesse impostazioni di generazione usate nel benchmark prestazionale. Per i modelli piccoli e' possibile calcolare il delta rispetto alla baseline FP16. Per medium e large, dove FP16 non e' parte del confronto completo, l'analisi e' descrittiva.

Il parser e' volutamente semplice. MMLU accetta una lettera tra A e D; GSM8K usa l'ultimo numero riconosciuto nella risposta. Questa scelta rende il benchmark riproducibile e facile da ispezionare, ma lo rende dipendente dal formato dell'output. Alcuni modelli producono testo ragionativo prima della risposta, oppure non rispettano la richiesta di una sola lettera; in questi casi lo score automatico puo' confondere errore di ragionamento ed errore di formato. Per questo i risultati di qualita' vengono usati come segnale di controllo e non come misura definitiva delle capacita' dei modelli.

4.9 Riproducibilita'

La riproducibilita' e' gestita a livello di dati, comandi e risultati. I file di configurazione registrano modello sorgente, classe, licenza, directory Hugging Face, directory MLX, varianti richieste e parametri di generazione. I risultati sono salvati sia in JSONL sia in CSV, con run id e timestamp, cosi' da non sovrascrivere misure precedenti. Per ogni run vengono registrati anche versione macOS, versione MLX, versione `mlx-lm`, comando, hardware e metadati del modello quando disponibili.

Risultati grezzi e aggregati vengono salvati separatamente. Le tabelle del Capitolo 5 usano l'ultimo risultato disponibile per modello, variante, fase e classe di prompt, mentre i file grezzi rimangono disponibili per controlli successivi. In questo modo l'analisi puo' essere aggiornata senza perdere tracciabilita'.

4.10 Gestione degli artefatti e dei fallimenti

Il repository sperimentale e' pensato per lavorare anche con spazio disco limitato. Lo script conservativo processa un modello alla volta: scarica lo snapshot Hugging Face, produce l'artefatto MLX FP16, genera le varianti quantizzate, esegue i benchmark e conserva i risultati prima di rimuovere gli artefatti pesanti quando non servono piu'. Questa politica non riduce la riproducibilita', perche' i metadati essenziali restano nei file TSV e nei risultati JSONL/CSV: identificativo del modello sorgente, variante, dimensione, hash quando disponibile, versione runtime, comando e timestamp.

I fallimenti non vengono filtrati. Un modello puo' non essere disponibile per licenza, non convertirsi correttamente, non caricare, esaurire memoria oppure produrre output non parsabile. In tutti questi casi il risultato viene registrato con uno `status` esplicito. La scelta conta in una valutazione di fattibilita': su hardware con 16 GB, sapere che una configurazione non e' utilizzabile e' informazione sperimentale tanto quanto misurare il throughput di una configurazione riuscita.

4.11 Ipotesi sperimentali

Le ipotesi iniziali sono quattro. La prima e' che Q8, Q6 e Q4 riducano in modo quasi proporzionale la dimensione dei pesi, ma non producano necessariamente riduzioni proporzionali della latenza. La

seconda e' che Q4 sia la variante piu' vantaggiosa per memoria e throughput sui modelli piccoli, pur richiedendo una verifica qualitativa. La terza e' che i modelli medium e large diventino praticabili solo nelle varianti quantizzate, perche' il margine dei 16 GB e' troppo stretto per FP16. La quarta e' che i subset MMLU e GSM8K possano evidenziare degradazioni evidenti, ma non siano abbastanza ampi da misurare differenze sottili di qualita'.

Queste ipotesi derivano dalla letteratura, ma devono essere verificate sul setup locale. I risultati di altri studi usano spesso M1 Max, M2 Ultra, M3 Ultra o GPU NVIDIA; il presente esperimento misura invece il caso piu' vincolato di un MacBook Pro M1 Pro con 16 GB di memoria unificata.

4.12 Minacce alla validita'

Il primo limite e' l'hardware singolo. Tutte le misure derivano da una sola macchina, quindi non possono essere generalizzate direttamente a Mac con piu' memoria, piu' bandwidth o generazioni diverse di GPU. Il secondo limite e' il runtime singolo: MLX viene scelto per coerenza con Apple Silicon, ma i risultati non dimostrano superiorita' rispetto a MLC-LLM, llama.cpp o Ollama.

Il terzo limite riguarda i benchmark di qualita'. MMLU e GSM8K sono usati in subset ridotti; cio' rende possibile l'esecuzione locale ma amplifica il peso di ogni singolo errore. Il quarto limite riguarda il contesto: non vengono valutati long context, KV-cache quantization o kernel fusi come quelli discussi da Open-TQ-Metal. Il quinto limite e' l'assenza di una valutazione umana delle risposte: l'esperimento misura segnali automatici e metriche di sistema, non preferenze qualitative fini.

Questi limiti non invalidano l'esperimento, ma ne definiscono il perimetro. Il Capitolo 5 va quindi letto come una valutazione controllata di fattibilita', efficienza e segnali qualitativi su MLX/Apple Silicon, non come benchmark universale dei modelli considerati.

4.13 Sintesi del capitolo

Il capitolo ha definito un protocollo sperimentale circoscritto: modelli open-weight tra 3B e 14B parametri, runtime MLX, hardware Apple M1 Pro con 16 GB di memoria unificata e varianti FP16, Q8, Q6 e Q4 quando disponibili. La separazione tra misure di sistema e controlli qualitativi ridotti evita di ridurre il problema a una sola metrica. Memoria e dimensione dell'artefatto stabiliscono la fattibilita', TTFT e throughput descrivono la reattivita', MMLU e GSM8K forniscono un segnale minimo sulla robustezza dell'output.

Il disegno non pretende di misurare l'intero sistema RAG in produzione. Serve piuttosto a capire quali configurazioni siano realistiche prima di inserirle in un ambiente con Qdrant, FastAPI, Next.js e Ollama concorrenti sulla stessa macchina. Il capitolo successivo applica questo protocollo e discute i compromessi osservati.

Capitolo 5

Valutazione sperimentale e analisi dei risultati

Questo capitolo analizza i risultati ottenuti con il protocollo definito nel Capitolo 4. I benchmark sono stati eseguiti sulla repository sperimentale e includono tre famiglie di misure: performance MLX, subset MMLU e subset GSM8K. Per ogni combinazione di modello, variante, fase e classe di prompt viene usato l'ultimo risultato disponibile; i file grezzi JSONL e CSV restano la base riproducibile dell'analisi.

L'obiettivo non è costruire una graduatoria assoluta dei modelli. L'analisi misura quale compromesso produca la quantizzazione MLX su Apple Silicon con 16 GB di memoria unificata. Per i modelli piccoli il confronto è tra FP16 e Q8/Q6/Q4. Per i modelli medi e grandi l'analisi riguarda soprattutto la fattibilità delle varianti quantizzate: la baseline FP16 resta utile come riferimento di dimensione e come controllo metodologico, ma non viene trattata come configurazione completa quando la policy la considera troppo onerosa per il target hardware.

5.1 Dataset dei risultati

I risultati aggregati derivano da 19 CSV di performance, 11 CSV MMLU e 11 CSV GSM8K. Le tabelle non riportano tutti i singoli run: usano medie o valori finali aggregati per mantenere leggibile il confronto. I dati grezzi rimangono disponibili nel repository sperimentale e consentono di ricostruire ogni valore aggregato.

Tabella 5.1: Copertura dei benchmark disponibili.

Famiglia risultati	CSV	Contenuto
Performance MLX	19	Cold load, warm generation, TTFT, throughput, dimensione e memoria.
MMLU ridotto	11	16 domande a scelta multipla per variante eseguibile.
GSM8K ridotto	11	16 problemi aritmetici con estrazione automatica del numero finale.

5.2 Risultati sui modelli piccoli

I quattro modelli piccoli sono il gruppo piu' adatto per stimare l'effetto della quantizzazione, perche' dispongono di una baseline FP16 completa. La Tabella 5.2 riporta le medie sulle tre classi di prompt per TTFT, throughput di decodifica, throughput end-to-end e picco di memoria. Il load time e' quello del cold load.

Tabella 5.2: Prestazioni medie dei modelli piccoli.

Modello	Var.	Size GiB	Load s	TTFT s	Decode tok/s	E2E tok/s	Mem. picco GB
Gemma 3 4B	FP16	7.26	3.92	0.462	20.94	19.92	7.92
Gemma 3 4B	Q8	3.87	3.90	0.530	37.33	33.90	4.35
Gemma 3 4B	Q6	2.97	2.40	0.525	41.58	37.60	3.38
Gemma 3 4B	Q4	2.06	2.06	0.513	53.64	47.69	2.41
Qwen3 4B	FP16	7.50	2.89	0.395	20.15	19.23	8.18
Qwen3 4B	Q8	3.99	2.93	0.475	36.80	33.57	4.52
Qwen3 4B	Q6	3.05	1.39	0.471	41.94	37.71	3.58
Qwen3 4B	Q4	2.12	0.94	0.452	55.13	48.18	2.60
Llama 3.2 3B	FP16	6.00	2.25	0.340	25.39	24.06	6.59
Llama 3.2 3B	Q8	3.20	2.07	0.450	46.57	41.19	3.67
Llama 3.2 3B	Q6	2.45	1.27	0.445	53.39	45.67	2.93
Llama 3.2 3B	Q4	1.70	1.13	0.439	70.51	58.95	2.18
Phi-3.5 mini	FP16	7.12	1.90	0.352	21.54	20.83	7.88
Phi-3.5 mini	Q8	3.78	1.04	0.433	39.81	37.00	4.44
Phi-3.5 mini	Q6	2.90	0.72	0.437	45.48	42.04	3.48
Phi-3.5 mini	Q4	2.01	0.54	0.419	60.05	54.60	2.59

La dimensione si riduce in modo quasi regolare: Q8 porta i modelli piccoli intorno a 3.2–4.0 GiB, Q6 intorno a 2.4–3.1 GiB e Q4 intorno a 1.7–2.1 GiB. Rispetto alla baseline FP16, Q4 riduce la dimensione del 71.6–71.8%. Il picco di memoria cala in modo simile, anche se non coincide con la dimensione su disco, perche' il runtime mantiene strutture aggiuntive oltre ai pesi.

Anche il throughput cambia in modo netto. Su tutti i modelli piccoli, Q4 e' la variante piu' veloce in decode: 53.64 tok/s per Gemma 3 4B, 55.13 tok/s per Qwen3 4B, 70.51 tok/s per Llama 3.2 3B e 60.05 tok/s per Phi-3.5 mini. Lo speedup rispetto a FP16 va da 2.56x a 2.79x. Nel setup misurato, quindi, la riduzione del traffico di memoria supera il costo di dequantizzazione per le varianti MLX considerate.

Tabella 5.3: Riduzione e speedup delle varianti quantizzate rispetto a FP16 nei modelli piccoli.

Modello	Var.	Rid. size	Rid. mem.	Speedup decode	Speedup load
Gemma 3 4B	Q8	46.7%	45.1%	1.78x	1.01x
Gemma 3 4B	Q6	59.1%	57.3%	1.99x	1.63x
Gemma 3 4B	Q4	71.6%	69.6%	2.56x	1.90x
Qwen3 4B	Q8	46.8%	44.8%	1.83x	0.98x
Qwen3 4B	Q6	59.3%	56.3%	2.08x	2.08x
Qwen3 4B	Q4	71.8%	68.2%	2.74x	3.08x
Llama 3.2 3B	Q8	46.7%	44.4%	1.83x	1.09x
Llama 3.2 3B	Q6	59.2%	55.5%	2.10x	1.77x
Llama 3.2 3B	Q4	71.7%	67.0%	2.78x	1.99x
Phi-3.5 mini	Q8	46.8%	43.7%	1.85x	1.82x
Phi-3.5 mini	Q6	59.3%	55.8%	2.11x	2.65x
Phi-3.5 mini	Q4	71.8%	67.1%	2.79x	3.50x

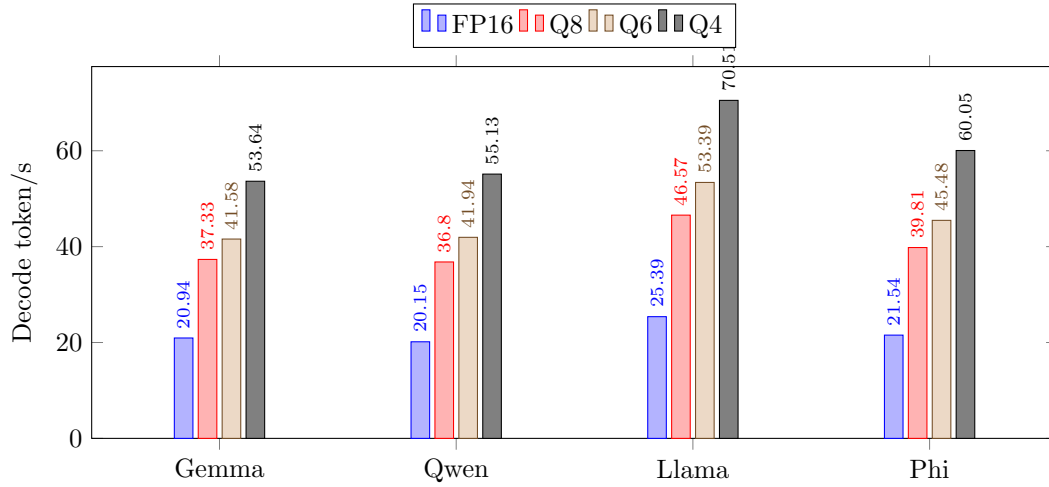


Figura 5.1: Throughput di decodifica medio per i modelli piccoli.

Il TTFT e' meno lineare. Le varianti quantizzate hanno spesso un TTFT leggermente superiore a FP16, soprattutto Q8 e Q6, pur offrendo un decode molto piu' rapido. Il tempo al primo token non dipende quindi dallo stesso fattore che domina il throughput di decodifica: prefill, inizializzazione dei buffer e gestione del prompt possono pesare in modo diverso dal ciclo autoregressivo. In applicazioni interattive, le due metriche vanno lette insieme.

5.3 Risultati sui modelli medi e grandi

Per i modelli medi e grandi non interessa soprattutto il delta rispetto a FP16 completo, ma la fattibilita' delle varianti compresse. La Tabella 5.4 mostra che Q4 rende eseguibili modelli tra 7B e 14B entro un picco di memoria compreso tra circa 4.35 GB e 8.56 GB. Q6 rimane fattibile ma piu' costosa, arrivando a 12.20 GB per Qwen3 14B.

Tabella 5.4: Prestazioni medie dei modelli medi e grandi quantizzati.

Modello	Var.	Size GiB	Load s	TTFT s	Decode tok/s	E2E tok/s	Mem. picco GB
Qwen3 8B	Q8	8.12	2.55	0.781	21.11	19.82	8.90
Qwen3 8B	Q6	6.21	2.01	0.785	24.08	22.39	6.89
Qwen3 8B	Q4	4.30	1.36	0.752	32.28	29.45	4.89
Mistral 7B v0.3	Q8	7.18	2.01	0.783	22.60	20.85	7.88
Mistral 7B v0.3	Q6	5.49	1.55	0.786	26.10	23.63	6.11
Mistral 7B v0.3	Q4	3.80	0.94	0.756	35.12	31.15	4.35
Gemma 3 12B	Q6	8.94	4.01	1.310	13.90	12.82	9.85
Gemma 3 12B	Q4	6.20	3.16	1.259	18.55	16.66	6.96
Qwen3 14B	Q6	11.19	3.63	1.405	13.52	12.59	12.20
Qwen3 14B	Q4	7.75	2.47	1.348	18.20	16.61	8.56

Nei modelli medi, Q4 raggiunge valori di decode paragonabili o superiori alla baseline FP16 dei modelli piccoli: 32.28 tok/s per Qwen3 8B e 35.12 tok/s per Mistral 7B. Per l'uso locale questo e'

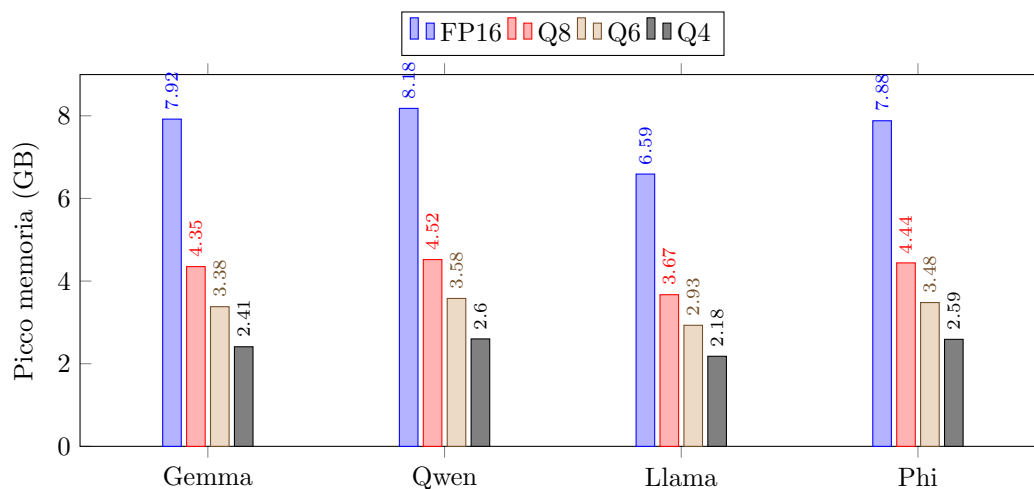


Figura 5.2: Picco di memoria rilevato per i modelli piccoli.

un dato pratico: un modello quantizzato da 7–8B puo' essere piu' lento di un piccolo Q4, ma resta abbastanza reattivo per molte interazioni non real-time. Nei modelli grandi, invece, il throughput Q4 scende a circa 18 tok/s e il TTFT supera 1.2 secondi. La generazione rimane possibile, ma l'esperienza utente cambia: questi modelli si adattano meglio a risposte meno frequenti o a uso batch che a chat molto interattive su macchina condivisa.

La differenza tra Q6 e Q4 e' coerente in tutte le classi: Q4 e' piu' piccolo, carica piu' velocemente e decodifica piu' rapidamente. Per Gemma 3 12B, Q4 riduce il picco da 9.85 GB a 6.96 GB e aumenta il decode da 13.90 a 18.55 tok/s. Per Qwen3 14B, Q4 riduce il picco da 12.20 GB a 8.56 GB e aumenta il decode da 13.52 a 18.20 tok/s. Nel contesto dei 16 GB questo margine conta, perche' lascia spazio al sistema operativo e ad altri processi; Q6 su Qwen3 14B e' molto piu' vicino al limite pratico.

5.4 Effetto della lunghezza del prompt

La media aggregata per classe di prompt mostra un andamento prevedibile. Il throughput di decodifica cambia poco tra prompt brevi, medi e lunghi: 34.82, 34.33 e 34.20 tok/s. Il throughput end-to-end cala leggermente sui prompt lunghi, mentre il TTFT cresce in modo chiaro: da 0.359 s sui prompt brevi a 0.704 s sui medi e 0.910 s sui lunghi. Il costo del prefill e della gestione del contesto pesa quindi soprattutto prima del primo token; il ciclo di decodifica successivo rimane relativamente stabile.

Per un sistema RAG questa distinzione ha effetti pratici. Il prompt finale include domanda, istruzioni e contesto recuperato; quindi assomiglia piu' spesso a un prompt medio o lungo che a un prompt breve. Anche con decode veloce, un contesto piu' esteso puo' rendere piu' percepibile il ritardo prima dell'inizio della risposta.

La Tabella 5.6 riporta una misura più vicina alla percezione dell'utente: la somma tra TTFT e durata della generazione per le varianti Q4. Il valore non coincide con il tempo di una risposta

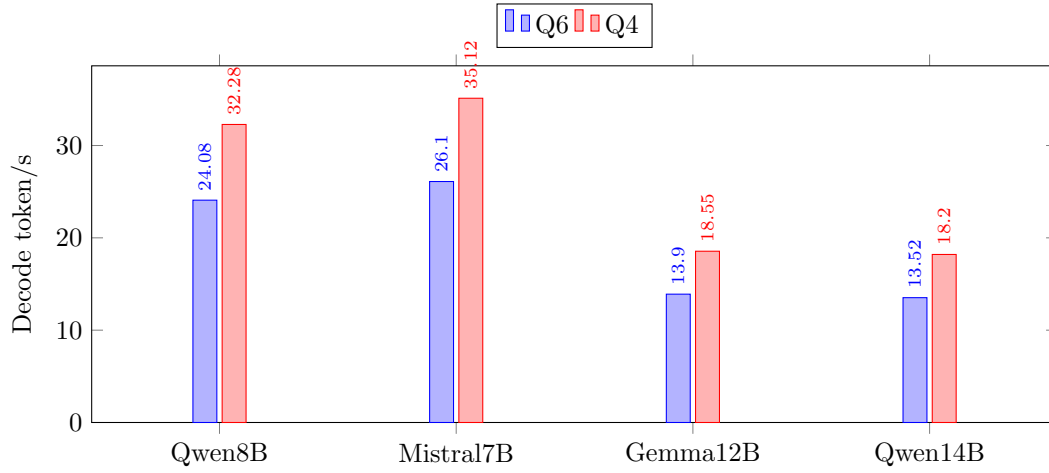


Figura 5.3: Throughput di decodifica dei modelli medi e grandi quantizzati per le varianti disponibili su tutte le taglie. Le misure Q8 sono disponibili solo per i modelli medium e sono riportate in Tabella 5.4.

Tabella 5.5: Medie aggregate per classe di prompt.

Classe prompt	TTFT medio (s)	Decode tok/s	E2E tok/s
Short	0.359	34.82	31.13
Medium	0.704	34.33	31.46
Long	0.910	34.20	30.65

RAG completa, perché il benchmark isola il modello, ma permette di distinguere configurazioni che hanno throughput simile e tempi totali diversi.

5.5 Stabilita' delle misure

I tre run temporizzati per prompt mostrano una variabilità contenuta nel throughput di decodifica. Il coefficiente di variazione medio è pari a 0.21% per i modelli piccoli, 0.07% per i medi e 0.03% per i grandi; il massimo osservato sui gruppi modello-variante-prompt rimane 1.07%. Il dato non rende i valori universali, ma indica che, nel setup locale e nelle condizioni di test, il decode token/s è una metrica stabile. Le differenze osservate tra FP16, Q8, Q6 e Q4 sono molto più grandi del rumore dei run ripetuti.

La tokenizzazione effettiva conferma anche che le tre classi di prompt rappresentano carichi distinti, pur senza coincidere con soglie assolute. Nei run aggregati, i prompt brevi producono in media 23.4 token di input, i medi 90.0 e i lunghi 124.1, con differenze dovute ai tokenizer e ai template di chat dei singoli modelli. Il passaggio da prompt breve a lungo aumenta il TTFT medio di 0.312 s nei modelli piccoli, 0.776 s nei medi e 1.173 s nei grandi. L'effetto cresce con la dimensione del modello, come ci si aspetta in un RAG dove il prompt contiene anche contesto recuperato.

Tabella 5.6: Tempo medio tra richiesta, primo token e fine generazione per le varianti Q4.

Modello	Classe	TTFT (s)	Generazione (s)	Totale (s)
Llama 3.2 3B	Small	0.456	3.507	3.964
Phi-3.5 mini	Small	0.429	4.289	4.718
Qwen3 4B	Small	0.505	4.355	4.860
Gemma 3 4B	Small	0.591	4.521	5.112
Mistral 7B v0.3	Medium	0.777	6.969	7.746
Qwen3 8B	Medium	0.803	8.758	9.561
Gemma 3 12B	Large	1.337	13.100	14.437
Qwen3 14B	Large	1.439	15.527	16.967

Tabella 5.7: Stabilita' del decode throughput e token effettivi dei prompt.

Gruppo	CV medio decode	CV massimo decode	Gruppi misurati
Small	0.21%	1.07%	48
Medium	0.07%	0.34%	18
Large	0.03%	0.17%	12

5.6 Qualita' su MMLU ridotto

Il subset MMLU contiene 16 domande. I modelli piccoli mantengono risultati stabili attraverso la quantizzazione: Gemma 3 4B, Qwen3 4B, Llama 3.2 3B e Phi-3.5 mini restano tra 15/16 e 16/16. In due casi, Gemma 3 4B Q4 e Llama 3.2 3B Q4, il punteggio risulta 16/16 contro 15/16 della rispettiva baseline FP16. Non va letto come miglioramento intrinseco della quantizzazione: con 16 esempi, una sola risposta puo' dipendere da parsing, forma dell'output o caso specifico.

Tabella 5.8: Accuratezza su MMLU ridotto.

Modello	FP16	Q8	Q6	Q4
Gemma 3 4B	15/16	15/16	15/16	16/16
Qwen3 4B	15/16	15/16	15/16	15/16
Llama 3.2 3B	15/16	15/16	15/16	16/16
Phi-3.5 mini	15/16	15/16	15/16	15/16
Qwen3 8B	n.d.	0/16	0/16	0/16
Mistral 7B v0.3	n.d.	16/16	16/16	16/16
Gemma 3 12B	n.d.	n.d.	15/16	15/16
Qwen3 14B	n.d.	n.d.	0/16	0/16

I risultati anomali sono Qwen3 8B e Qwen3 14B, entrambi a 0/16 nelle varianti misurate. Poiche' lo stesso comportamento non appare su Qwen3 4B, il risultato non puo' essere attribuito automaticamente alla famiglia Qwen o alla quantizzazione. Le righe grezze mostrano output con contenuto ragionativo e formati non sempre compatibili con l'estrazione automatica della risposta. La lettura piu' prudente e' metodologica: per questi modelli, il benchmark MMLU ridotto segnala un problema di compatibilita' tra prompting, modalita' di risposta e parser, oltre a eventuali limiti qualitativi. Una valutazione futura dovrebbe distinguere errore reale, errore di formato e fallimento di estrazione.

L'ispezione dei file JSONL chiarisce il problema. Nei casi Qwen3 8B e Qwen3 14B, le risposte MMLU spesso iniziano con testo di ragionamento e non con una lettera isolata. Il parser cerca una risposta nel formato richiesto, ma assegna **None** quando la generazione non lo rispetta. In un'applicazione reale l'utente potrebbe leggere comunque una spiegazione utile, ma una pipeline automatica non avrebbe un valore strutturato affidabile da usare. Il risultato sperimentale e' quindi duplice: il modello puo' essere eseguibile in termini di memoria e throughput, ma non essere subito integrabile senza prompt engineering o parsing piu' robusto.

Mistral 7B v0.3 ottiene 16/16 in Q8, Q6 e Q4; Gemma 3 12B ottiene 15/16 in Q6 e Q4. Almeno su questo subset, Q4 non produce un degrado macroscopico per i modelli non affetti da problemi di parsing.

5.6.1 Analisi dei casi a punteggio nullo su MMLU

Le sezioni a 0/16 non vanno lette come un fallimento qualitativo puro. Nei run Qwen3 8B e Qwen3 14B, molte risposte iniziano con un blocco di ragionamento marcato da **<think>** e non con la lettera richiesta. In un esempio, il campione `mmlu-001` attendeva la risposta **A**, ma l'output iniziava con una spiegazione discorsiva; il parser non ha trovato una scelta finale nel formato atteso e ha registrato **None**. Lo stesso schema si ripete su più domande: il contenuto può contenere ragionamento, ma non espone un valore strutturato.

Questo è rilevante per la tesi perché riproduce un problema tipico dei sistemi RAG applicativi. Un modello può generare testo leggibile e allo stesso tempo non rispettare il contratto richiesto dal software che lo usa. Nel caso MMLU il contratto è una lettera tra A e D; nel prototipo i3Guild può essere una citazione di fonte, un campo JSON o un evento NDJSON. La mitigazione non consiste soltanto nel cambiare modello: servono prompt più vincolanti, stop sequence, eventuale disattivazione del ragionamento visibile quando supportata, e parser che distinguano errore di formato da errore semantico.

5.7 Qualita' su GSM8K ridotto

Il subset GSM8K misura un segnale elementare di ragionamento aritmetico. Anche qui la dimensione del dataset richiede cautela. I modelli piccoli mantengono risultati molto alti: Gemma 3 4B passa da 15/16 in FP16 a 16/16 nelle varianti quantizzate, Qwen3 4B e Phi-3.5 mini ottengono 16/16 in tutte le varianti, mentre Llama 3.2 3B passa da 13/16 in FP16 e Q8 a 15/16 in Q6 e Q4.

Tabella 5.9: Accuratezza su GSM8K ridotto.

Modello	FP16	Q8	Q6	Q4
Gemma 3 4B	15/16	16/16	16/16	16/16
Qwen3 4B	16/16	16/16	16/16	16/16
Llama 3.2 3B	13/16	13/16	15/16	15/16
Phi-3.5 mini	16/16	16/16	16/16	16/16
Qwen3 8B	n.d.	6/16	5/16	3/16
Mistral 7B v0.3	n.d.	15/16	15/16	16/16
Gemma 3 12B	n.d.	n.d.	16/16	15/16
Qwen3 14B	n.d.	n.d.	6/16	4/16

GSM8K conferma il comportamento problematico di Qwen3 8B e Qwen3 14B nel setup di scoring automatico. Qwen3 8B scende da 6/16 in Q8 a 3/16 in Q4; Qwen3 14B passa da 6/16 in Q6 a 4/16

in Q4. Poiche' il benchmark richiede estrazione numerica, una parte dell'errore puo' dipendere dal formato della risposta. Il calo tra Q8/Q6 e Q4 su Qwen3 8B e tra Q6 e Q4 su Qwen3 14B segnala pero' anche un possibile degrado operativo per questa combinazione di modello, prompt e parser.

Il caso GSM8K mostra un limite diverso rispetto a MMLU. Qui il modello puo' produrre una catena di ragionamento plausibile, ma il parser usa l'ultimo numero presente nella risposta. Se la risposta contiene numeri intermedi dopo il risultato corretto, oppure se conclude con una formulazione non conforme a **Final answer: <number>**, il punteggio automatico puo' risultare errato. Per questo i valori bassi di Qwen3 8B e Qwen3 14B vengono trattati come fallimento del sistema di valutazione nel suo insieme: modello, prompt e parser. In una pipeline RAG aziendale il problema sarebbe analogo alla generazione di citazioni o metadati in formato non parseabile.

Mistral 7B v0.3 e Gemma 3 12B sono invece stabili: Mistral raggiunge 16/16 in Q4, mentre Gemma 3 12B passa da 16/16 in Q6 a 15/16 in Q4. Per questi modelli, la quantizzazione aggressiva non compromette in modo osservabile il segnale qualitativo del subset.

5.7.1 Analisi degli errori GSM8K

GSM8K evidenzia un errore diverso da MMLU. Qui il parser riesce spesso a estrarre un numero, ma non sempre quello corretto. Per esempio, in un campione su Qwen3 14B Q4 la risposta attesa era 30, mentre il parser ha registrato 2: l'output conteneva passaggi intermedi sul numero di file prodotti da ciascun run, e l'estrazione automatica ha selezionato un numero non finale. In un altro caso, la risposta attesa era 20 token/s, ma il parser ha letto 6 perché la spiegazione conteneva anche il numero di secondi usato nel problema.

Questi casi mostrano che la catena di ragionamento non è sufficiente se il risultato finale non è isolato in modo stabile. Per una valutazione successiva sarebbe preferibile imporre una riga conclusiva univoca, ad esempio **Final answer: <number>**, e verificare separatamente due condizioni: correttezza del ragionamento e conformità del formato. In un sistema RAG, la stessa distinzione permette di capire se una risposta è sbagliata perché il modello non ha usato correttamente il contesto o perché ha prodotto metadati in una forma non interpretabile dal frontend.

5.8 Qualita' per GiB e trade-off pratico

La qualita' per GiB e' una metrica derivata utile in un contesto memory-bound: rapporta l'accuratezza alla dimensione dell'artefatto. Non sostituisce l'accuratezza assoluta, perche' un modello piccolo ma scorretto non diventa utile solo perche' leggero. Quando l'accuratezza resta stabile, pero', la metrica mostra quanto la quantizzazione migliori l'efficienza rispetto alla memoria.

Nei modelli piccoli questo effetto e' chiaro. Phi-3.5 mini su GSM8K passa da 0.140 accuratezza/GiB in FP16 a 0.499 in Q4; Qwen3 4B passa da 0.133 a 0.472; Gemma 3 4B passa da 0.129 a 0.484; Llama 3.2 3B passa da 0.135 a 0.552. Su MMLU, Gemma 3 4B Q4 e Llama 3.2 3B Q4 raggiungono rispettivamente 0.484 e 0.588 accuratezza/GiB. L'aumento deriva soprattutto dalla riduzione della dimensione, non da un miglioramento qualitativo stabile.

5.9 Efficienza prestazionale per GiB

Una seconda metrica derivata e' il throughput di decodifica per GiB di artefatto. Anche questa non va usata da sola: un modello molto piccolo puo' avere ottima efficienza per GiB ma capacita'

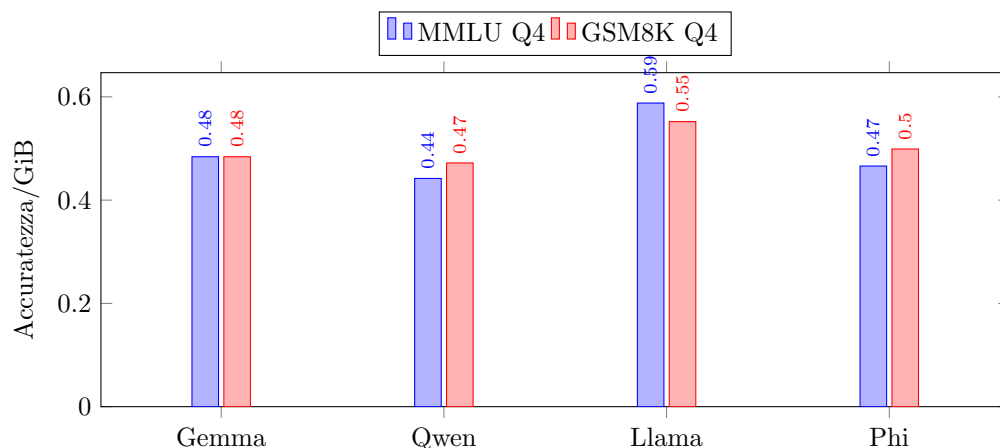


Figura 5.4: Accuratezza per GiB sulle varianti Q4 dei modelli piccoli.

inferiori su compiti complessi. Nel contesto della tesi, la metrica aiuta a stimare il costo pratico di una scelta quando memoria e disco sono risorse limitate.

Tabella 5.10: Efficienza di decodifica delle varianti Q4.

Modello	Classe	Size GiB	Decode tok/s	Tok/s/GiB
Llama 3.2 3B	Small	1.70	70.52	41.50
Phi-3.5 mini	Small	2.01	59.80	29.82
Qwen3 4B	Small	2.12	55.16	26.04
Gemma 3 4B	Small	2.06	53.69	26.00
Mistral 7B v0.3	Medium	3.80	35.11	9.24
Qwen3 8B	Medium	4.30	32.28	7.50
Gemma 3 12B	Large	6.20	18.55	2.99
Qwen3 14B	Large	7.75	18.20	2.35

La tabella mostra un aspetto che il solo throughput rende meno visibile: il costo marginale dei modelli più grandi è alto. Passare da Llama 3.2 3B Q4 a Mistral 7B Q4 dimezza circa il throughput e riduce di oltre quattro volte l'efficienza per GiB; passare ai modelli large riduce ancora l'efficienza. Non significa che i modelli grandi siano inutili, ma che il loro uso locale deve essere giustificato da un guadagno qualitativo osservabile nel compito applicativo.

5.10 Criteri di lettura delle metriche

Le metriche raccolte non hanno tutte lo stesso peso decisionale. In un sistema RAG locale, alcune misure sono vincoli duri, altre descrivono l'esperienza utente, altre ancora servono come controllo qualitativo. La dimensione dell'artefatto e il picco di memoria sono vincoli duri: se il modello non lascia spazio agli altri servizi, la configurazione non è utilizzabile anche quando l'accuratezza è buona. TTFT, decode ed end-to-end throughput descrivono invece la reattività percepita. Un

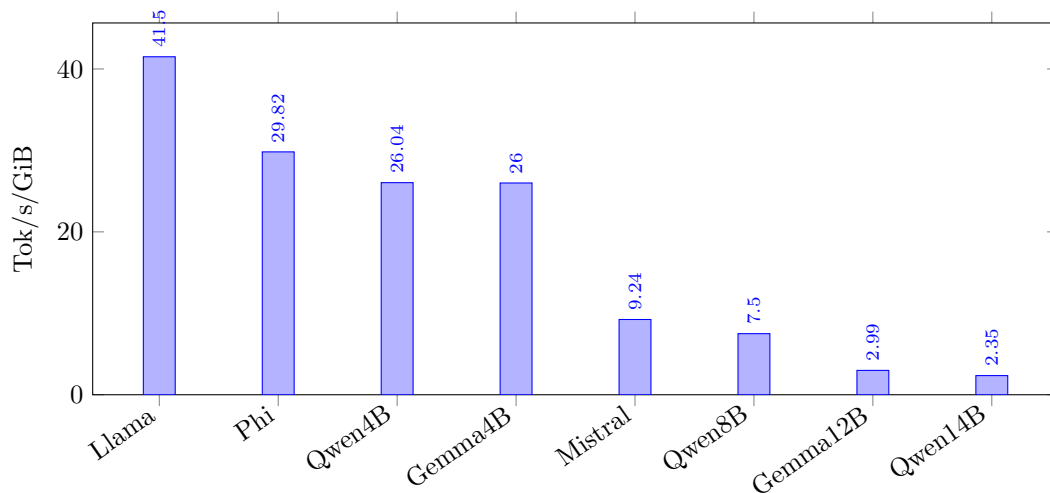


Figura 5.5: Efficienza di decodifica per GiB nelle varianti Q4.

modello con decode elevato ma TTFT alto può risultare fluido dopo l'avvio dello streaming, ma l'utente percepisce comunque un ritardo iniziale. MMLU e GSM8K misurano solo segnali ridotti di qualità generale e non sostituiscono una valutazione RAG sul corpus aziendale.

Tabella 5.11: Ruolo decisionale delle metriche sperimentali.

Metrica	Ruolo	Interpretazione operativa
Size e memoria di picco	Vincolo di fattibilità	Stabilisce se modello, retriever, API, Qdrant e frontend possono convivere sulla stessa macchina.
Load time	Costo di cold start	Incide quando il modello non resta sempre caricato o dopo riavvii del servizio.
TTFT	Latenza iniziale percepita	Pesa nelle chat interattive, soprattutto con prompt RAG lunghi.
Decode tok/s	Velocità di generazione	Misura la fluidità dello streaming dopo il primo token.
MMLU/GSM8K ridotti	Controllo qualitativo leggero	Evidenzia regressioni grossolane, ma non misura groundedness o utilità sul corpus i3Guild.

Questo criterio spiega perché Q4 risulti spesso preferibile. Nei modelli piccoli Q4 riduce il footprint, accelera la decodifica e non introduce perdite evidenti nei controlli automatici ridotti. Q8 e Q6 restano varianti più conservative, ma nel dataset osservato non offrono un vantaggio qualitativo misurabile. La scelta non va generalizzata senza contesto: se una valutazione RAG end-to-end mostrasse perdita di completezza, citazioni meno accurate o più allucinazioni, il vantaggio prestazionale di Q4 andrebbe rivalutato.

5.11 Analisi degli errori di output e del parsing

I risultati di qualità non vanno letti come proprietà isolate del modello. Nel benchmark automatico il punteggio dipende da prompt, modello e parser. Se uno di questi elementi non è coerente con gli altri, il sistema può fallire anche quando la risposta contiene parte dell'informazione corretta. Il caso di Qwen3 8B e Qwen3 14B è il caso più chiaro: nei file grezzi MMLU diverse risposte non sono lettere isolate, ma testi ragionativi. Il parser, progettato per estrarre una risposta strutturata, assegna valore nullo quando il formato non è riconoscibile.

Tabella 5.12: Tipologie di errore osservate o attese nel benchmark automatico.

Errore	Effetto sul benchmark	Mitigazione
Risposta non parseabile	Punteggio nullo anche in presenza di contenuto utile	Prompt più vincolante, stop sequence, parser tollerante o output strutturato.
Ragionamento prima della scelta	Fallimento su MMLU quando la lettera non è isolata	Separare eventuale spiegazione dalla risposta finale.
Numeri intermedi dopo il risultato	Errore su GSM8K se il parser legge l'ultimo numero	Richiedere una riga finale univoca nel formato Final answer: <number> .
Prompt lungo in contesto RAG	TTFT più alto prima dello streaming	Limitare il contesto, deduplicare i chunk e selezionare solo fonti rilevanti.
Pressione di memoria	Swap, rallentamenti o instabilità con servizi concorrenti	Preferire Q4, ridurre il modello o distribuire i servizi su macchine separate.

Lo stesso problema vale per il sistema RAG. In produzione non basta generare testo plausibile: il sistema deve restituire citazioni, fonti, metadati e, quando necessario, risposte in formato controllabile dal frontend. Un output non strutturato rende più difficile mostrare le fonti, applicare controlli di groundedness o costruire funzioni successive sulla risposta. Per questo i risultati anomali non vengono usati per scartare definitivamente una famiglia di modelli, ma per indicare una condizione minima di integrazione: prima di promuovere un modello nel prototipo, prompt e parser devono essere validati sul formato richiesto dall'applicazione.

5.12 Discussione complessiva

I risultati portano a tre letture principali. La prima riguarda i modelli piccoli: nel setup locale, Q4 e' il compromesso migliore tra memoria e prestazioni. Riduce la dimensione di circa il 72%, riduce il picco di memoria di circa 67–70%, aumenta il decode di circa 2.6–2.8x e non mostra un degrado macroscopico nei subset MMLU/GSM8K. Q8 e Q6 sono configurazioni più conservative, ma nei dati raccolti non offrono un vantaggio qualitativo misurabile sufficiente a compensare il footprint maggiore.

La seconda riguarda i modelli medi e grandi. In questo caso la quantizzazione serve soprattutto a rendere il modello eseguibile. Qwen3 8B e Mistral 7B diventano utilizzabili con throughput Q4 intorno a 32–35 tok/s e picco inferiore a 5 GB. Gemma 3 12B e Qwen3 14B rimangono eseguibili in Q4 con circa 18 tok/s, ma con TTFT superiore al secondo e maggiore pressione sulla memoria.

In un'applicazione locale interattiva, questi modelli richiedono una valutazione caso per caso: sono possibili, ma non necessariamente preferibili ai modelli piccoli Q4.

La terza riguarda le metriche automatiche di qualità. I risultati nulli di Qwen3 8B e Qwen3 14B su MMLU ridotto indicano un problema reale per il setup di valutazione, ma da soli non separano errore di ragionamento, errore di formato ed errore di parsing. Per una tesi su sistemi questo punto è centrale: non basta che un modello *sappia* produrre la risposta; deve produrla in un formato che la pipeline possa usare.

Tabella 5.13: Sintesi del compromesso osservato per classe di modello.

Classe	Variante preferibile	Motivazione
Small	Q4	Massimo risparmio di memoria, throughput più alto e qualità stabile sui subset.
Medium	Q4	Rende 7-8B praticabili sotto 5 GB di picco con throughput intorno a 30 tok/s.
Large	Q4 con cautela	Fattibile su 16 GB, ma TTFT e throughput indicano uso meno interattivo.

5.13 Profili operativi consigliati

I risultati suggeriscono tre profili d'uso. Il primo è il profilo *interattivo leggero*: modello piccolo Q4, prompt controllato, streaming della risposta e obiettivo di bassa latenza percepita. In questo profilo Llama 3.2 3B Q4 e Phi-3.5 mini Q4 sono le configurazioni più coerenti, perché massimizzano throughput e margine di memoria. Sono adatte a prototipi RAG in cui il corpus recuperato è già informativo e il modello deve soprattutto riorganizzare, sintetizzare e citare il contesto.

Il secondo profilo è *qualità moderata entro budget locale*. Qui la scelta può spostarsi su Mistral 7B Q4: il throughput resta sopra 35 tok/s, il picco di memoria rimane sotto 4.4 GB e i subset automatici non mostrano degradi evidenti. Questa configurazione costa più dei piccoli Q4, ma può essere preferibile quando si vuole una maggiore capacità generativa senza uscire dal vincolo dei 16 GB.

Il terzo profilo è *fattibilità di modelli grandi*. Gemma 3 12B Q4 e Qwen3 14B Q4 mostrano che l'esecuzione locale è possibile, ma non indicano automaticamente una scelta operativa migliore. Il decode intorno a 18 tok/s e il TTFT superiore al secondo rendono questi modelli più adatti a richieste meno frequenti, batch o analisi offline. In un'applicazione web locale, dove Qdrant, FastAPI, Next.js e altri servizi condividono la stessa macchina, il margine residuo deve essere verificato con il sistema completo in esecuzione.

La matrice non sostituisce test end-to-end sul prototipo. Serve a ridurre lo spazio delle opzioni prima della validazione applicativa: i modelli piccoli Q4 sono il punto di partenza ragionevole, Mistral 7B Q4 è il candidato da confrontare quando la qualità delle risposte diventa prioritaria, mentre i modelli large richiedono un caso d'uso più specifico. Nel perimetro della tesi questa distinzione è più utile di una classifica unica, perché il vincolo principale non è il punteggio assoluto su benchmark generali, ma la capacità di fornire risposte utili dentro un sistema locale con risorse condivise.

Tabella 5.14: Matrice decisionale per l'uso dei modelli nel prototipo RAG.

Scenario	Configurazione candidata	Motivazione
Chat RAG interattiva	Small Q4	Miglior margine di memoria, TTFT contenuto e throughput elevato.
Risposte più articolate	Mistral 7B Q4	Compromesso tra capacità generativa, picco sotto 5 GB e risultati automatici stabili.
Analisi offline o batch	Gemma 3 12B Q4	Fattibile localmente, ma meno adatto a interazione frequente per latenza e throughput.
Valutazione da ripetere	Qwen3 8B/14B	Output non sempre compatibile con il parser; serve prima validare prompt e formato di risposta.
Configurazione conservativa	Q6	Utile solo se test RAG successivi mostrano degrado concreto in Q4.

5.14 Implicazioni per un sistema RAG locale

Anche se il benchmark è isolato dal prototipo RAG, i risultati hanno implicazioni dirette per l'architettura descritta nei capitoli precedenti. Un sistema RAG locale deve gestire prompt più lunghi di una chat semplice, perché include contesto recuperato, metadati e istruzioni di groundedness. Le misure per classe di prompt mostrano che l'aumento del contesto incide soprattutto sul TTFT. La scelta del modello non può quindi basarsi solo sul decode token/s: per la percezione utente conta anche quanto tempo passa prima che inizi lo streaming della risposta.

Per un MacBook Pro M1 Pro con 16 GB, i modelli piccoli in Q4 sono i candidati più solidi per un RAG locale interattivo. Llama 3.2 3B Q4 e Phi-3.5 mini Q4 offrono il throughput più elevato tra i piccoli; Gemma 3 4B Q4 e Qwen3 4B Q4 mantengono comunque valori sopra 50 tok/s in decode e picchi di memoria intorno a 2.4–2.6 GB. Queste configurazioni lasciano spazio al retriever, al servizio RAG, al database vettoriale e all'applicazione web.

I modelli medi Q4 sono utili quando si vuole provare una capacità maggiore del modello generativo senza superare il budget hardware. Mistral 7B Q4 è un candidato interessante: 3.80 GiB di artefatto, 4.35 GB di picco, 35.12 tok/s in decode e risultati elevati sui subset. I modelli large Q4 sono invece dimostrazioni di fattibilità più che scelte operative immediate: il margine di memoria e la latenza sono peggiori, e l'uso concorrente con altri servizi potrebbe ridurre la stabilità.

5.15 Limiti dell'analisi

L'analisi ha quattro limiti principali. Il primo è la dimensione dei subset di qualità: 16 esempi per MMLU e 16 per GSM8K sono sufficienti per un controllo leggero, ma non per stimare accuratamente capacità generali. Il secondo è l'assenza di valutazione umana o RAG end-to-end: non vengono misurate groundedness, completezza o allucinazioni nel corpus aziendale. Il terzo è l'hardware singolo: i valori non devono essere trasferiti direttamente a M2 Ultra, M3 Ultra o GPU NVIDIA. Il quarto riguarda il parser automatico: per modelli che producono output ragionativi o formati inattesi, parte del punteggio può riflettere incompatibilità di estrazione più che errore semantico.

Mancano inoltre tre misure utili in una valutazione produttiva: consumo energetico, comportamento termico su run lunghi e prestazioni con servizi concorrenti attivi. Il benchmark isola il modello generativo, mentre un RAG reale usa anche retriever, database vettoriale, API server, frontend, logging e possibili processi di ingestione. I risultati indicano quali modelli sono candidati realistici; non garantiscono che la stessa latenza rimanga invariata durante l'uso end-to-end del sistema completo.

Questi limiti non cambiano la lettura complessiva. Nel perimetro misurato, la quantizzazione MLX rende praticabili piu' classi di modelli su Apple Silicon consumer e, sui modelli piccoli, migliora efficienza e throughput senza perdita qualitativa osservabile nei controlli automatici ridotti.

Conclusioni e Sviluppi Futuri

Questa tesi ha affrontato la progettazione, l'implementazione e la valutazione di un sistema RAG aziendale integrato in i3Guild. Il punto di partenza e' stato un problema concreto: trasformare il corpus delle Gilde di Intré in una base di conoscenza interrogabile tramite linguaggio naturale, mantenendo i dati entro un perimetro controllato e usando risorse hardware realistiche. Il lavoro non ha proposto un nuovo modello linguistico, ma ha studiato come combinare componenti esistenti in un prototipo on-premise coerente con vincoli di privacy, manutenibilita' e sostenibilita' operativa.

Il primo risultato e' architetturale. La tesi ha mostrato che il corpus i3Guild puo' essere trasformato in una knowledge base RAG attraverso una pipeline separata dall'applicazione principale: PostgreSQL rimane la sorgente autoritativa, la pipeline di ingestione normalizza i dati e produce documenti, Qdrant conserva rappresentazioni dense, sparse e metadati, il microservizio FastAPI espone retrieval e generazione, mentre Next.js integra il servizio nel flusso applicativo. Questa separazione consente di mantenere chiari i confini tra dati applicativi, indice vettoriale e generazione LLM. Inoltre, rende possibile aggiornare o sostituire singoli componenti senza riscrivere l'intero sistema.

Il secondo risultato riguarda la logica di retrieval. Nel caso delle Gilde, una ricerca puramente vettoriale non e' sufficiente a sfruttare tutti i segnali del dominio. Il prototipo integra query rewriting, filtri strutturati, deduplicazione, cache e selezione dinamica dei documenti. Queste scelte riducono il rumore del contesto passato al modello e rendono la risposta piu' aderente al corpus recuperato. La generazione aumentata da retrieval non viene quindi trattata come una semplice concatenazione di documenti e prompt, ma come una pipeline in cui normalizzazione, ranking, soglie e metadati influenzano direttamente la qualita' dell'output.

Il terzo risultato riguarda la scelta tra approccio RAG classico e componente agentica. Durante il progetto e' stato implementato un workflow agentic per guidare la proposta di nuove Gilde. L'esperimento ha chiarito che tale approccio puo' essere utile per organizzare passaggi complessi, ma nel perimetro hardware disponibile introduce piu' chiamate al modello, maggiore latenza e maggiore complessita' di controllo. La soluzione finale privilegia quindi un RAG piu' deterministico, con retrieval e generazione separati e con una generazione strutturata delle bozze gestita in modo controllato. La componente agentica rimane una linea di sviluppo, non il nucleo del sistema realizzato.

Il quarto risultato e' sperimentale. La valutazione dell'inferenza locale su Apple Silicon ha permesso di misurare il ruolo della quantizzazione MLX nel rendere praticabili modelli open-weight su un MacBook Pro M1 Pro con 16 GB di memoria unificata. Nei modelli piccoli, la variante Q4 e' risultata il miglior compromesso osservato: riduce la dimensione degli artefatti di circa il 72%, riduce il picco di memoria di circa 67-70%, aumenta il throughput di decodifica di circa 2.6-2.8 volte rispetto a FP16 e non mostra un degrado macroscopico sui subset automatici MMLU e GSM8K.

Nei modelli medi e grandi, la quantizzazione opera soprattutto come leva di fattibilit : consente di eseguire modelli tra 7B e 14B parametri, ma con un costo maggiore in latenza, TTFT e margine operativo.

Nel complesso, i risultati confermano l'ipotesi di fondo della tesi: un sistema RAG locale per un caso d'uso aziendale   possibile, ma richiede controllo simultaneo di pi  livelli. Non basta scegliere un modello linguistico; occorre curare il corpus, progettare l'indice, definire la pipeline di retrieval, stabilire formati di output compatibili con l'applicazione e verificare che l'inferenza sia sostenibile sulla macchina scelta. La parte sperimentale mostra anche che le metriche non sono intercambiabili: throughput, TTFT, memoria, dimensione su disco e qualit  automatica raccontano aspetti diversi dello stesso problema.

Limiti

Il lavoro presenta alcuni limiti. Il primo riguarda la valutazione applicativa del RAG. Il prototipo   stato progettato e implementato, ma manca ancora una ground truth estesa, costruita con stakeholder aziendali, per misurare in modo sistematico groundedness, completezza, utilit  della risposta e presenza di allucinazioni sul corpus reale delle Gilde. Le metriche automatiche usate nella parte MLX sono utili per controllare segnali di qualit  generale, ma non sostituiscono una valutazione end-to-end del caso d'uso i3Guild.

Il secondo limite riguarda il corpus. I dati delle Gilde sono eterogenei: descrizioni, risultati, metadati, partecipanti, epoch e contenuti editoriali non sempre hanno la stessa granularit  o lo stesso livello di dettaglio. La normalizzazione testuale riduce parte del problema, ma non elimina la necessit  di un glossario aziendale, di convenzioni pi  stabili e di una validazione umana dei contenuti pi  rilevanti.

Il terzo limite riguarda l'inferenza locale. Gli esperimenti sono stati eseguiti su una sola macchina, un MacBook Pro M1 Pro con 16 GB di memoria unificata, e usano MLX come runtime principale. I risultati non vanno generalizzati direttamente ad altri Mac, a GPU NVIDIA, a runtime diversi o a regimi di contesto molto lunghi. Inoltre, i subset MMLU e GSM8K sono volutamente ridotti: servono a individuare degradazioni macroscopiche e problemi di formato, non a stabilire classifiche generali dei modelli.

Sviluppi futuri

Uno sviluppo prioritario riguarda la valutazione del sistema RAG sul dominio i3Guild. Il passo successivo consiste nella costruzione di un dataset di domande reali o realistiche, associate a risposte attese e documenti di riferimento. Tale dataset dovrebbe essere validato con persone interne all'azienda, in modo da distinguere errori di retrieval, errori di generazione, incompletezza del corpus e ambiguit  della domanda. Su questa base sarebbe possibile confrontare versioni diverse della pipeline e misurare il contributo effettivo di query rewriting, filtri, soglie dinamiche e deduplicazione.

Un secondo sviluppo riguarda l'evoluzione della base di conoscenza. La strategia attuale di full re-index periodico   semplice e adatta al prototipo, ma pu  essere sostituita da una sincronizzazione incrementale basata sui campi di aggiornamento del database o da un meccanismo event-driven collegato alle operazioni CRUD di i3Guild. Questa evoluzione ridurrebbe il costo di indicizzazione e renderebbe pi  tempestiva la disponibilit  dei nuovi contenuti nel sistema RAG.

Un terzo sviluppo riguarda la qualità del contesto recuperato. L'introduzione di un glossario aziendale, di sinonimi controllati, di tag più espliciti e di regole di normalizzazione condivise potrebbe migliorare il retrieval nei casi in cui il linguaggio usato dagli utenti non coincide con quello presente nei documenti. In prospettiva, si potrebbero sperimentare reranker, strategie di query expansion più mirate e valutazioni separate per retrieval denso, sparso e ibrido.

Un quarto sviluppo riguarda i modelli generativi locali. I risultati su MLX indicano che i modelli piccoli in Q4 sono candidati solidi per un uso interattivo, mentre i modelli medi possono essere interessanti quando serve una maggiore capacità generativa. Una valutazione futura dovrebbe quindi misurare questi modelli direttamente nel flusso RAG completo, includendo servizi concorrenti, prompt con contesto reale, consumo energetico, comportamento termico e latenza percepita dall'utente. Solo questa verifica può stabilire se il miglior compromesso sperimentale rimane valido anche nel sistema end-to-end.

Infine, la componente agentica può essere ripresa dopo aver consolidato il RAG di base. Un agente potrebbe supportare la proposta di nuove Gilde, guidare la raccolta di requisiti, suggerire collegamenti con iniziative passate o produrre bozze strutturate più complete. Tuttavia, l'adozione di un workflow agentico dovrebbe essere subordinata a criteri misurabili: riduzione del carico per l'utente, qualità superiore delle bozze, controllo dei costi computazionali e robustezza del formato prodotto. In assenza di questi criteri, la maggiore complessità rischierebbe di peggiorare un sistema che, nella forma RAG classica, risulta già più controllabile e adatto al contesto aziendale.

Il lavoro svolto lascia quindi un prototipo funzionante e un quadro sperimentale per valutarne la sostenibilità. La direzione più utile non è aumentare la complessità del modello o dell'orchestrazione senza una misura del beneficio, ma rafforzare il legame tra corpus, retrieval, valutazione e vincoli reali dell'infrastruttura locale.

Appendice A

Annotazioni e Convenzioni

Questa appendice raccoglie le convenzioni usate nella descrizione del sistema RAG. L'obiettivo è evitare ripetizioni nei capitoli principali e rendere esplicito il significato di nomi, formati e parametri ricorrenti.

A.1 Repository e componenti

Tabella A.1: Convenzioni di denominazione dei repository.

Nome	Significato
i3guild	Applicazione web principale, basata su Next.js, PostgreSQL e Keycloak.
i3guild-embedding-pipeline	Job Python che estrae, normalizza e indicizza il corpus delle Gilde.
i3guild-rag	Microservizio FastAPI che espone retrieval ibrido, chat RAG e generazione controllata.

A.2 Formato dei documenti indicizzati

I documenti salvati in Qdrant derivano dai record applicativi di i3Guild. Ogni documento contiene un testo normalizzato e un insieme di metadati usati per filtri, diagnostica e deduplicazione.

A.3 Eventi NDJSON

Gli endpoint streaming restituiscono una sequenza di righe JSON indipendenti. La scelta del formato NDJSON permette al frontend di elaborare subito i metadati RAG e poi aggiornare l'interfaccia man mano che arrivano i token.

Tabella A.2: Metadati principali associati ai documenti indicizzati.

Campo	Uso
<code>guild_id</code>	Identificativo della Gilda sorgente.
<code>guild_name</code>	Nome della Gilda, usato anche nei risultati mostrati all'utente.
<code>epoch_id</code>	Identificativo dell'epoca a cui appartiene la Gilda.
<code>participants</code>	Lista dei partecipanti, utile come metadato filtrabile.
<code>participation_mode</code>	Modalità di partecipazione, ad esempio in presenza, remota o ibrida.
<code>is_individual</code>	Indica se la proposta riguarda una Gilda individuale.
<code>chunk_index</code>	Posizione del chunk quando una Gilda viene suddivisa.
<code>chunk_count</code>	Numero totale di chunk generati per la stessa Gilda.

Tabella A.3: Tipi di evento usati dagli endpoint streaming.

Evento	Significato
<code>rag</code>	Contiene documenti recuperati, filtri, query riscritta e motivo del taglio dei risultati.
<code>status</code>	Indica l'avanzamento di una generazione asincrona lato applicazione.
<code>stream_start</code>	Segnala l'inizio della risposta generata.
<code>token</code>	Contiene un frammento incrementale della risposta.
<code>stream_end</code>	Chiude lo stream dopo l'ultimo token generato.
<code>error</code>	Comunica un errore gestibile senza interrompere il contratto dello stream.

Appendice B

Codice Utilizzato

Questa appendice raccoglie i listati richiamati nei capitoli principali. Nel testo sono mantenute le scelte progettuali, le decisioni implementative e il razionale; i frammenti di codice sono spostati qui per non interrompere la lettura.

B.1 Listati richiamati nel Capitolo 2

Listing B.1: Struttura della pipeline di ingestione Haystack 2.x (pseudocodice).

```
from haystack import Pipeline
from haystack.components.writers import DocumentWriter
from haystack.integrations.components.embedders.fastembed import (
    FastembedDocumentEmbedder,
    FastembedSparseDocumentEmbedder,
)
from haystack.integrations.document_stores.qdrant import QdrantDocumentStore

qdrant_store = QdrantDocumentStore(url="http://qdrant:6333",
                                   index="i3guild_knowledge",
                                   embedding_dim=768,
                                   use_sparse_embeddings=True)

indexing_pipeline = Pipeline()
indexing_pipeline.add_component("dense_embedder",
                                FastembedDocumentEmbedder(
                                    model="sentence-transformers/paraphrase-multilingual-mpnet-base-v2"
                                ))
indexing_pipeline.add_component("sparse_embedder",
                                FastembedSparseDocumentEmbedder(
                                    model="Qdrant/bm25"
                                ))
indexing_pipeline.add_component("writer", DocumentWriter(
```

```
document_store=qdrant_store ,
policy=DuplicatePolicy.OVERWRITE))

indexing_pipeline.connect("dense_embedder.documents", "sparse_embedder.
documents")
indexing_pipeline.connect("sparse_embedder.documents", "writer.
documents")

indexing_pipeline.run({"dense_embedder": {"documents": raw_docs}})
```

Listing B.2: Esempio di request body per l'endpoint `/rag/retrieve`.

```
{
  "query":          "vorrei fondare una gilda sulla musica",
  "top_k":          20,
  "score_threshold": 0.5,
  "max_documents":  10,
  "gap_threshold":  0.08,
  "filters":        null
}
```

Listing B.3: Struttura della response dell'endpoint `/rag/retrieve`.

```
{
  "documents":      [ { "content": "...", "score": 0.85,
                        "meta": { "guild_name": "...",
                                  "participants": [...] } } ],
  "total_candidates": 15,
  "threshold_used":  0.5,
  "cutoff_reason":   "relative_gap",
  "topic_query":     "musica"
}
```

B.2 Listati richiamati nel Capitolo 3: pipeline di ingestione

Listing B.4: Campi principali estratti da PostgreSQL per l'ingestione.

```
SELECT
  g.id AS guild_id ,
  g.name AS guild_name ,
  g.summary ,
  g.goals ,
  g.opportunities ,
  g.outcome ,
  g.risks ,
```

```

    g."otherResources" AS other_resources ,
    g.budget ,
    g."achievedResultsDescription" AS achieved_results_description ,
    g.journal ,
    g."isIndividualGuild" AS is_individual ,
    g."participationMode" AS participation_mode ,
    e.id AS epoch_id ,
    e."startDate" AS epoch_start ,
    e."endDate" AS epoch_end
FROM "Guild" g
LEFT JOIN "Epoch" e ON g."epochId" = e.id

```

Listing B.5: Schema semplificato della costruzione del documento testuale.

```

parts = [f"[GUILD:-{guild_name}]"]

if row.get("summary"):
    parts.append(f"[SOMMARIO] - {clean_html(row.get('summary'))}")
if row.get("goals"):
    parts.append(f"[OBIETTIVI] - {clean_html(row.get('goals'))}")
if row.get("risks"):
    parts.append(f"[RISCHI] - {clean_html(row.get('risks'))}")
if participants:
    parts.append(f"[PARTECIPANTI] - {', - '.join(participants)}")

full_text = "\n".join(parts)

```

Listing B.6: Pipeline Haystack usata per embedding e scrittura.

```

document_store = QdrantDocumentStore(
    url=QDRANT_URL,
    index=QDRANT_COLLECTION_NAME,
    embedding_dim=EMBEDDING_DIMENSIONS,
    similarity="cosine",
    recreate_index=RECREATE_INDEX,
    use_sparse_embeddings=True,
)

dense_embedder = FastembedDocumentEmbedder(
    model=FASTEMBED_DENSE_MODEL,
    cache_dir=FASTEMBED_CACHE_DIR,
)

sparse_embedder = FastembedSparseDocumentEmbedder(
    model=FASTEMBED_SPARSE_MODEL,
    cache_dir=FASTEMBED_CACHE_DIR,
)

```

```
writer = DocumentWriter(document_store=document_store, policy=policy)
indexing_pipeline = Pipeline()
indexing_pipeline.add_component("dense_embedder", dense_embedder)
indexing_pipeline.add_component("sparse_embedder", sparse_embedder)
indexing_pipeline.add_component("writer", writer)
indexing_pipeline.connect("dense_embedder.documents", "sparse_embedder.
documents")
indexing_pipeline.connect("sparse_embedder.documents", "writer.
documents")
```

B.3 Listati richiamati nel Capitolo 3: retrieval e generazione

Listing B.7: Schema Pydantic della richiesta di retrieval.

```
class RetrieveRequest(BaseModel):
    query: str
    top_k: int = 20
    score_threshold: float = 0.7
    max_documents: int = 10
    gap_threshold: float = 0.08
    filters: dict | None = None
```

Listing B.8: Pipeline Haystack per la ricerca ibrida.

```
document_store = QdrantDocumentStore(
    url=QDRANT_URL,
    index=QDRANT_COLLECTION_NAME,
    embedding_dim=768,
    similarity="cosine",
    recreate_index=False,
    use_sparse_embeddings=True,
)
text_embedder = FastembedTextEmbedder(
    model=FASTEMBED_DENSE_MODEL,
)
sparse_text_embedder = FastembedSparseTextEmbedder(
    model=FASTEMBED_SPARSE_MODEL,
)
retriever = QdrantHybridRetriever(document_store=document_store)

pipeline = Pipeline()
pipeline.add_component("text_embedder", text_embedder)
pipeline.add_component("sparse_text_embedder", sparse_text_embedder)
pipeline.add_component("retriever", retriever)
pipeline.connect("text_embedder.embedding", "retriever.query_embedding")
```

```
pipeline.connect("sparse_text_embedder.sparse_embedding",
                 "retriever.query_sparse_embedding")
```

Listing B.9: Logica semplificata di taglio dinamico dei risultati.

```
scored_docs.sort(key=lambda x: x[0], reverse=True)
above_threshold = [
    d for score_value, d in scored_docs
    if score_value >= score_threshold
]

filtered = [above_threshold[0]]
for i in range(1, len(above_threshold)):
    prev = float(getattr(above_threshold[i - 1], "score", 0.0) or 0.0)
    cur = float(getattr(above_threshold[i], "score", 0.0) or 0.0)
    relative_drop = (prev - cur) / prev if prev > 0 else 1.0
    if relative_drop > gap_threshold:
        cutoff_reason = "relative-gap"
        break
    filtered.append(above_threshold[i])
```

Listing B.10: Formato del messaggio utente aumentato con contesto RAG.

```
final_message = (
    f"{message}\n\n"
    "____-CONTESTO-RAG-____\n"
    "Usa queste informazioni per rispondere.\n"
    f"{final_context}\n"
    "____-FINE-CONTESTO-____"
)
```

Listing B.11: Esempio di eventi NDJSON prodotti dalla chat standard.

```
{"type": "rag", "data": {"documents": [...], "cutoff_reason": "
    relative-gap"}}
{"type": "stream_start", "model": "qwen2.5:1.5b"}
{"type": "token", "content": "La"}
{"type": "token", "content": " gilda"}
{"type": "stream_end"}
```

B.4 Listati richiamati nel Capitolo 3: frontend e generazione bozza

Listing B.12: Esempio di eventi NDJSON prodotti dalla generazione bozza.

```
{ "type": "status", "message": "generation_started", "promptLength":  
  4200 }  
{ "type": "status", "message": "generation_in_progress" }  
{ "type": "draft", "source": "llm", "draft": { "name": "..." } }
```

Listing B.13: Chiamata TypeScript per generazione bozza senza retrieval.

```
const ragResponse = await generateRAGChatReply(prompt, [], {  
  model,  
  useRag: false,  
  jsonMode: true,  
  options: { num_predict: -1, timeout_seconds: 0 },  
});
```

Listing B.14: Chiamata TypeScript al retrieval RAG.

```
const response = await fetch(`${RAG_API_URL}/rag/retrieve`, {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({  
    query,  
    filters,  
    top_k: 20,  
    score_threshold: options?.scoreThreshold ?? 0.7,  
    max_documents: options?.maxDocuments ?? 10,  
  }),  
});
```

Bibliografia

Riferimenti Bibliografici

- [1] Meftun Akarsu, Recep Kaan Karaman e Christopher Mierbach. “From BM25 to Corrective RAG: Benchmarking Retrieval Strategies for Text-and-Table Documents”. In: *arXiv preprint arXiv:2604.01733* (2026). URL: <https://arxiv.org/abs/2604.01733>.
- [2] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *arXiv preprint arXiv:2310.11511* (2023). URL: <https://arxiv.org/abs/2310.11511>.
- [3] Yuntao Bai et al. “Constitutional AI: Harmlessness from AI Feedback”. In: *arXiv preprint arXiv:2212.08073* (2022). URL: <https://arxiv.org/abs/2212.08073>.
- [4] Devi Prasad Bal e Subhashree Puhan. “Benchmarking Retrieval Strategies for Biomedical Retrieval-Augmented Generation: A Controlled Empirical Study”. In: *arXiv preprint arXiv:2605.02520* (2026). URL: <https://arxiv.org/abs/2605.02520>.
- [5] Afsara Benazir. “Benchmarking and Characterization of Large Language Model Inference on Apple Silicon”. In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems* (2025). Articolo peer-reviewed.
- [6] Afsara Benazir e Felix Xiaozhu Lin. “Profiling Large Language Model Inference on Apple Silicon: A Quantization Perspective”. In: *arXiv preprint arXiv:2508.08531* (2025). URL: <https://arxiv.org/abs/2508.08531>.
- [7] Rishi Bommasani et al. “On the Opportunities and Risks of Foundation Models”. In: *arXiv preprint arXiv:2108.07258* (2021). URL: <https://arxiv.org/abs/2108.07258>.
- [8] Tom Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901. URL: <https://arxiv.org/abs/2005.14165>.
- [9] Chi-Min Chan, Xinyu Chen, Rui Yin et al. “RQ-RAG: Learning to Rewrite Queries for Retrieval Augmented Generation”. In: *arXiv preprint* (2024).
- [10] Jiawei Chen et al. “Benchmarking Large Language Models in Retrieval-Augmented Generation”. In: *arXiv preprint arXiv:2309.01431* (2023). URL: <https://arxiv.org/abs/2309.01431>.
- [11] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: <https://arxiv.org/abs/2305.14314>.

- [12] Darren Edge et al. “From Local to Global: A Graph RAG Approach to Query-Focused Summarization”. In: *arXiv preprint arXiv:2404.16130* (2024). URL: <https://arxiv.org/abs/2404.16130>.
- [13] European Parliament e Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council — General Data Protection Regulation*. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [14] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)*. Rapp. tecn. L 119. Official Journal of the European Union, 2016, pp. 1–88. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32016R0679>.
- [15] Yunfan Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997* (2024). URL: <https://arxiv.org/abs/2312.10997>.
- [16] Sepp Hochreiter e Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [17] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: (2022). URL: <https://arxiv.org/abs/2106.09685>.
- [18] Intré S.r.l. *10 pratiche per costruire una Learning Organization: il modello Intré*. <https://www.intre.it/2024/01/15/10-pratiche-per-costruire-una-learning-organization-il-modello-intre/>. Ultimo accesso: maggio 2026. 2024.
- [19] Intré S.r.l. *Archivio Gilde*. <https://www.intre.it/gilde/>. Ultimo accesso: aprile 2026. 2024.
- [20] Abdurrahman Javat e Allan Kazakov. “Silicon Showdown: Performance, Efficiency, and Ecosystem Barriers in Consumer-Grade LLM Inference”. In: *arXiv preprint arXiv:2605.00519* (2026). URL: <https://arxiv.org/abs/2605.00519>.
- [21] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38. DOI: 10.1145/3571730.
- [22] Albert Q. Jiang et al. “Mistral 7B”. In: *arXiv preprint arXiv:2310.06825* (2023). URL: <https://arxiv.org/abs/2310.06825>.
- [23] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [24] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [25] Shih-Yang Liu et al. “DoRA: Weight-Decomposed Low-Rank Adaptation”. In: *International Conference on Machine Learning (ICML)*. 2024. URL: <https://arxiv.org/abs/2402.09353>.
- [26] Yury A. Malkov e Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.

- [27] Qwen Team e Alibaba Cloud. *Qwen2 Technical Report*. 2024. URL: <https://arxiv.org/abs/2407.10671>.
- [28] Varun Rajesh et al. “Production-Grade Local LLM Inference on Apple Silicon: A Comparative Study of MLX, MLC-LLM, Ollama, llama.cpp, and PyTorch MPS”. In: *arXiv preprint arXiv:2511.05502* (2025). Technical report non peer-reviewed. URL: <https://arxiv.org/abs/2511.05502>.
- [29] Peter M. Senge. *The Fifth Discipline: The Art and Practice of the Learning Organization*. Edizione italiana: *La quinta disciplina*, Sperling & Kupfer, 1990. Doubleday/Currency, 1990.
- [30] Kurt Shuster et al. “Retrieval Augmentation Reduces Hallucination in Conversation”. In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 3784–3803. URL: <https://arxiv.org/abs/2104.07567>.
- [31] Aparajita Sinha e Kunal Chakma. “NITATREC at TREC 2025: A Report on Exploring Sparse, Dense, and Hybrid Retrieval for Retrieval-Augmented Generation”. In: *Proceedings of the 34th Text REtrieval Conference (TREC 2025)*. Gaithersburg, Maryland, 2025. URL: <https://pages.nist.gov/trec-browser/trec34/rag/proceedings/>.
- [32] Hugo Touvron et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971* (2023). URL: <https://arxiv.org/abs/2302.13971>.
- [33] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [34] Sai Vegasena. “Open-TQ-Metal: Fused Compressed-Domain Attention for Long-Context LLM Inference on Apple Silicon”. In: *arXiv preprint arXiv:2604.16957* (2026). URL: <https://arxiv.org/abs/2604.16957>.
- [35] Wenxiao Wang et al. “Model Compression and Efficient Inference for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2402.09748* (2024). URL: <https://arxiv.org/abs/2402.09748>.
- [36] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837. URL: <https://arxiv.org/abs/2201.11903>.
- [37] Shi-Qi Yan et al. “Corrective Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2401.15884* (2024). URL: <https://arxiv.org/abs/2401.15884>.
- [38] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/abs/2210.03629>.
- [39] Shenglai Zeng, Jiankun Zhang, Pengfei He et al. “The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)”. In: *arXiv preprint arXiv:2402.16893* (2024). URL: <https://arxiv.org/abs/2402.16893>.

Sitografia

- [40] AIMultiple Research. *RAG Frameworks: LangChain vs LangGraph vs LlamaIndex*. Benchmark comparativo di 5 framework RAG su 100 query standardizzate. 2026. URL: <https://research.aimultiple.com/rag-frameworks/>.

- [41] Anthropic. *Models Overview*. 2026. URL: <https://docs.anthropic.com/en/docs/about-claude/models/overview>.
- [42] Apple. *Introducing M1 Pro and M1 Max: The Most Powerful Chips Apple Has Ever Built*. 2021. URL: <https://www.apple.com/newsroom/2021/10/introducing-m1-pro-and-m1-max-the-most-powerful-chips-apple-has-ever-built/>.
- [43] Apple. *MacBook Pro (16-inch, 2021) – Technical Specifications*. 2021. URL: <https://support.apple.com/kb/SP858>.
- [44] Apple. *MLX: An Array Framework for Apple Silicon*. 2026. URL: <https://opensource.apple.com/projects/mlx/>.
- [45] Apple Developer Documentation. *Choosing a Resource Storage Mode for Apple GPUs*. 2026. URL: <https://developer.apple.com/documentation/metal/choosing-a-resource-storage-mode-for-apple-gpus>.
- [46] Athenic AI. *LangChain vs LlamaIndex vs Haystack: RAG Framework Comparison*. 2025. URL: <https://getathenic.com/blog/langchain-vs-llamaindex-vs-haystack-rag-frameworks>.
- [47] deepset. *FastembedDocumentEmbedder*. Ultimo accesso: 25 maggio 2026. 2026. URL: <https://docs.haystack.deepset.ai/docs/fastembeddocumentembedder>.
- [48] deepset. *FastembedSparseDocumentEmbedder*. Ultimo accesso: 25 maggio 2026. 2026. URL: <https://docs.haystack.deepset.ai/docs/fastembedsparsedocumentembedder>.
- [49] deepset. *QdrantHybridRetriever*. Ultimo accesso: 25 maggio 2026. 2026. URL: <https://docs.haystack.deepset.ai/docs/qdranthybridretriever>.
- [50] deepset GmbH. *Haystack: Open Source NLP Framework to Build Production-Ready LLM Applications*. 2023. URL: <https://haystack.deepset.ai>.
- [51] Docker Inc. *Docker Documentation*. 2024. URL: <https://docs.docker.com/>.
- [52] FastAPI Contributors. *FastAPI Documentation*. 2024. URL: <https://fastapi.tiangolo.com/>.
- [53] Google AI for Developers. *Gemini API Models*. 2026. URL: <https://ai.google.dev/gemini-api/docs/models>.
- [54] Google AI for Developers. *Gemma Models*. 2026. URL: <https://ai.google.dev/gemma/docs/core>.
- [55] Nirant Kasliwal. *pgvector vs Qdrant: Results from the 1M OpenAI Benchmark*. 2024. URL: <https://nirantk.com/writing/pgvector-vs-qdrant/>.
- [56] LangChain AI. *LangGraph: Build Resilient Language Agents as Graphs*. 2024. URL: <https://langchain-ai.github.io/langgraph/>.
- [57] MLC LLM Contributors. *MLC LLM Documentation*. 2025. URL: https://llm.mlc.ai/docs/get_started/introduction.html.
- [58] MLX Contributors. *mlx-lm: LLMs on Apple Silicon with MLX*. 2026. URL: <https://github.com/ml-explore/mlx-lm>.
- [59] Ollama Contributors. *Ollama: Get Up and Running With Large Language Models Locally*. 2023. URL: <https://ollama.com>.

- [60] OpenAI. *Learning to Reason with LLMs*. 2024. URL: <https://openai.com/index/learning-to-reason-with-llms/>.
- [61] OpenAI. *Models*. 2026. URL: <https://platform.openai.com/docs/models>.
- [62] Pelican. *Haystack vs LangChain: Explained*. 2025. URL: <https://pelican.io/blog/haystack-vs-langchain/>.
- [63] Ethan Perez. *RAG Evaluation Series: Validating the RAG Performance of LangChain vs Haystack*. Tonic.ai Blog. 2024. URL: <https://www.tonic.ai/blog/rag-evaluation-series-validating-the-rag-performance-of-langchain-vs-haystack>.
- [64] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/>.
- [65] Pydantic Contributors. *Pydantic Documentation*. 2024. URL: <https://docs.pydantic.dev/>.
- [66] Qdrant Contributors. *Hybrid and Multi-Stage Queries*. 2026. URL: <https://qdrant.tech/documentation/search/hybrid-queries/>.
- [67] Qdrant Contributors. *Indexing*. 2026. URL: <https://qdrant.tech/documentation/manage-data/indexing/>.
- [68] Qdrant Contributors. *Qdrant: Vector Database for the Next Generation of AI Applications*. 2023. URL: <https://qdrant.tech>.
- [69] Qdrant Team. *Vector Search Benchmarks*. Benchmark ufficiale Qdrant su dataset ann-benchmarks; aggiornato gennaio/giugno 2024. 2024. URL: <https://qdrant.tech/benchmarks/>.
- [70] Qwen Team e Alibaba Cloud. *Qwen3 Technical Report*. 2025. URL: <https://qwenlm.github.io/blog/qwen3/>.
- [71] Leonard Richardson. *Beautiful Soup Documentation*. 2024. URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.
- [72] Tiger Data. *Pgvector vs. Qdrant: Benchmarking Open-Source Vector Databases for AI at Scale*. Benchmark su 50M embedding Cohere a 768 dimensioni con fork ANN-Benchmarks. 2025. URL: <https://www.tigerdata.com/blog/pgvector-vs-qdrant>.
- [73] Vercel. *Next.js Documentation*. 2024. URL: <https://nextjs.org/docs>.